

FROM LEGACY TO MODERN: A FRAMEWORK FOR DIAGNOSING AND
MODERNIZING DATA PIPELINES FOR THE DATA-DRIVEN WORKFORCE

BY

LISETTE M. KAMPER-HINSON

A Thesis Submitted to the Graduate Faculty of

WAKE FOREST UNIVERSITY GRADUATE SCHOOL OF ARTS AND SCIENCES

in Partial Fulfillment of the Requirements

for the Degree of

MASTER OF SCIENCE

Computer Science

April 2026

Winston-Salem, North Carolina

Approved By:

William H. Turkett Ph.D. , Advisor

Ron P. Doyle, Ph.D. , Chair

Jeffrey D. Camm, Ph.D.

TABLE OF CONTENTS

LIST OF FIGURES	iii
LIST OF ABBREVIATIONS	iii
ABSTRACT	iv
INTRODUCTION	v
CHAPTER 1: THEORETICAL FOUNDATIONS OF PIPELINE MODERNIZATION	1
CHAPTER 2: RESEARCH DESIGN	6
CHAPTER 3: LEGACY PIPELINE	13
CHAPTER 4: ITERATIVE MODERNIZATION AND FRAMEWORK DEVELOPMENT	18
CHAPTER 5: THE DATA PIPELINE MODERNIZATION FRAMEWORK	23
STEP 1: MEET WITH THE DOMAIN EXPERT	23
STEP 2: EXPLICIT STRUCTURAL AUDIT AND VISUAL DECOMPOSITION	27
Stage-Level Audit Checklist (Required Artifact)	29
STEP 3: FUNCTIONAL EVALUATION AND MODERNIZATION TARGET IDENTIFICATION	32
Step 3.1: Confirm Modernization Categories	33
Step 3.2: Evaluate Components Along Decision-Relevant Dimensions	33
Step 3.3: Conduct Relative Impact–Effort Reasoning	34
Step 3.4: Produce a Preliminary Modernization Roadmap.....	35
Step 3.5: Validate the Modernization Direction Through a Second Domain Expert Review	36
STEP 4: MODELING AND MACHINE LEARNING CONSIDERATIONS	37
4.1 Systematic Consideration of Plausible Modeling Techniques.....	38
4.2 Data Audit and Modeling Consideration	39
4.3 Explicitly Evaluate Interpretability, Technical Debt, and Long-Term Maintainability	41
4.4 Full-Scale Application and Reassessment on the Complete Dataset.....	45
STEP 5: VISUALIZATION AND COMMUNICATION DESIGN	47
5.1 Anchor Each Visualization to a Specific Decision Context	48
5.2 Preserve Operational Knowledge Through Visual Continuity	50
5.3 Visually Distinguish Descriptive, Forecasted, and Modeled Outputs	51
5.4 Reflect Model Behavior and Validity Constraints in Visual Design.....	53
5.5 Communicate Data Provenance, Assumptions, and Update Behavior	55

5.6 Enforce Separation Between Exploratory and Decision-Facing Visuals	56
STEP 6: STANDARDIZE IMPLEMENTATION, REPRODUCIBILITY, AND SYSTEM-LEVEL VALIDATION.....	60
The Data Pipeline Modernization Framework Diagram.....	65
<i>CHAPTER 6: APPLICATION AND TECHNICAL SYNTHESIS OF THE MANUFACTURING ANALYTICAL PIPELINE.....</i>	<i>66</i>
<i>CONCLUSION</i>	<i>80</i>
CONTRIBUTIONS.....	81
FUTURE WORK	82
<i>REFERENCES</i>	<i>85</i>
<i>APPENDIX A: PROPOSED FRAMEWORK PROCEDURE SCORE SHEET</i>	<i>88</i>
<i>APPENDIX B: Operations Manager User Manual.....</i>	<i>157</i>
<i>CURRICULUM VITAE.....</i>	<i>175</i>

LIST OF FIGURES

- Figure 1 N-Gram Frequency of "Data Science" and its variances from 1950-Present
Figure 2 N-Gram Frequency of "Computer Science" and its variances from 1950-Present
Figure 3 Visual Overview of Proposed Modernization Process
Figure 4 Process Metrics Used in Framework Evaluation
Figure 5 Current Pipeline Diagram
Figure 6 Legacy Pipeline Architecture Diagram
Figure 7 Modernized Pipeline Architecture Diagram

LIST OF ABBREVIATIONS

API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
CFR	Capacity Fill Rate
CRISP-DM	Cross-Industry Standard Process for Data Mining
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DSR	Design Science Research
GLM	Generalized Linear Model
GQM	Goal-Question-Metric
ML	Machine Learning
MLOps	Machine Learning Operations
SQL	Structured Query Language

ABSTRACT

Modernization of operational data pipelines frequently emphasizes analytical sophistication while neglecting structural governance, reproducibility, and decision usability. In manufacturing environments, this imbalance can exacerbate misalignment between reported and realized production capacity. This thesis presents a structured framework for modernizing legacy pipelines while preserving embedded domain knowledge and minimizing technical debt. The framework is constructed through iterative design, implementation, and evaluation of candidate modernization steps within a real operational pipeline. The framework formalizes six stages: domain alignment, structural audit, prioritized modernization, governed modeling introduction, decision-aligned visualization design, and reproducible implementation with system-level validation. The approach is demonstrated through a single-plant capacity modeling case study using read-only operational data, integrating forecasting, uncertainty quantification, and decision optimization within a unified analytical system. Evaluation focuses on structural and methodological improvements rather than predictive performance alone. Results indicate that modernization guided by explicit structural and governance constraints improves maintainability, interpretability, and scalability without increasing system fragility. These findings suggest that effective pipeline modernization requires treating modeling, visualization, and implementation standards as integrated system design decisions rather than isolated technical upgrades.

INTRODUCTION

Data Science, as a formalized discipline, is relatively young. Although John Tukey (1962) argued for an expansion of statistics toward what he described as “the unrecognized science of learning from data,” the term *Data Science* did not gain widespread academic and industrial recognition until the early 2000s, following Cleveland’s (2001) call to formalize it as an applied field bridging statistics and computing. The rapid acceleration of the term in recent decades reflects both the field’s growth and its expanding influence across medicine via predictive diagnostics and personalized treatment strategies, business through data-driven decision-making and optimization, public policy through evidence-based governance and population-level modeling, and manufacturing through forecasting, process optimization, and capacity planning.

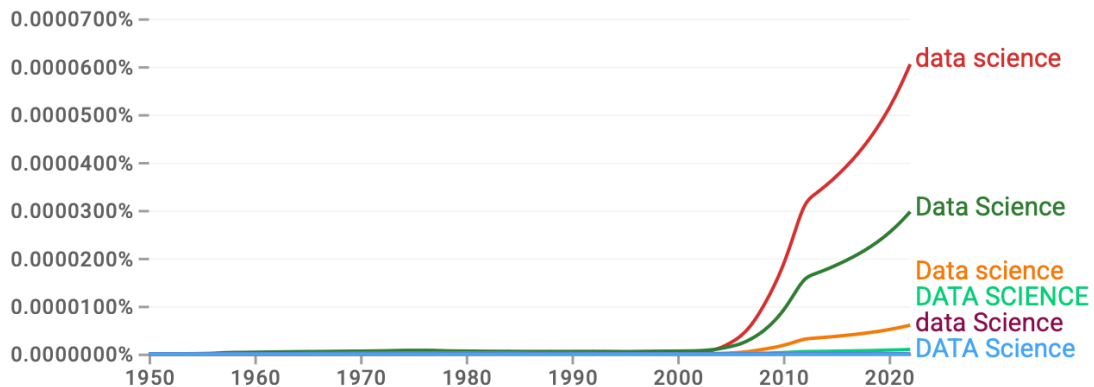


Figure 1: N-Gram Frequency of “Data Science” and its variances from 1950-Present

Source: Google Ngram Viewer.

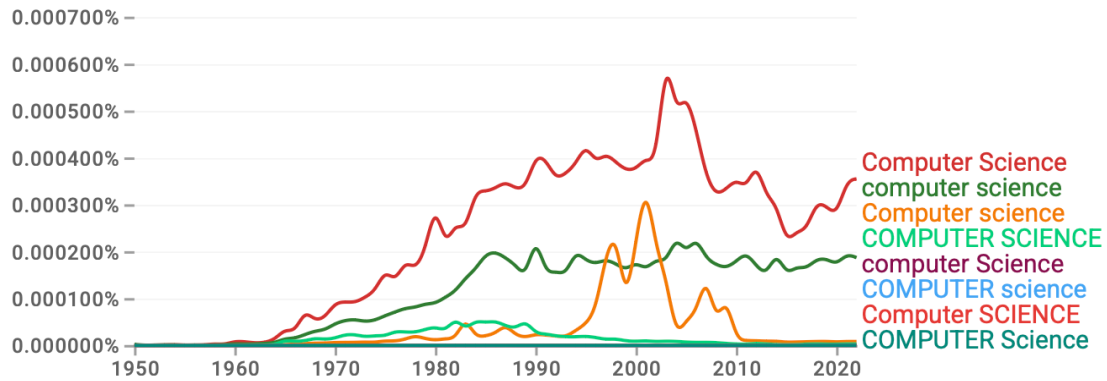


Figure 2: N-Gram Frequency of “Computer Science” and its variances from 1950-Present

Source: Google Ngram Viewer.

As Data Science matures, structural challenges are emerging. Many organizations now operate complex analytical infrastructures layered on top of legacy data pipelines. While system-level modernization efforts increasingly focus on cloud migration, distributed storage, and scalable infrastructure (Assunção et al., 2025; Kadakia et al., 2023), the internal analytical workflows responsible for extraction, transformation, modeling, and reporting often remain unchanged. These pipelines, built incrementally over time, frequently contain embedded heuristics, undocumented transformations, and tightly coupled components that resist adaptation. The result is a paradox: organizations adopt modern infrastructure while maintaining fragile and opaque analytical processes.

This distinction between system modernization and pipeline modernization is critical. Infrastructure modernization upgrades storage environments, improves interoperability, and enhances computational scalability. Pipeline modernization, by contrast, addresses the analytical logic that operates within these systems: the structure of transformations, the modularity of code, the transparency of assumptions, and the reproducibility of outputs. Modern infrastructure cannot compensate for brittle analytical workflows.

Without structured pipeline governance, technical debt accumulates within otherwise modern systems.

Current data science training compounds this issue. The CRISP-DM methodology (Chapman et al., 2000; IBM, 2013) provides structured guidance for building new analytical pipelines. However, it offers limited direction for diagnosing and modernizing existing systems that already encode substantial domain knowledge. In practice, data scientists rarely begin with blank architectures. Instead, they inherit pipelines containing undocumented dependencies, manual preprocessing steps, embedded heuristics, and context-specific adaptations. Replacing such systems entirely is often impractical. Modernization therefore requires structured methods for improving inherited architectures without erasing domain expertise.

Existing research identifies important aspects of this problem but does not resolve it. Studies of data science pipelines categorize structural representations and highlight modular inconsistencies (Biswas et al., 2022). Research on pipeline quality documents systemic failure factors, including hidden transformations and insufficient testing (Foidl et al., 2024). Machine learning systems research warns of hidden technical debt arising from entanglement and undeclared dependencies (Sculley et al., 2015). Software engineering emphasizes modular decomposition as a foundation for maintainability (Parnas, 1972). Yet these bodies of work remain fragmented. They diagnose symptoms and articulate risks but do not provide a prescriptive, experimentally validated methodology for modernizing legacy analytical pipelines in real organizational contexts.

The absence of such methodology is nontrivial. Pipeline modernization is inherently a problem of managing competing objectives: maintainability must be improved without eroding embedded domain knowledge, modularity must be introduced without excessive abstraction, and modeling enhancements must be integrated without compounding technical debt. At the same time, these changes must preserve usability for decision-makers, ensuring that improvements in system structure do not come at the cost of interpretability or practical adoption. These tensions are not independent; they interact and compound, making isolated best practices insufficient. Instead, they necessitate a structured, repeatable approach grounded in empirical evaluation.

This thesis addresses that gap by developing and empirically validating a framework for diagnosing and modernizing legacy data pipelines. The framework is constructed using Design Science Research (Design Science Research; Hevner et al., 2004), in which proposed modernization interventions are iteratively designed, implemented within a real manufacturing pipeline, and evaluated through explicitly defined process metrics. These metrics are not ad hoc; they are grounded in established evaluation traditions that collectively emphasize measurable objectives, structural clarity, cognitive usability, stakeholder alignment, and practical feasibility.

Rather than prescribing specific tools or technologies, the proposed framework formalizes a set of governing principles that guide modernization decisions. In particular, it emphasizes structural governance and modular separation as mechanisms for maintaining clarity, reproducibility as a requirement for reliable evaluation and iteration, and operational alignment as a constraint ensuring that system improvements remain

actionable in practice. Through this structure, the framework provides a repeatable modernization procedure that improves maintainability, interpretability, and scalability while minimizing hidden technical debt and preserving domain context.

The research is guided by the following questions:

1. How can legacy data pipelines be systematically diagnosed to reveal structural weaknesses and modernization priorities?
2. What criteria should govern the introduction of structural and analytical changes within existing pipelines?
3. How can modernization steps be evaluated in a manner that avoids metric recursion and prioritizes methodological rigor over isolated performance gains?
4. Can a structured, hypothesis-driven modernization process produce a generalizable framework applicable beyond a single domain?

The framework is constructed and validated through iterative modernization of a legacy manufacturing data pipeline used for operational decision-making. The manufacturing context serves as an experimental environment rather than the primary research focus.

The central contribution of this work is the framework itself: a structured Standard Operating Procedure for pipeline modernization derived from empirically validated steps.

This thesis contributes:

- A structured methodology for diagnosing legacy data pipelines.
- A process-metric evaluation system for validating modernization steps.

- An empirically derived modernization framework grounded in modular design and usability principles.
- A validated case study demonstrating framework application within an operational manufacturing context.

The remainder of this thesis is organized as follows. Chapter 1 examines the current state of research and theoretical foundations motivating pipeline modernization. Chapter 2 presents the research design and methodological approach. Chapter 3 describes the legacy pipeline baseline used as the experimental environment. Chapter 4 documents the iterative modernization cycles and framework development process. Chapter 5 contains the framework and Chapter 6 discusses the application of the framework to a practical legacy data pipeline.

CHAPTER 1: THEORETICAL FOUNDATIONS OF PIPELINE MODERNIZATION

Data Science has matured rapidly as an applied discipline, yet its structural practices remain uneven. While modeling techniques, distributed computing, and cloud infrastructures have advanced significantly, the internal analytical workflows that govern data extraction, transformation, modeling, and reporting often evolve informally. This mirrors patterns observed in early Computer Science, where functional systems accumulated technical debt before architectural standards emerged.

Research has increasingly examined the structure of data science pipelines. Biswas, Wardat, and Rajan (2022) provide one of the most comprehensive analyses, distinguishing between theoretical pipelines, pipelines-in-the-small, and pipelines-in-the-large. Their work reveals substantial inconsistency in how pipeline stages are defined and modularized. Pipelines-in-the-small, frequently constructed for local analytical tasks, prioritize short-term functionality over modular separation and reuse. Even pipelines-in-the-large exhibit variability in interface design and lifecycle management. While the authors call for greater standardization of pipeline architecture, their work remains primarily descriptive. It identifies structural fragmentation but does not offer a prescriptive methodology for transforming fragmented pipelines into modular systems.

Research has also focused on pipeline quality and failure modes. Foidl et al. (2024) identify 41 factors influencing data pipeline degradation, ranging from unstable data

characteristics to insufficient testing and opaque transformations. Their findings emphasize that pipeline failures are rarely isolated coding errors; rather, they emerge from systemic architectural weaknesses and organizational practices. This work strengthens the argument that pipeline modernization must address structural and procedural deficiencies.

Machine learning systems research further complicates the modernization landscape. Sculley et al. (2015) introduce the concept of hidden technical debt in ML systems, arguing that machine learning components introduce entanglement, undeclared consumers, correction cascades, and feedback loops. These risks suggest that introducing advanced modeling into fragile pipelines may amplify instability rather than resolve it. The implication is clear: modernization cannot simply layer predictive sophistication onto structurally weak foundations. Yet while this research identifies systemic risk, it does not articulate a governance framework for safely integrating modeling changes into legacy analytical workflows.

Software engineering provides a foundational design principle that directly informs this problem: modular decomposition. Parnas (1972) argues that systems should be decomposed according to design decisions likely to change, isolating volatile components behind stable interfaces. Applied to data pipelines, this principle implies separating ingestion, transformation, modeling, and reporting logic in ways that reduce entanglement and improve maintainability.

Beyond technical structure, organizational and cognitive considerations shape modernization outcomes. Camm et al. (2023) distinguish between data science and

decision science skill sets, emphasizing that technical capability alone does not guarantee effective decision support. Similarly, Sanders and Stappers (2008) argue that innovation emerges through co-creation, where domain stakeholders and designers collaborate iteratively. These perspectives highlight a critical oversight in many modernization efforts: structural changes must remain aligned with domain context and user cognition.

Current professional training frameworks further illustrate the gap. CRISP-DM (Chapman et al., 2000; IBM, 2013) provides structured guidance for building new data mining processes but offers limited direction for diagnosing and modernizing inherited pipelines. In real organizational contexts, practitioners rarely begin with blank architectures. Instead, they inherit systems containing embedded heuristics, undocumented transformations, and domain-specific adaptations. The absence of modernization methodology leaves practitioners relying on ad hoc refactoring, incremental patching, or wholesale replacement, none of which provide structured governance.

A final body of literature examines modernization at the infrastructure level. Assunção et al. (2025) and Kadakia et al. (2023) analyze large-scale system modernization strategies, including cloud migration, interoperability enhancement, and architectural redesign. While these works provide valuable guidance for infrastructure transformation, they focus primarily on storage environments, interoperability standards, and system scalability. They do not address the internal analytical logic embedded within data pipelines. Thus, infrastructure modernization may coexist with fragile analytical workflows.

Taken together, the literature reveals a consistent pattern. Researchers have:

- Classified pipeline types.
- Identified failure factors.
- Warned against hidden ML debt.
- Advocated modularity.
- Distinguished between technical and decision skills.
- Proposed infrastructure modernization strategies.

However, no integrated, experimentally validated methodology exists for guiding data scientists through the modernization of legacy data pipelines.

The absence of such methodology is not trivial. Pipeline modernization requires balancing competing objectives: improving maintainability without erasing domain knowledge, increasing modularity without introducing unnecessary complexity, integrating modeling enhancements without amplifying technical debt, and preserving usability for domain stakeholders. Addressing these tensions requires more than isolated best practices; it requires a structured, hypothesis-driven framework validated within real operational contexts.

Design Science Research (Hevner et al., 2004) provides an appropriate methodological foundation for constructing such a framework. By iteratively building and evaluating modernization steps within a real legacy pipeline, it becomes possible to derive prescriptive procedures grounded in empirical evidence rather than theoretical abstraction. The Goal–Question–Metric approach (Basili et al., 1994) further ensures that

evaluation criteria are explicitly tied to defined modernization objectives rather than convenience metrics.

The current state of research identifies the symptoms of pipeline fragmentation and technical debt but does not provide a repeatable modernization protocol. This thesis responds by developing and empirically validating a structured framework for diagnosing and modernizing legacy data pipelines, grounded in modular design principles, usability constraints, operational validation, and feasibility criteria.

CHAPTER 2: RESEARCH DESIGN

This research follows the Design Science Research (DSR) paradigm as articulated by Hevner et al. (2004). Design Science Research advances knowledge through the iterative construction and evaluation of artifacts designed to address real-world problems. Unlike purely descriptive or theoretical research, DSR emphasizes building, testing, refining, and validating artifacts within applied environments. In this study, the artifact under construction is a structured framework for modernizing legacy data pipelines.

The central objective is not to improve a single manufacturing pipeline, but to derive a generalizable modernization procedure grounded in empirical evidence. The manufacturing pipeline serves as an experimental testbed through which candidate modernization steps are proposed, implemented, evaluated, and either incorporated into or excluded from the emerging framework.

Modernization is guided by three core objectives:

- **Maintainability** – The pipeline should support modular updates without cascading structural changes.
- **Interpretability** – Analytical logic and outputs must be transparent to both data scientists and domain stakeholders.
- **Scalability** – The architecture should accommodate evolving data volume and analytical complexity.

Modernization in this work is conducted through an iterative, hypothesis-driven cycle. Each step is implemented within the legacy pipeline and evaluated against structured process criteria before being accepted into the framework.

The iterative process follows five stages:

1. Identify a structural weakness or inefficiency within the legacy pipeline.
2. Propose a modernization step targeting that weakness.
3. Implement the step within the pipeline.
4. Evaluate the step using predefined process metrics.
5. Accept, revise, or reject the step based on evaluation outcomes.

This cycle is repeated until a coherent, validated modernization framework emerges.

Figure 3 illustrates the iterative modernization cycle. The diagram visually represents how proposed steps move from hypothesis to implementation to structured evaluation before integration into the framework.

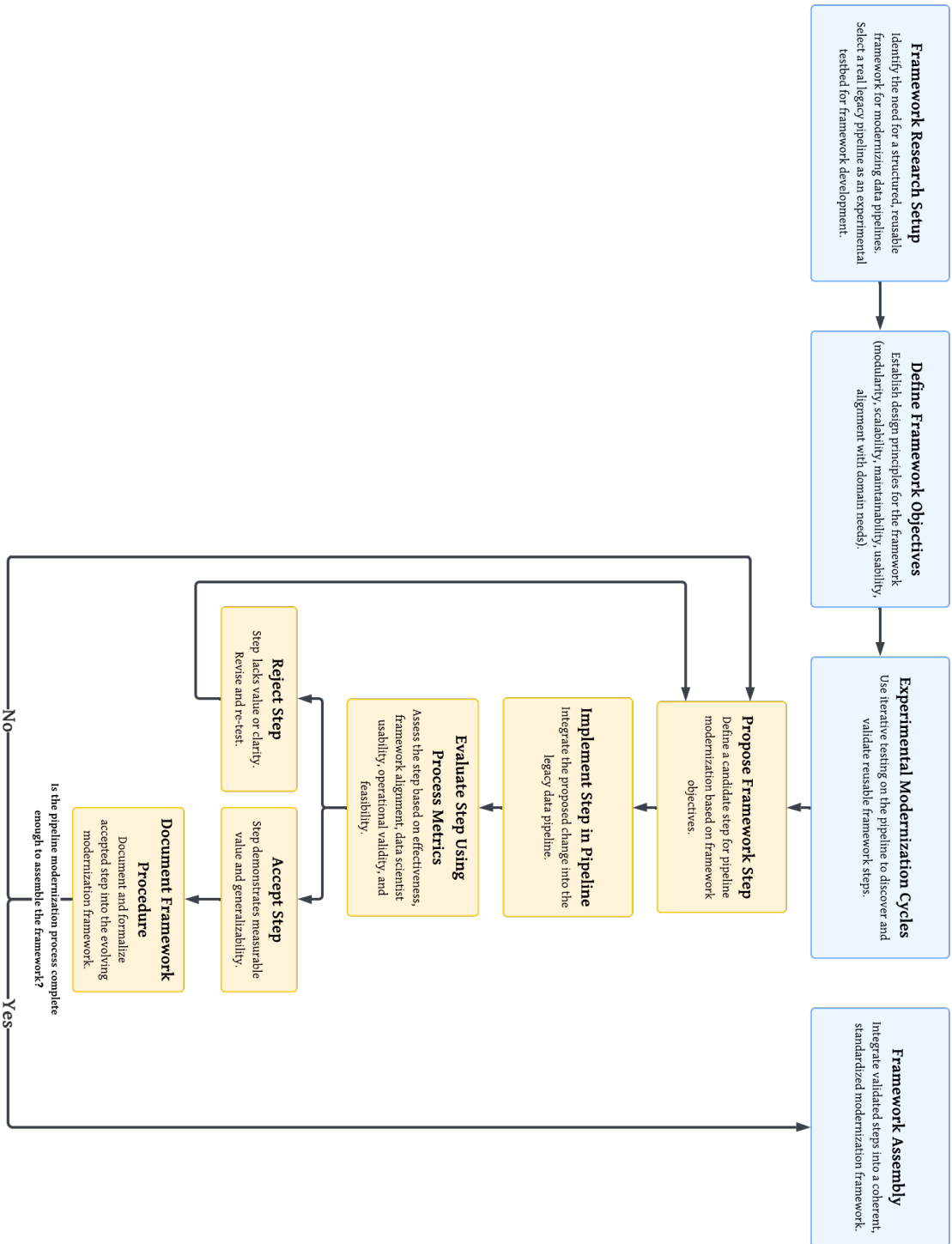


Figure 3: Visual Overview of Proposed Modernization Process

A critical design decision in this research is the use of process-oriented metrics rather than pipeline output metrics. Evaluating the framework based solely on improvements in forecasting accuracy, runtime efficiency, or business performance would introduce metric recursion, whereby the framework is judged by the same outputs it might later influence. To avoid this circularity, each modernization step is evaluated based on methodological rigor and structural improvement rather than downstream performance alone.

Evaluation is conducted using five metric categories derived from established traditions in software engineering, usability analysis, and design research:

- **Effectiveness** – Does the step demonstrably improve maintainability, interpretability, or scalability?
- **Framework Alignment** – Does the step enhance modular separation and reduce entanglement, consistent with modular design principles (Parnas, 1972)?
- **Data Scientist Usability** – Is the step reproducible, externally configurable, and executable in a clean environment without manual intervention, consistent with cognitive usability principles (Green & Petre, 1996)?
- **Operational Validity** – Does the step remain aligned with domain context and stakeholder expectations, consistent with co-creation principles (Sanders & Stappers, 2008)?
- **Feasibility** – Can the step be implemented within reasonable effort constraints, consistent with Design Science evaluation criteria (Hevner et al., 2004)?

These categories collectively ensure that modernization practices are structurally sound, cognitively accessible, operationally relevant, and practically adoptable.

Figure 4 presents the structured evaluation table used to assess candidate framework steps. Practicality is maintained through a focus on feasibility. Although task completion times vary by user, feasibility serves as a metric to ensure each framework step is detailed comprehensively. This clarity prevents resource over-exertion and confusion resulting from vague process explanations. This constraint prevents the framework from incorporating procedures that are theoretically desirable but operationally unrealistic for individual practitioners.

Metric Category	Metric	What It Measures
Effectiveness	Which of the three metrics is the step attempting to improve? Has it done so sufficiently? Yes/No	Does the step improve at least one of the three core goals: maintainability, interpretability, scalability?
Framework Alignment	Modularity Improvement Score (Yes/No)	Evaluates whether the step promotes a modular pipeline architecture.
Data Scientist Usability	Clarity & Reproducibility Score (1–5)	A step is marked usable only if it meets all of the following in a clean environment or after kernel restart: 1. Runs successfully using only the provided code + documentation (no manual intervention).

		2. All parameters and file paths are configurable (not hard-coded). 3. Code includes comments/docstrings describing purpose, inputs, and outputs.
Operational Validity	Domain Expert Feedback Outcome (Accept / Revise / Reject)	Ensures the step produces results that remain aligned with domain context and business needs
Feasibility	Implementation Effort	Total time required to design, code, document, test, and integrate the step into the pipeline. Used to determine whether the step is practical and scalable.

Figure 4: Process Metrics Used in Framework Evaluation

To illustrate how these metrics function in practice, consider a candidate modernization step such as externalizing hard-coded parameters into a configuration file. The step is implemented within the pipeline, then evaluated against the five metric categories. If it improves modularity, enhances reproducibility, remains domain-aligned, and can be implemented within feasible effort bounds, it is accepted into the framework. If it fails one or more criteria, it is revised or rejected.

Importantly, this study distinguishes between evaluating the framework and evaluating the modernized pipeline. While the pipeline serves as the experimental environment, the primary contribution is the methodology governing modernization. Success is therefore defined not solely by pipeline performance improvements but by the framework’s ability

to produce structured, reproducible, and modular transformations across iterative applications.

CHAPTER 3: LEGACY PIPELINE

The modernization framework developed in this research is constructed and evaluated within a real operational data pipeline from a manufacturing context. This pipeline supports production and capacity decision-making processes within a liquid manufacturing environment. While functionally operational, the pipeline evolved incrementally over time in response to emerging business needs rather than through deliberate architectural design. As a result, its structure reflects accumulated adaptations rather than formal modular planning.

The legacy pipeline integrates multiple heterogeneous data sources that collectively represent system performance and operational constraints. Historical production logs provide a record of realized output over time, while downtime records capture periods of reduced or unavailable capacity. Changeover data reflect the operational cost associated with transitions between production states, and manually maintained adjustments encode operator-informed modifications that incorporate domain knowledge not fully captured in automated systems.

Data ingestion is performed through exported files and spreadsheet-based transformations, after which intermediate outputs are generated through a sequence of scripts and formulas that apply smoothing, aggregation, and adjustment logic. Final outputs are delivered through static reports and dashboards used by operational stakeholders.

in Chapter 2 and validated with domain stakeholders to ensure fidelity to operational practice.

Stage boundaries are implicit rather than formally defined. Data ingestion, cleaning, transformation, and estimation logic are partially intertwined. While functional groupings exist, there is limited modular separation between components. Changes to upstream data formats frequently require adjustments across multiple downstream scripts, indicating structural entanglement.

Configuration parameters are embedded within scripts and spreadsheets. Buffer values, smoothing factors, and threshold rules are hard-coded rather than externally configurable. This design choice increases fragility, as parameter modifications require direct code editing and risk introducing unintended side effects.

Reproducibility is limited. Although the pipeline produces consistent outputs under stable conditions, it depends on manual preprocessing steps and locally maintained files.

Execution within a controlled environment necessitates specific contextual knowledge that currently lacks comprehensive documentation. As a result, onboarding new practitioners or auditing historical outputs becomes difficult.

Transformation logic includes embedded heuristics derived from domain expertise. These heuristics, such as buffer adjustments for downtime or manual overrides reflecting production constraints, encode valuable operational knowledge. However, their documentation is informal, and their location within the pipeline is not always

transparent. This creates a tension central to modernization: improving structural clarity without erasing embedded expertise.

The reporting layer emphasizes static outputs over structural transparency. While reports support operational decisions, they do not explicitly distinguish between raw data, adjusted values, or derived estimates. Consequently, diagnosing discrepancies between reported and realized outcomes requires tracing transformations across multiple intermediate steps.

The pipeline resembles what Biswas et al. (2022) describe as a pipeline-in-the-small: locally functional but structurally inconsistent. The entanglement of transformations reflects risks described by Sculley et al. (2015) regarding hidden technical debt. Limited documentation and implicit assumptions mirror quality factors identified by Foidl et al. (2024).

However, it is important to emphasize that the legacy pipeline is not inherently flawed. It represents accumulated domain adaptation and iterative improvement. Its continued operational use indicates that it satisfies baseline business needs. The purpose of modernization is therefore not replacement but structured refinement.

To guide modernization, the baseline pipeline was evaluated qualitatively against the three modernization objectives defined in Chapter 2: maintainability, interpretability, and scalability.

With respect to maintainability, the absence of clear modular separation and externalized configuration limits the ease of modification. Structural changes require navigating intertwined scripts and spreadsheet logic.

With respect to interpretability, embedded heuristics and undocumented transformations reduce transparency for both data scientists and domain stakeholders attempting to audit outputs.

With respect to scalability, reliance on manual preprocessing and static file dependencies constrains expansion to larger data volumes or additional production contexts.

This baseline assessment establishes the experimental conditions for iterative modernization. The objective is not to optimize performance metrics directly, but to introduce structured improvements that enhance maintainability, interpretability, and scalability while preserving domain validity.

The next chapter documents the iterative modernization cycles through which candidate framework steps are proposed, evaluated, and incorporated into the emerging modernization methodology.

CHAPTER 4: ITERATIVE MODERNIZATION AND FRAMEWORK DEVELOPMENT

The modernization framework developed in this research did not begin as a predefined sequence of best practices. The framework was developed and refined through empirical evaluation within the legacy manufacturing pipeline detailed in Chapter 3. Each modernization step was treated as a hypothesis: if introduced, the step should improve, scalability, maintainability, or interpretability without compromising operational continuity. The framework is therefore the result of disciplined evolution rather than theoretical prescription.

The most consequential decision occurred at the outset. Upon reviewing the legacy pipeline, the immediate reaction was to rebuild it entirely. The system exhibited entangled transformations, embedded configuration parameters, undocumented heuristics, and limited reproducibility. From an architectural standpoint, starting from scratch appeared cleaner and more efficient. However, that instinct was deliberately resisted.

The pipeline was deeply embedded within operational workflows. Its heuristics reflected accumulated domain adaptation. Replacing the system wholesale would have risked erasing operational knowledge and destabilizing decision processes. Modernization therefore required restraint. The goal became not architectural replacement, but structured refinement under the constraint of domain preservation.

The first modernization cycle focused on contextual grounding. Conversations with the domain expert revealed that many structural “inefficiencies” were deliberate accommodations for production variability. Buffer adjustments and smoothing logic were not arbitrary; they were responses to real operational constraints. Early attempts at documentation were overly broad and impractical. Iterative refinement narrowed the objective to essential artifacts: explicit articulation of decision context, identification of embedded assumptions, and visual mapping of the pipeline structure. Establishing these boundaries clarified what must remain stable and what could evolve.

Unexpectedly, modernization accelerated once these boundaries were formalized. In prior experiences following CRISP-DM, early business understanding phases were often compressed, and modeling proceeded quickly. In this case, investing deeply in domain preservation did not slow structural progress, it enabled it. By defining preservation constraints collaboratively, subsequent changes could be introduced confidently within those limits. The experience demonstrated that modernization benefits from explicit constraint rather than exploratory freedom.

With contextual grounding established, attention shifted to structural entanglement. The pipeline lacked explicit stage boundaries. Ingestion, transformation, and estimation logic were partially intertwined, and configuration values were embedded directly within scripts. The theoretical ideal, complete modular refactoring, proved destabilizing in early experimentation. Aggressive restructuring introduced short-term fragility and risked altering validated outputs.

This revealed a central modernization tension: modular purity must be balanced against operational continuity. The refinement strategy shifted from rewriting logic to isolating decisions. Instead of reconstructing stages entirely, the process formalized input-output contracts, externalized configuration parameters, and clarified transformation boundaries. These adjustments improved maintainability incrementally while preserving domain-calibrated behavior. Modularity was achieved not through replacement, but through decision isolation.

Following structural clarification, a distinct step titled “Analytical Strategy Selection and Readiness Assessment” was introduced. Its purpose was to evaluate whether the pipeline was sufficiently prepared for modeling enhancements. Conceptually, the step appeared rigorous and aligned with governance principles. However, implementation revealed redundancy. The readiness conditions it sought to formalize, clarity of boundaries, explicit assumptions, configuration separation, were already satisfied by outputs of the structural audit.

Under evaluation, the step failed to demonstrate unique structural value. It increased procedural complexity without improving maintainability or interpretability. Its removal was therefore intentional and instructive. The redundancy indicated that modeling readiness is not an independent phase. It is a byproduct of structural clarity. The elimination of this step reinforced the principle that modernization stages must add distinct structural value to justify inclusion.

With structural clarity stabilized, modeling integration became viable. Machine learning systems research cautions that introducing analytical sophistication into fragile pipelines

amplifies hidden technical debt. Early experimentation focused on predictive enhancement, but iterative refinement redirected attention toward governance rather than ambition. Modeling was required to remain isolated from transformation logic, externally configurable, and explicitly documented. Assumptions were formalized and validated with domain stakeholders. The shift from optimization focus to governance focus reflected a broader insight: modernization prioritizes structural discipline over predictive novelty.

Subsequent cycles addressed visualization and reproducibility. The legacy reporting layer lacked consistent labeling of derived versus observed values, and execution required contextual knowledge. Standardizing visual conventions and introducing a unified execution entry point did not alter analytical logic, yet materially improved interpretability and maintainability. These adjustments reinforced that modernization is not synonymous with modeling complexity; it includes cognitive transparency and execution clarity.

A modernization step should meaningfully reduce ambiguity and effort for future practitioners. Feasibility is evaluated in terms of replicability rather than elapsed time, emphasizing whether a modernization step has been sufficiently specified, validated, and documented to support consistent, repeatable implementation.

Across iterations, evaluation metrics disciplined rather than overruled intuition. The removal of the analytical readiness step, for example, aligned with both empirical results and conceptual redundancy. No retained step survived solely on aesthetic appeal; each

demonstrated structural benefit under explicit criteria. Complete evaluation sheets documenting these decisions are provided in Appendix A.

Several insights emerged consistently across cycles. First, modernization succeeds when domain preservation constrains structural change rather than impedes it. Second, modular clarity must precede analytical expansion. Third, governance mechanisms often matter more than algorithmic sophistication. Finally, modernization is finite: it seeks stabilized structural clarity rather than perpetual iteration.

By the conclusion of iterative experimentation, a coherent sequence of stabilization stages had emerged. These stages were not imposed in advance but discovered through disciplined refinement. The next chapter formalizes these stages into a structured modernization framework and articulates their general applicability beyond the manufacturing context.

CHAPTER 5: THE DATA PIPELINE MODERNIZATION FRAMEWORK

STEP 1: MEET WITH THE DOMAIN EXPERT

Meeting with your Domain Expert before starting to modernize the data pipeline is crucial because contextual understanding is the foundation of effective data work; technical redesigns that are disconnected from domain goals often fail to generate meaningful improvements Biswas et al. (2022). Often, the pitfalls of data pipelines stem from their inability to fulfill the objectives initially defined by the Domain Expert within the operational constraints and infrastructure of the organization. Green and Petre (1996) By prioritizing the contextualization of the pipeline within the broader data infrastructure, the organizational limitations, and the goals articulated by the Domain Expert, the data scientist is better equipped to modernize a system whose structure is rooted in the practicality of the original problem statement and aligned with real operational needs.

As a data scientist it may be tempting to begin your audit of the data pipeline independently as part of the first step of the CRISP-DM framework (IBM, 2013). However the modernization of a data pipeline has distinctive properties that require the data scientist practitioner to work in tandem with the Domain Expert to both gain business understanding as well as guide a formalized conversation surrounding the overall goals of the modernization process.

As previously stated data pipelines are considered pipelines-in-the-small Biswas et al. (2022) in which they have been designed to carry out a very distinct set of steps to solve a very specific problem. However, oftentimes pipelines-in-the-small lack modularity and documentation. Biswas et al. (2022) Because of this the way in which data scientists need to approach a conversation with the domain expert requires them to create questions that guide the Domain Expert through an explanation of the legacy data pipeline that:

1. Allows the data scientist to understand key aspects of the pipeline in which the Domain Expert are content with the current functionality and output
2. Ensures the data scientist has an in-depth understanding of the current workflow of the legacy data pipeline
3. Highlights the key aspects of the legacy data pipeline that need to be improved and where these aspects of the pipeline fall within the larger pipeline and overall data workflow of the organization.

Legacy data pipelines need modernization for a plethora of reasons whether it is simply incapable of producing the correct analytical insight for the organization, not compatible with the current infrastructure or shows an inability to be maintained by a large portion of the organization who rely on this pipeline. To target these issues it is important pipelines and the modernization process of legacy data pipelines prioritize modularity.

Modularity describes the practice of designing a system as distinct parts each responsible for a set of functions. The property of modularity is a foundational aspect of software engineering practices set forth to ensure code infrastructure is resistant to bugs and change Parnas (1972).

When meeting with your Domain Expert to discuss the current data pipeline, it is essential for you, as the data scientist, to structure the conversation around a clear, step-by-step walkthrough of the pipeline's functions. Although a legacy pipeline may lack explicit modularity, it still follows a series of implicit stages, collecting, formatting, analyzing, and distributing data, that are fundamental to its operation. Developing a deep understanding of these stages not only clarifies how the legacy pipeline currently performs but also equips you to design a modernization strategy that preserves and enhances modularity throughout the new system process.

Operational Usability is another important aspect to prioritize when first meeting with your Domain Expert. Through discussions it needs to be made clear who within the organization will be interacting with your modernized data pipeline. This answer will inform how you will structure not only your documentation of the overall pipeline but also the finalized analytical output of the data pipeline. Understanding who your end-user is, is exceedingly important in the design process McGowan et al. (2023). This step is crucial to ensure usability along with maintainability within the organization.

An observed pitfall of legacy data pipelines is their inability to be followed, used and

managed by a large portion of the organization. This is an inherent issue within legacy data pipelines leading to their obsolescence. In your discussion with your Domain Expert make a clear decision about the end user of this pipeline to ensure its longevity and utility within the organization.

Finally, data scientists must develop a clear understanding of the resources, constraints, and expectations associated with the modernized data pipeline. This involves identifying the current implementation environment (e.g., the programming language, platform, or orchestration system in which the legacy pipeline operates), as well as the target environment in which the modernized version is expected to be delivered. Equally important is assessing the availability, accessibility, and reliability of the data currently flowing through the system.

This stage is not the time for a deep exploratory dive into all available organizational data. Doing so may lead to misguided assumptions about the pipeline's potential, particularly when portions of relevant data are incomplete, unavailable, or not yet collected. Overanalyzing the data too early can create false confidence in the pipeline's apparent capacity or its integration within the broader data infrastructure. Instead, data scientists should focus their inquiry on the specific inputs and outputs currently being processed by the pipeline. This targeted approach allows for a deeper understanding of how data flows and interacts within the system, without constructing premature or inaccurate narratives about its limitations.

It is also important to recognize that the data scientist's role is not to simply replicate the existing pipeline under a new structure or technology stack. The partnership between

Domain Expert and data scientist should instead aim to reimagine the pipeline's design by identifying where untapped data sources or latent metadata could enhance its performance and functionality. In many legacy systems, the data required to achieve modernization already exists, either in underutilized data stores, embedded metadata, or parallel processes awaiting integration. Kadakia and DeSalvo (2023)

At the conclusion of this meeting with the Domain Expert, the data scientist should walk away with a comprehensive understanding of the legacy pipeline's strengths and weaknesses, a clear articulation of its overarching functional goal, an identification of the intended end users and their requirements, and a prioritized outline of the specific stages that require improvement or redesign. Most importantly, the data scientist will have established a shared contextual foundation with the Domain Expert, clarifying the pipeline's operational purpose, technical dependencies, and the organizational expectations guiding modernization. This foundational understanding serves as the bridge between the existing system and its envisioned future state, ensuring that any subsequent technical work is grounded in practical reality rather than speculative assumptions about data, infrastructure, or capability.

STEP 2: EXPLICIT STRUCTURAL AUDIT AND VISUAL DECOMPOSITION

Before beginning the development of any modernization strategy, the data scientist must conduct a deliberate and structured examination of the existing data pipeline to establish technical orientation and system-level grounding. The purpose of this step is not to propose redesigns or improvements, but to externalize how the pipeline currently operates, how data moves through it, and where structure, assumptions, and dependencies

are implicitly encoded. Without this intermediate diagnostic phase, modernization efforts risk being driven by isolated technical observations rather than a coherent understanding of the system as it exists in practice.

Following the initial meeting with the Domain Expert, the data scientist independently inspects the artifacts supporting the legacy pipeline, including datasets, scripts, intermediate files, transformation logic, and analytical outputs. This independent review allows the data scientist to identify visible inefficiencies, inconsistencies, or breakdowns in pipeline behavior. However, technical inspection in isolation is insufficient. Without the contextual grounding established in Step 1, such observations risk misrepresenting the original intent of the pipeline, overlooking operational constraints, or incorrectly interpreting the significance of outputs (Green & Petre, 1996). The role of Step 1 is therefore foundational: it anchors technical inspection within domain goals, organizational realities, and operational usage, ensuring that subsequent analysis remains aligned with the pipeline's actual function rather than an abstract notion of technical correctness.

At this stage, the data scientist constructs an explicit visualization of the data pipeline. Many data science pipelines function as pipelines-in-the-small, systems designed to execute a narrow set of tasks for a specific problem and often developed without formal documentation or explicit modular boundaries (Biswas et al., 2022). As a result, much of the pipeline's structure exists implicitly, embedded in scripts, ad hoc processes, or undocumented conventions. The visualization serves as a cognitive and analytical artifact that externalizes this structure, making data dependencies, transformations, assumptions,

and coupling visible for systematic evaluation. Beginning with the high-level outline articulated by the Domain Expert, the data scientist should trace the flow of data from its point of origin through each transformation, analytical computation, and output stage.

When the pipeline operates within a broader operational context, such as a manufacturing process, supply chain, or service workflow, the visualization must extend beyond computational steps alone. The data scientist must explicitly represent where data originates within the physical or organizational process and where analytical outputs are ultimately consumed. This system-level representation reduces the risk of optimizing individual computational components without understanding their downstream operational impact, a common failure mode in legacy system modernization (Biswas et al., 2022).

Stage-Level Audit Checklist (Required Artifact)

For each stage identified in the pipeline visualization, the data scientist should create a structured audit capturing the following elements:

- **Input(s):**

What data is consumed at this stage? Specify variables, files, tables, or streams used as inputs.

- **Input Origin:**

Does the input originate from a previous stage within the same pipeline, or is it sourced from an external system, manual process, or parallel pipeline?

- **Output(s) and Format:**

Where is the data produced at this stage sent next, and in what format is it expected to exist (e.g., database table, CSV file, spreadsheet, API payload)?

- **Current Data State:**

What is the condition of the data at this point in the pipeline? For example, is the data raw, partially processed, fully structured, metadata-driven, manually curated, or inconsistently formatted? Explicitly documenting data state is critical for identifying sources of uncertainty, quality degradation, and hidden preprocessing assumptions (Foidl et al., 2024; Sculley et al., 2015).

- **Domain Expert Commentary:**

Record observations, concerns, and explanations provided by the Domain Expert regarding this stage, including which components are trusted, which are brittle, and which outputs are operationally meaningful.

- **Open Questions and Uncertainties:**

Identify ambiguities, assumptions, or gaps in understanding uncovered during the audit. These questions should be explicitly tracked and revisited in later engagements with the Domain Expert. Maintaining a clear separation between confirmed knowledge and unresolved uncertainty reduces the risk of assumption-driven redesign (Green & Petre, 1996).

This audit should be treated as a living artifact that evolves through continued interaction with the Domain Expert. The goal is not to exhaustively analyze all available organizational data, but to precisely characterize how data currently flows through the pipeline and how each stage contributes to decision-making. Premature exploration of

unrelated datasets or speculative integrations can generate misleading narratives about pipeline capability and should be avoided at this stage (IBM, 2013).

This structured audit directly supports modular system design by encouraging decomposition of the pipeline into functionally coherent components with clearly defined responsibilities. Modularity is a foundational principle of robust software systems and is essential for building pipelines that are maintainable, extensible, and resilient to change (Parnas, 1972). By identifying functional boundaries, data dependencies, and interfaces early, the data scientist establishes the structural foundation necessary for principled modernization.

Finally, this step enables the data scientist and Domain Expert to move beyond simple replication of the legacy pipeline. The shared understanding created through visualization and structured documentation makes it possible to identify underutilized data sources, latent metadata, or parallel processes that could be integrated to enhance analytical relevance and decision quality. Evidence from public-sector data modernization efforts suggests that substantial gains often arise from better integration of existing, fragmented data assets rather than the introduction of entirely new data collection mechanisms (Kadakia & DeSalvo, 2023).

By the conclusion of Step 2, the data scientist should possess a comprehensive, explicitly documented understanding of the legacy pipeline's structure, data flow, dependencies, and constraints. While this step captures descriptive observations regarding functional limitations, inefficiencies, and sources of friction, it deliberately avoids prescriptive conclusions. Decisions regarding preservation, refactoring, or redesign are explicitly

deferred to the next stage of the framework, ensuring that modernization choices are grounded in diagnostic evidence rather than premature technical judgment.

STEP 3: FUNCTIONAL EVALUATION AND MODERNIZATION TARGET IDENTIFICATION

Following the classification of pipeline components into preliminary modernization categories, the data scientist advances from descriptive diagnosis to structured prioritization. The purpose of this step is to determine not only what should be modernized, but where modernization effort should be allocated first, given finite time, organizational capacity, and operational risk tolerance. Crucially, this step ensures that modernization decisions remain grounded in business objectives and operational context rather than being driven by technical convenience or architectural aesthetics.

Prioritization serves as a key mechanism for preserving domain knowledge throughout the redesign process. Without explicit prioritization, modernization efforts risk drifting toward technically interesting components that are peripheral to decision-making, while neglecting stages that are central to operational outcomes. By embedding operational impact and business relevance directly into the prioritization logic, this step maintains continuity between the legacy system's original purpose and its modernized form.

Importantly, this step remains agnostic to specific technologies, modeling approaches, or optimization techniques. Instead, it translates earlier qualitative understanding into a structured decision framework that constrains and guides subsequent technical work.

Step 3.1: Confirm Modernization Categories

Using the classification artifact produced in the previous step, the data scientist first confirms the assigned modernization category for each pipeline component:

- **Preserve:** Components that reliably fulfill their functional role, produce trusted outputs, and remain aligned with current operational needs.
- **Refactor:** Components that generate correct outputs but exhibit structural fragility, poor documentation, limited usability, or hidden dependencies that increase maintenance burden.
- **Redesign:** Components that are misaligned with present-day objectives, introduce disproportionate operational risk, or constrain future extensibility and integration.

These categories represent strategic modernization pathways rather than implementation decisions. They are grounded in principles of modular system design and pipeline quality, emphasizing functional responsibility, interface clarity, and resilience to change (Parnas, 1972; Biswas et al., 2022). Confirming these categories at this stage ensures that prioritization decisions reflect shared understanding between the data scientist and Domain Expert, rather than unilateral technical judgment.

Step 3.2: Evaluate Components Along Decision-Relevant Dimensions

Each pipeline component is then evaluated along a common set of decision-oriented dimensions that explicitly encode operational and business considerations. These dimensions are intentionally limited in number to balance analytical rigor with usability and transparency (Foidl et al., 2024; Green & Petre, 1996):

1. **Operational Impact:** To what extent does this component influence downstream decisions, reporting, or system behavior? Components that directly shape planning, forecasting, or execution outcomes carry greater operational weight.
2. **Risk and Fragility:** How likely is failure, error propagation, or misinterpretation at this stage, and what are the operational consequences? This dimension captures both technical brittleness and business exposure.
3. **Maintainability:** How difficult is the component to modify, debug, or extend given its current structure, documentation, and ownership? Poor maintainability often translates into organizational risk over time.
4. **Alignment with Current Goals:** Does the component support present-day business or operational objectives, or is it optimized for outdated requirements, legacy reporting structures, or obsolete decision processes?

For each dimension, the data scientist records qualitative assessments grounded in evidence from the structural audit and validated through Domain Expert input. This evaluative process explicitly embeds business reasoning into technical prioritization, ensuring that modernization decisions reflect how the pipeline is used, not merely how it is built.

Step 3.3: Conduct Relative Impact–Effort Reasoning

Rather than attempting precise cost or timeline estimation, this framework employs relative impact–effort reasoning. For each component, the data scientist assesses:

- **Expected impact of modernization:** low / medium / high

- **Estimated effort required:** low / medium / high

This comparative approach reflects best practices in early-stage system redesign, where uncertainty is high but relative tradeoffs are still informative. By reasoning comparatively rather than absolutely, the data scientist avoids false precision while still making defensible prioritization decisions.

Critically, this step prevents modernization from becoming decoupled from operational value. Components with high technical complexity but low business impact are deprioritized, while components offering substantial operational benefit at manageable effort are surfaced as early candidates. High-effort components are deferred unless they pose unacceptable operational risk or block critical decision processes.

Step 3.4: Produce a Preliminary Modernization Roadmap

Based on the evaluations above, the data scientist produces a preliminary modernization roadmap that:

- Orders pipeline components by relative priority
- Distinguishes near-term, mid-term, and long-term modernization candidates
- Explicitly documents tradeoffs, dependencies, and deferrals

This roadmap functions as a decision artifact rather than an implementation plan. Its primary role is to make prioritization logic explicit and reviewable, reducing the risk of scope creep and unmanaged complexity during later stages (Sculley et al., 2015). By

clearly identifying which components will not be addressed immediately, the roadmap protects operational stability while enabling incremental progress.

Step 3.5: Validate the Modernization Direction Through a Second Domain Expert Review

At the conclusion of Step 3, a second dedicated meeting with the Domain Expert is required. At this stage, the data scientist presents the results of the pipeline audit and structured prioritization process, using the pipeline visualization as a shared reference artifact. Rather than serving solely as a validation tool, this artifact enables both the data scientist and the Domain Expert to examine the legacy pipeline from a unified, system-level perspective.

In many legacy systems, no single individual possesses a complete, end-to-end view of the pipeline. Knowledge is often distributed across roles, embedded in ad hoc practices, or obscured by incremental changes over time. The pipeline visualization therefore functions as a mechanism for joint sense-making: it allows the Domain Expert to see the pipeline as the data scientist currently understands it, while simultaneously allowing the data scientist to observe how the Domain Expert interprets, contextualizes, and challenges that representation.

This review is also the point at which high-level sequencing and expectations are aligned. The data scientist and Domain Expert should discuss the anticipated order of modernization efforts, the rationale behind prioritization decisions, and approximate timelines or phases for each stage. Establishing this shared understanding ensures that

modernization proceeds with realistic expectations regarding scope, effort, and operational disruption.

If material disagreements or misalignments emerge during this meeting, whether related to prioritization, perceived risk, or operational feasibility, the modernization plan must be revisited. Step 3 is therefore explicitly iterative. The data scientist may need to revise component evaluations, reweight decision-relevant dimensions, or reconsider identified tradeoffs considering newly surfaced domain knowledge.

Advancing to design or implementation without achieving congruence between the data scientist's analytical perspective and the Domain Expert's operational understanding risks modernizing a pipeline in ways that misrepresent functional requirements, disrupt established workflows, or fail to deliver intended analytical or decision-support value.

STEP 4: MODELING AND MACHINE LEARNING CONSIDERATIONS

***Scope Note:** The considerations and constraints defined in Step 4 are applied at the level of individual pipeline components, not as a single, global decision. For each component identified in the modernization roadmap that may benefit from predictive forecasting, or learning-based behavior, the data scientist must reapply the requirements of this step. Components that do not meet the criteria for modeling are explicitly excluded and remain governed by deterministic or descriptive logic.*

As a result, Step 4 may be revisited multiple times across the pipeline, but only for components where analytical complexity is under consideration. Modeling decisions are therefore localized, traceable, and constrained, rather than applied uniformly across the system.

Once modernization priorities have been established and validated through the iterative prioritization process in Step 3, the data scientist may begin to evaluate whether

predictive or learning-based models should be introduced into specific components of the modernized pipeline. The purpose of this step is not to maximize analytical sophistication, but to ensure that any modeling techniques introduced are justified by decision value, compatible with operational constraints, and governable over time.

This step represents a critical inflection point in the modernization framework. While earlier steps focus on structure, flow, and prioritization, Step 4 governs the introduction of analytical complexity. Without explicit constraints, modeling decisions are often driven by familiarity, perceived novelty, or marginal performance improvements, leading to systems that are difficult to interpret, maintain, or adapt. Step 4 exists to prevent this failure mode by requiring modeling decisions to be reasoned, documented, and reviewed before implementation.

4.1 Systematic Consideration of Plausible Modeling Techniques

Using the finalized artifacts from Step 3, including the pipeline visualization, component prioritization, and operational considerations, the data scientist must first consider which modeling or analytical techniques are plausibly applicable to each candidate pipeline component. This consideration must be explicit and comparative rather than implicit or singular.

The data scientist is expected to reason through multiple candidate approaches, which may include deterministic rules, statistical models, forecasting techniques, or machine learning methods. This does not require a formal written list, but it does require the data scientist to avoid premature convergence on a single technique.

In evaluating plausible approaches, the data scientist must consider:

- the scale of the data involved and its implications for training time and computational cost,
- how frequently the model would need to be retrained or recalibrated,
- the sensitivity of the model to data volume, feature dimensionality, and distributional shifts,
- and whether the model's computational and operational demands are compatible with the pipeline's execution cadence. (Khuri, Natalie. CSC 791: Data-Centric AI. Fall 2025, Wake Forest University)

This step ensures that modeling choices are constrained not only by theoretical suitability, but by practical feasibility within the organization's technical environment.

4.2 Data Audit and Modeling Consideration

Before applying any modeling techniques, the data scientist must conduct a focused data audit on the specific dataset feeding the pipeline component under consideration. This audit is intentionally narrower than full exploratory analysis and is conducted solely to inform modeling feasibility and selection.

At a minimum, the data scientist must assess:

- the dimensionality of the dataset (number of observations and features),
- data types and schema consistency,
- the extent and structure of missing data,

- and temporal coverage and granularity where applicable.

If substantial missing data or irregularity is identified, the data scientist must investigate the origin of these issues. When missing data arises from upstream collection processes or external data sources, the data scientist may need to consult the Domain Expert to understand operational causes. However, caution is required to avoid unintentionally expanding the scope of the modernization effort beyond what was established in Step 3. The purpose of this audit is not to redesign upstream processes, but to determine which modeling approaches are viable given the data as it currently exists. This audit anchors model selection in empirical data properties rather than abstract assumptions.

Many modeling and machine learning techniques are computationally expensive and scale nonlinearly with data size. To avoid excessive iteration cost and premature commitment, the framework explicitly recommends evaluating candidate models on a representative subsample of the data.

The subsample should preserve key structural characteristics of the full dataset, including relevant distributions, temporal patterns, and known sources of variability. Applying candidate models to this subsample allows the data scientist to observe how the data behaves within each modeling framework and to identify analytical bottlenecks, instability, or sensitivity to preprocessing decisions.

This stage is exploratory but constrained. Its purpose is to understand model behavior and feasibility, not to optimize performance. Hyperparameter tuning, extensive feature engineering, and performance maximization are deferred to later stages.

For each candidate model evaluated on the subsample, the data scientist must establish at least one baseline. Baselines may include naïve forecasts, historical averages, or deterministic rules, depending on the nature of the pipeline component.

The role of the baseline is to ensure that candidate models provide meaningful improvement over minimal-information approaches. Models that fail to outperform an appropriate baseline are excluded from further consideration. This exclusion is definitive; models eliminated at this stage are not scaled to the full dataset.

By enforcing baseline comparison, this step prevents the introduction of modeling complexity that does not translate into substantive decision value.

4.3 Explicitly Evaluate Interpretability, Technical Debt, and Long-Term Maintainability

Before any model is scaled to the full dataset or integrated into the modernized pipeline, the data scientist must explicitly evaluate how the model will be interpreted, maintained, and governed over time. This evaluation is not secondary to predictive performance; it is a primary design constraint. Modernization efforts frequently fail when models that perform well in development introduce long-term technical debt through opacity, brittle assumptions, or maintenance requirements that exceed organizational capacity. (Sculley et al., 2015)

Interpretability in this framework is defined broadly. It includes not only the transparency of model internals, but also the extent to which model behavior, outputs, and failure modes can be understood and communicated by individuals other than the original

developer. A model that cannot be meaningfully explained to downstream users, system owners, or future maintainers increases dependency on tacit knowledge and creates a single point of failure within the organization.

The data scientist must therefore assess candidate models along three interrelated dimensions: interpretability, technical debt accumulation, and operational sustainability.

The data scientist must determine whether a model's outputs can be explained at the level required by its decision context. This assessment should be made before full-scale training and should be grounded in how outputs will be used in practice.

Examples of modeling approaches with relatively high inherent interpretability include:

- Linear regression and generalized linear models (GLMs)
- Rule-based systems and deterministic heuristics
- Small decision trees with limited depth
- Classical time-series methods such as moving averages, ARIMA, or exponential smoothing when components (trend, seasonality, residuals) are explicitly documented

These approaches allow stakeholders to reason about why an output was produced and how changes in inputs affect results. They are often preferable when models inform planning, resource allocation, or other decisions that require explanation or justification.

By contrast, models such as random forests, gradient-boosted trees, support vector machines, and neural networks introduce increasing levels of opacity. While post hoc

explanation techniques (e.g., feature importance measures or local explanations) may partially mitigate this opacity, they do not eliminate the underlying complexity. The data scientist must therefore assess whether the decision context can tolerate explanations that are approximate rather than exact.

Highly opaque approaches, such as deep neural networks or large ensemble methods, should be treated as exception cases. These models are discouraged unless the decision value is substantial and the organization has the capacity to support rigorous monitoring, documentation, and failure containment.

Beyond interpretability, the data scientist must consider how candidate models contribute to long-term technical debt. Prior work has shown that machine learning systems accrue debt not primarily through code complexity, but through *entanglement* with data pipelines, preprocessing logic, configuration parameters, and downstream consumers (Sculley et al., 2015).

Specific sources of technical debt that must be considered include:

- **Feature entanglement:** Models that rely on large numbers of engineered features tightly couple modeling logic to upstream data representations, making future changes risky.
- **Data dependency fragility:** Models that assume stable distributions, consistent schemas, or uninterrupted data collection can fail silently when upstream conditions change.

- **Retraining and recalibration burden:** Complex models often require frequent retraining, hyperparameter tuning, or monitoring to maintain acceptable performance.
- **Evaluation debt:** Models whose performance can only be assessed long after decisions are made (or not at all) are difficult to govern effectively.

Simpler modeling approaches often impose lower technical debt by virtue of their stability, transparency, and reduced dependence on fragile assumptions. As a result, marginal gains in predictive accuracy must be weighed against increases in system complexity and maintenance cost.

The data scientist must explicitly evaluate whether a model can be maintained by individuals other than themselves. This includes assessing:

- Whether the model can be retrained or recalibrated without deep familiarity with its internals.
- Whether documentation can adequately capture assumptions, limitations, and expected behavior.
- Whether failures can be diagnosed without extensive manual investigation.

Models that require bespoke interpretation, manual intervention, or continuous oversight by a specific individual are discouraged unless their decision impact clearly justifies this dependency. Modernization should reduce organizational reliance on individual expertise, not reinforce it.

Only models that satisfy all the following conditions may proceed to full-scale implementation:

1. Their outputs can be meaningfully explained at the level required by downstream users.
2. Their technical debt profile is consistent with the organization's capacity for maintenance and monitoring.
3. Their operational burden (training time, retraining frequency, monitoring needs) is feasible within the pipeline's execution environment.

This explicit evaluation ensures that modernization enhances the pipeline's durability and usability rather than introducing hidden fragility. By forcing the data scientist to account for interpretability and technical debt before implementation, the framework prevents a common failure mode in which analytically sophisticated models undermine the long-term effectiveness of the system.

This step ensures that learning-based components are introduced as stable, governable system elements rather than experimental artifacts, preserving both operational context and business relevance as analytical complexity increases.

4.4 Full-Scale Application and Reassessment on the Complete Dataset

Only models that satisfy the interpretability, technical debt, and maintainability criteria defined in the previous subsection may be applied to the full dataset. At this stage, the data scientist evaluates whether behaviors observed on the subsample persist at scale, including stability of outputs, sensitivity to data volume, and computational feasibility.

This step is not a license for unrestricted optimization. While models may be refined to ensure stable operation on the full dataset, the data scientist must remain within the modeling approach and assumptions previously evaluated. Significant changes in behavior or feasibility at scale must be documented and may require revisiting earlier decisions in Step 4.

Once a modeling approach has been evaluated on the full dataset, the data scientist must meet with the Domain Expert before proceeding further. In this meeting, the data scientist presents:

- the modeling techniques considered,
- the rationale for selecting the proposed model,
- how the model behaves and what its outputs represent,
- and the operational implications of training, retraining, and maintenance.

This meeting is not informational; it is a decision checkpoint. The Domain Expert's approval is required to proceed. If the proposed model is rejected, the data scientist must return to alternative techniques previously evaluated on the subsample and discuss substitutes. This ensures that rejection does not reopen unconstrained experimentation but redirects effort within already explored analytical space.

Step 4 ensures that modeling and machine learning are introduced as deliberate, governable system components, rather than opportunistic enhancements. By enforcing explicit evaluation of interpretability, technical debt, and maintainability before

implementation, this step preserves operational context and business relevance now when analytical sophistication most often undermines them.

By the conclusion of Step 4, the data scientist and Domain Expert share a concrete, defensible understanding of why a particular modeling approach is being used, how it behaves with the data, and what obligations it introduces into the system. Only at this point may the framework proceed to full integration and deployment.

STEP 5: VISUALIZATION AND COMMUNICATION DESIGN

Scope Note:

The visualization and communication requirements defined in Step 5 are applied whenever a pipeline component produces outputs intended for human interpretation. These requirements are not satisfied by a single dashboard or reporting layer but must be considered for each decision-facing output introduced or modified during modernization.

Accordingly, Step 5 may be revisited at multiple points in the pipeline as new outputs are exposed, analytical behavior changes, or decision contexts evolve. Each decision-facing visualization must independently satisfy the design, stability, and governance constraints defined in this step.

After analytical intent has been established and modeling decisions have been explicitly governed, the data scientist must design visual artifacts that translate analytical outputs into forms that support correct interpretation and responsible decision-making. The purpose of this step is to ensure that analytical results are not only technically valid, but operationally legible within the organization. Visualization is therefore treated as a core component of the system design, tightly coupled to analytical logic, modeling

assumptions, and decision context, rather than as a presentation layer applied after technical work is complete.

This step exists to prevent a common failure mode in modernized pipelines: systems that generate improved analytics but degrade decision quality because outputs are misinterpreted, over-trusted, or detached from established operational practices. In many legacy pipelines, visual artifacts function as cognitive infrastructure. They encode institutional knowledge about what constitutes normal behavior, meaningful variation, and actionable deviation. Modernization efforts that replace these artifacts without careful consideration risk severing the link between analytical improvement and operational understanding.

Step 5 ensures that visual design preserves and extends this institutional knowledge rather than discarding it.

5.1 Anchor Each Visualization to a Specific Decision Context

For each pipeline component that produces outputs consumed by humans, the data scientist must explicitly determine whether a decision-facing visual artifact is required. A visual artifact is required whenever an output informs judgment, prioritization, planning, approval, or monitoring decisions. Outputs that are consumed solely by downstream systems or automated processes must not be visualized by default.

This requirement exists because visual artifacts are not neutral representations of data; they actively shape how information is interpreted and acted upon. When visualizations are created without a clearly defined decision context, they tend to accumulate

information indiscriminately, prioritizing completeness over usefulness. Such artifacts often appear analytically rich while providing little guidance for action, increasing the likelihood of misinterpretation or disengagement by users.

Accordingly, the data scientist must treat each visual artifact as an intentional interface between the pipeline and a specific operational decision. For every required visualization, the data scientist must explicitly document:

- the decision or workflow the visualization supports,
- the role of the intended user (e.g., operator, planner, manager, analyst),
- the frequency with which the decision is made,
- and whether the visualization is used to trigger action, support monitoring, or provide situational awareness.

This documentation step forces alignment between analytical outputs and organizational use. It prevents the creation of visual artifacts that exist “because the data is available” rather than because they support a concrete decision. Visual artifacts that cannot be clearly tied to a specific decision context must not be implemented. Leaving such artifacts out of the system is a deliberate design choice. Including visualizations without an operational role increases cognitive clutter, dilutes attention from critical signals, and undermines the usability of the overall pipeline (Endsley, 2011).

By requiring explicit registration of decision-facing visual artifacts, this step ensures that visualization design remains purpose-driven and constrained. It establishes a clear boundary between analytical outputs that require human interpretation and those that should remain programmatic, preserving clarity, focus, and decision relevance as the pipeline is modernized.

5.2 Preserve Operational Knowledge Through Visual Continuity

For each registered decision-facing visual artifact, the data scientist must select and fix a single visual encoding appropriate to the decision context. This includes the chart type, axis definitions, scale, aggregation level, color encodings, labeling conventions, and layout. Once selected, these elements must remain stable across time and updates.

This requirement exists because operational decision-making relies heavily on pattern recognition developed through repeated exposure. When visual encodings change between views or over time, through dynamic rescaling, shifting axes, or altered color schemes, users are forced to re-interpret the visualization on each encounter. This increases cognitive load, slows decision-making, and raises the likelihood of misinterpretation, particularly in time-constrained or high-frequency workflows (Sweller, 2011).

Accordingly, the data scientist must prioritize interpretive stability over visual optimization. For operational decision contexts, axes must remain fixed across time to allow meaningful comparison between periods. Dynamic rescaling, automatic normalization, or context-dependent axis adjustment is not permitted for decision-facing

visuals, as these techniques obscure changes in magnitude and undermine users' ability to detect trends or anomalies reliably (Adar et al., 2021).

Color encodings must be assigned consistently and used sparingly. Colors should map to stable semantic meanings (e.g., categories, status states, thresholds) rather than decorative emphasis. Once a color is associated with a particular concept, it must not be reused for a different purpose within the same visualization or across related artifacts. This consistency supports rapid recognition and reduces interpretive effort.

For analytical review contexts, limited flexibility in encoding may be permitted to support deeper inspection or comparison. However, even in these cases, the data scientist must ensure that visual complexity remains bounded and that changes in encoding are explicitly communicated. Visual artifacts intended for repeated use in decision workflows must always favor consistency over expressiveness.

By fixing visual encodings, the data scientist ensures that users interact with a stable visual language rather than a shifting analytical interface. This stability allows Domain Experts and other stakeholders to build and retain interpretive expertise over time, reducing cognitive burden and supporting efficient, reliable decision-making within the modernized pipeline (Few, 2006).

5.3 Visually Distinguish Descriptive, Forecasted, and Modeled Outputs

When a legacy visualization already exists for a given decision context, the data scientist must treat that visualization as the default design baseline for the modernized system.

This requirement exists because Domain Experts and other operational stakeholders have already internalized the visual structure through repeated use. Over time, these visual forms become tightly coupled to decision-making practices, enabling users to rapidly recognize patterns, deviations, and anomalies without conscious effort.

Altering a familiar visualization unnecessarily imposes additional cognitive load on users. Even when underlying analytics are improved, changes to chart type, layout, scale, color, or labeling require users to relearn how to interpret the information. This relearning process introduces friction, increases the likelihood of misinterpretation, and raises the risk that the visualization will be ignored, misused, or abandoned altogether. As a result, analytically superior outputs may fail to translate into improved decisions (Few, 2006).

Accordingly, when modernizing a pipeline, the data scientist must prioritize visual continuity over novelty. If an existing visualization is well understood and actively used, the preferred strategy is to preserve its visual structure while updating only the underlying data sources, calculations, or analytical logic. In practice, this means retaining the same chart type, axes, scales, labeling conventions, and layout wherever possible. If changes to the visualization are necessary, for example, to accommodate new analytical outputs or correct known misinterpretation, the data scientist must explicitly identify which visual elements can be preserved. Preservable elements may include color schemes, titles, font choices, ordering of elements, or spatial layout. Retaining these elements anchors the modernized visualization in familiar visual language, allowing users

to transfer existing interpretive knowledge to the updated system with minimal effort (Munzner, 2014).

By deliberately preserving visual conventions, the data scientist reduces the cognitive burden placed on Domain Experts and other stakeholders during the transition to the modernized pipeline. This approach supports ease of use, accelerates adoption, and increases the likelihood that the visualization will remain operationally relevant over time rather than becoming obsolete due to avoidable usability barriers.

5.4 Reflect Model Behavior and Validity Constraints in Visual Design

For each decision-facing visual artifact, the data scientist must explicitly encode the analytical status of the values being displayed. At a minimum, the visualization must make clear whether values represent observed historical data, descriptive aggregates, forecasts or predictions, or scenario-based or simulated outputs. These distinctions must be visible within the visualization itself and must not rely solely on external documentation or user memory.

This requirement exists because users routinely interpret visual continuity as factual continuity. When modeled or forecasted values are presented in a manner that visually resembles observed data, users are likely to treat predictions as realized outcomes rather than conditional estimates. This misinterpretation can lead to inappropriate confidence, delayed intervention, or incorrect operational decisions. Such failures are not caused by poor modeling, but by ambiguous visual communication (Gleicher, 2011).

Accordingly, the data scientist must ensure that modeled outputs are never visually indistinguishable from observed data. When historical data and model outputs appear in the same visualization, the transition point between observed and modeled values must be explicit. Acceptable techniques include changes in line style (e.g., solid to dashed), color shifts with consistent semantics, shaded forecast regions, or clear segmentation markers. Seamless continuation of lines or bars from historical to predicted values is not permitted. When forecasts are presented, the visualization must reflect their conditional nature. If uncertainty estimates are available, they must be visually encoded using bands, ranges, or multiple trajectories. Single-point forecasts displayed without context are discouraged unless explicitly justified by the decision context. The objective is to reinforce that predictions are estimates subject to uncertainty, not deterministic truths (Hullman, 2019).

For scenario-based or simulation outputs, the visualization must support comparison across scenarios rather than emphasizing a single outcome. Multiple trajectories, fan charts, or side-by-side comparisons must be used to convey the range of plausible futures. Visual designs that collapse scenario information into a single expected value are not permitted, as they obscure uncertainty and encourage overconfidence (Kay et al., 2016).

By enforcing explicit encoding of analytical status, the data scientist ensures that visual artifacts communicate not only values, but the epistemic conditions under which those values were produced. This protects decision-makers from unintentionally treating modeled outputs as facts and preserves responsible use of analytical results as the pipeline becomes more sophisticated.

5.5 Communicate Data Provenance, Assumptions, and Update Behavior

For each decision-facing visual artifact, the data scientist must ensure that users can readily access information describing how the displayed values were produced. This includes data provenance, update behavior, and any assumptions that materially affect interpretation. This information must be available at the point of use and must not require users to consult external documentation repositories or rely on institutional memory.

This requirement exists because analytical outputs are frequently interpreted without full awareness of their data sources, refresh cadence, or underlying assumptions. When this contextual information is hidden or unclear, users may unknowingly base decisions on outdated data, misaligned assumptions, or model outputs that no longer reflect current conditions. Such failures are not the result of incorrect analytics, but of insufficient transparency at the interface where decisions are made.

Accordingly, each visualization must expose, either directly or through an immediately accessible companion panel, the following information:

- the data sources used to generate the visualization,
- the timestamp of the most recent update,
- the expected update cadence,
- and whether the values are derived from observed data, learned relationships, or simulated scenarios.

For visual artifacts derived from predictive or learning-based models, the data scientist must additionally surface any assumptions or constraints that materially affect

interpretation. This includes known validity horizons, regime assumptions, or dependencies on upstream data stability. Users must be able to determine not only what the visualization shows, but under what conditions the information remains reliable (Sperrle et al., 2021).

This information must be presented in a concise and standardized form. Overly technical descriptions are discouraged; however, omission of materially relevant context is not permitted. The goal is to support informed interpretation without requiring users to infer or reconstruct analytical assumptions on their own.

Visual artifacts that do not provide accessible provenance and update information must not be used to support operational decisions. Treating transparency as optional increases the risk of silent misuse and undermines the reliability of the modernized pipeline. By enforcing visibility of provenance, assumptions, and update behavior, the data scientist ensures that visual artifacts remain interpretable over time, even as data sources, models, or organizational context evolve. This requirement supports responsible decision-making and prevents analytical outputs from becoming detached from their conditions of validity.

5.6 Enforce Separation Between Exploratory and Decision-Facing Visuals

The data scientist must maintain a strict separation between exploratory visualizations created during analysis or model development and decision-facing visual artifacts intended for operational use. Exploratory visuals must never be promoted directly into decision workflows without intentional redesign, documentation, and stabilization.

This requirement exists because exploratory visualizations are optimized for analytical discovery, not for repeated use or correct interpretation by non-technical stakeholders. They are often provisional, densely encoded, and sensitive to parameter changes. While such properties are valuable during development, they become liabilities in operational contexts, where consistency, interpretability, and stability are required (Tukey, 1977).

When exploratory visuals are reused without modification, users may encounter representations that change unexpectedly, rely on implicit assumptions, or expose intermediate analytical states that were never intended to inform decisions. This can lead to confusion, misinterpretation, and inconsistent decision-making. These failures are not attributable to poor analysis, but to a breakdown in the boundary between exploration and operation.

Accordingly, the data scientist must treat decision-facing visual artifacts as stable system components. Before any visualization is approved for operational use, the data scientist must:

- simplify the visual encoding to include only information relevant to the decision context,
- fix axes, scales, color mappings, and layout,
- document the visualization's purpose, assumptions, and update behavior,
- and assign a version identifier to the artifact.

Once a visualization is designated as decision-facing, any subsequent changes to its structure, encoding, or semantics must be treated as system changes. Such changes must be reviewed, documented, and communicated to users. Ad hoc adjustments or analyst-specific modifications are not permitted in operational contexts (Munzner, 2014).

Exploratory visualizations may continue to exist for analytical purposes, but they must remain clearly separated from decision workflows. This separation ensures that experimentation, iteration, and model debugging do not inadvertently influence operational decisions. By enforcing this boundary, the data scientist prevents exploratory complexity from being operationalized and ensures that decision-makers interact only with visual artifacts that have been intentionally designed, reviewed, and governed for their specific use. This distinction preserves interpretive stability and protects the integrity of the modernized pipeline as it evolves.

After analytical intent has been clarified, modeling decisions have been bounded, and visualization and communication artifacts have been designed, the data scientist proceeds with implementation under explicit framework constraints. The purpose of this step is not merely to operationalize technical components, but to verify that the implemented system behaves in accordance with the decisions, assumptions, and design intent established throughout earlier stages of the framework. Implementation is therefore treated as a validation activity rather than a terminal phase.

This step begins with controlled implementation, in which redesigned pipeline components are integrated incrementally rather than deployed wholesale. The data

scientist preserves validated functionality from the legacy system while introducing modernized components in the order specified by the modernization roadmap. This sequencing ensures that system behavior remains interpretable and that deviations from expected outcomes can be traced to specific changes rather than to cumulative disruption. Once implementation is complete, the data scientist performs alignment validation across three dimensions: analytical behavior, communicative behavior, and operational behavior.

Analytical validation ensures that the implemented system produces outputs consistent with the analytical role defined in Step 4 and the modeling constraints articulated in Step 5. This includes verifying that predictive outputs, where present, behave within expected ranges and respect documented assumptions, rather than silently introducing new analytical behavior.

Communicative validation ensures that visual artifacts and supporting documentation function as intended in practice. The data scientist confirms that end users can interpret outputs correctly, distinguish between observed and inferred values, and understand uncertainty where applicable. Visualizations are evaluated not for aesthetic quality, but for interpretive stability and decision alignment (Few, 2006). Any discrepancies between intended and actual interpretation are documented as design failures rather than user error, reinforcing the framework's responsibility for communicative clarity (Endsley, 2011).

Operational validation focuses on how the implemented pipeline integrates into real workflows. The data scientist verifies that outputs are delivered at appropriate times, that update frequencies align with decision cycles, and that failure modes are visible rather than silent. This includes confirming that fallback logic functions as designed, that degraded performance does not propagate undetected, and that maintenance responsibilities defined in earlier steps are feasible in practice.

STEP 6: STANDARDIZE IMPLEMENTATION, REPRODUCIBILITY, AND SYSTEM-LEVEL VALIDATION

After analytical intent has been clarified, modeling decisions have been bounded, and visualization artifacts have been finalized, the data scientist proceeds with implementation under explicit framework constraints. The purpose of this step is to ensure that the modernized pipeline executes in a standardized, reproducible, and inspectable manner, and that its overall health can be assessed without requiring deep inspection of individual components.

Implementation is treated as a validation and stabilization phase rather than a terminal handoff. At this stage, the framework remains active and prescriptive, governing how the system runs, how correctness is verified, and how failures are surfaced.

The data scientist must standardize pipeline execution such that the entire system can be run end-to-end by another data scientist without reliance on undocumented knowledge or ad hoc intervention.

At a minimum, this requires the data scientist to produce all three of the following:

1. **A single, explicit pipeline entry point:** The pipeline must have a clearly defined execution mechanism (e.g., a master script, Makefile target, workflow DAG, or orchestration command) that specifies the order in which components run. Execution order must not be implicit or dependent on manual sequencing.
2. **A configuration specification that externalizes assumptions:** All environment-dependent values (paths, thresholds, time windows, model parameters, refresh cadence) must be defined in a configuration file or structured parameter interface rather than embedded directly in code. This ensures that changes to execution behavior are deliberate and traceable.
3. **Deterministic execution guarantees where applicable:** Given the same inputs and configuration, the pipeline must produce the same outputs. If stochastic elements are present (e.g., sampling, randomized model training), the data scientist must explicitly document and control randomness through fixed seeds or bounded variability statements.

Together, these requirements guarantee that the modernized pipeline is reproducible, auditable, and transferable. A pipeline that cannot be reliably re-run by another data scientist is considered incomplete under the framework.

Once implemented, the data scientist must validate that the pipeline behaves according to the analytical, communicative, and operational intent defined in earlier steps.

This validation must include:

- confirming that descriptive components remain deterministic and interpretable

- confirming that predictive or learning-based components operate only within approved horizons and assumptions,
- verifying that decision-facing visual artifacts match the finalized designs from Step 5,
- and explicitly testing fallback or degraded-mode behavior for components where such behavior was specified.

Validation failures must be documented as framework misalignment, not implementation bugs. Corrective action must be taken by revisiting the relevant framework step rather than patching behavior locally.

In addition to component-level validation, the data scientist must implement a system-level health view that allows stakeholders to assess whether the pipeline is functioning as expected without examining individual scripts, models, or datasets.

This birds-eye view must be implemented as a concrete artifact and must include, at minimum, the following elements:

1. **Stage-level execution status**

Each major pipeline stage (ingestion, transformation, modeling, visualization output) must expose a clear success/failure indicator. This may take the form of:

- a status table,
- a checklist-style dashboard,
- or a structured log summary.

2. **Data freshness and completeness indicators**

The health view must indicate whether key inputs and outputs are up to date relative to their expected cadence (e.g., last updated timestamp, expected vs actual row counts). This allows users to detect stale or incomplete data without inspecting raw files.

3. **Degraded or fallback mode visibility**

If any component is operating in a fallback or degraded mode (e.g., baseline model used instead of predictive model), this state must be explicitly surfaced. Silent degradation is not permitted.

4. **High-level anomaly signals**

The health view should surface simple indicators of abnormal behavior, such as missing outputs, failed stages, or values outside expected ranges. These signals are not intended for diagnosis, but for early detection.

This birds-eye artifact may be implemented as a lightweight dashboard, a summary report generated at the end of each pipeline run, a standardized status output consumed by downstream systems. The key requirement is that it is interpretable by individuals who are not intimately familiar with the pipeline's internal implementation.

If discrepancies are observed, such as unexpected outputs, misaligned visuals, or operational failures, the data scientist must not apply ad hoc fixes. Each discrepancy must be traced back to the framework step in which the relevant decision was made and addressed at that level. This traceability ensures that iteration strengthens the framework rather than bypassing it.

By the conclusion of Step 6, the modernized pipeline must:

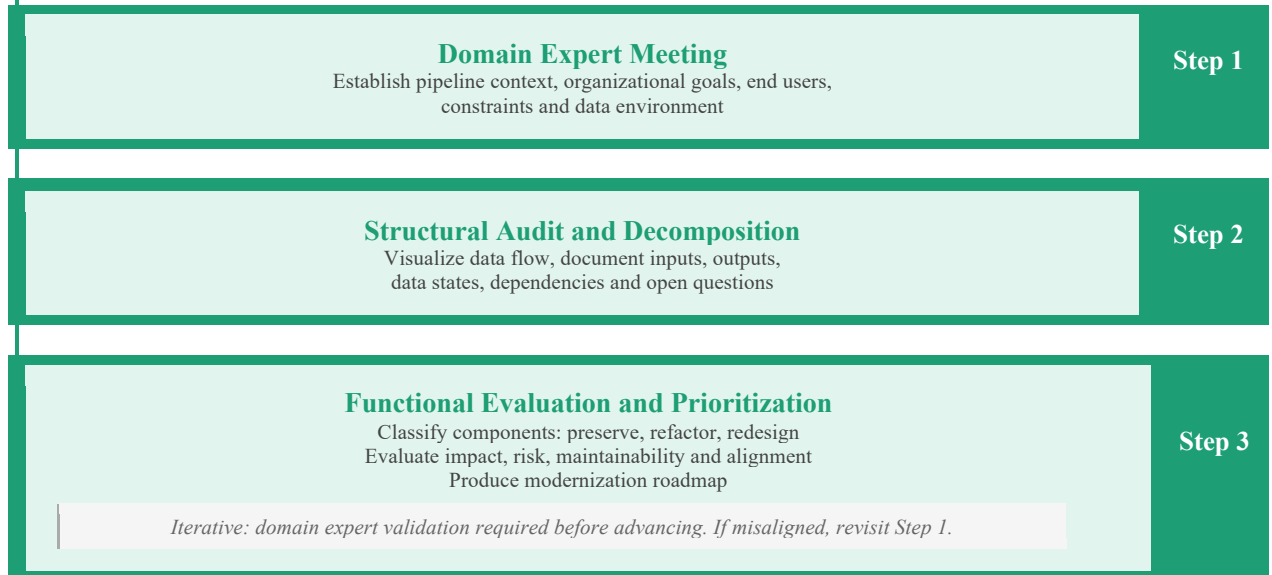
- execute through a standardized and reproducible mechanism,
- be runnable and understandable by another data scientist,
- expose a concrete birds-eye view of system health,
- and have been validated against the analytical, communicative, and operational intent defined by the framework.

At this point, modernization is considered complete not because the pipeline runs, but because it runs predictably, transparently, and observably. This distinction ensures that the framework produces systems that remain usable, maintainable, and interpretable beyond the initial implementation effort.

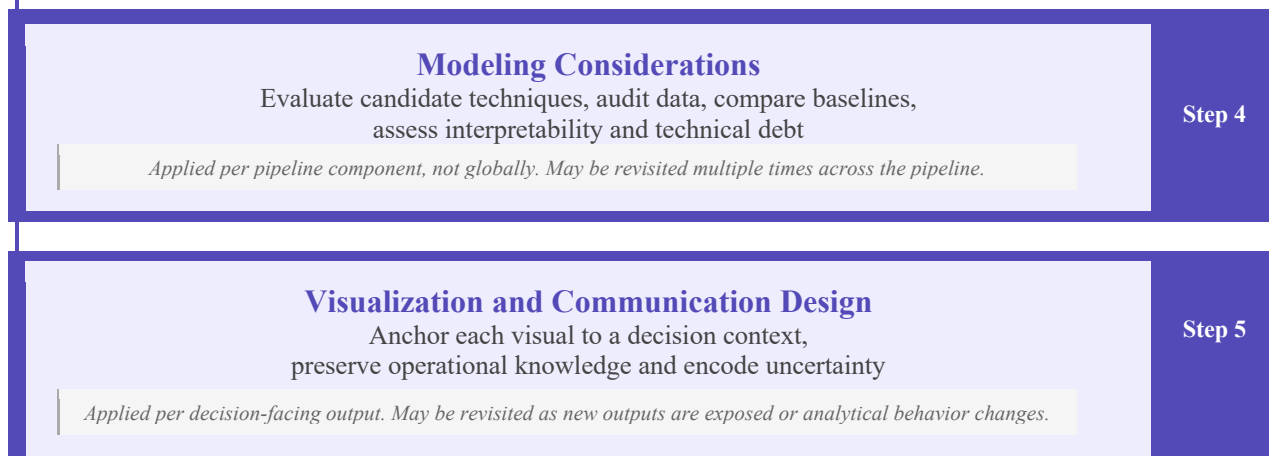
The Data Pipeline Modernization Framework Diagram

A six-step practitioner framework for modernizing legacy data pipelines in alignment with domain expertise and organizational context.

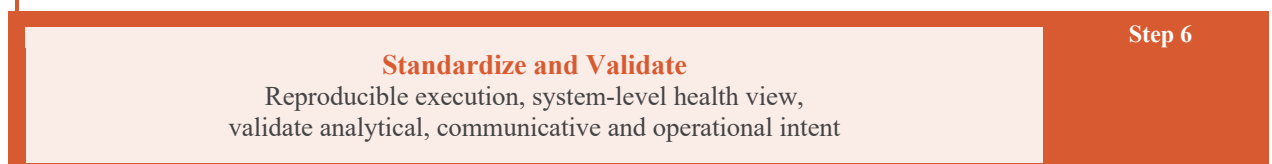
PHASE 1: UNDERSTANDING



PHASE 2: ANALYSIS AND DESIGN



PHASE 3: IMPLEMENTATION



CHAPTER 6: APPLICATION AND TECHNICAL SYNTHESIS OF THE MANUFACTURING ANALYTICAL PIPELINE

The modernization framework developed in earlier chapters was applied to a real operational analytical system supporting production monitoring and short-term capacity planning within a manufacturing environment. This chapter presents a comprehensive technical synthesis of that system. Rather than listing components independently, it narrates the full process of building, restructuring, and stabilizing the pipeline as an engineered artifact. The objective is to provide a complete architectural and analytical overview of the project, abstracted from proprietary details but faithful to structural and mathematical reality.

The pipeline was designed to transform raw operational production records into decision-support outputs through a sequence of descriptive, predictive, and interpretive stages. At its core, the system integrates daily performance measurement, capacity aggregation, short-horizon forecasting, comparative ranking, and interactive visualization within a reproducible computational framework.

The process begins with ingestion of structured production logs containing timestamped output quantities, operational unit identifiers, and indicators of downtime or interruption. Because the manufacturing environment operates continuously while decisions are made on daily cadence, the first architectural decision was to normalize all computations to daily granularity. Temporal aggregation therefore becomes the foundational

transformation upon which all subsequent analysis depends. Timestamped production records are grouped by date and operational unit, and aggregate daily output is computed. This normalization establishes the canonical time resolution for the entire pipeline and ensures consistent dimensionality across downstream components.

From these normalized records, the pipeline computes daily Capacity Fill Rate (CFR), a performance efficiency metric representing realized production relative to effective operational capacity. Structurally, the metric is defined as:

$$CFR_d = \frac{\text{Realized Output}_d}{\text{Effective Hours}_d}$$

At the batch level, CFR is computed as units produced per run hour and aggregated to daily values using the median, ensuring robustness to skewed batch-level performance distributions and preventing isolated high-performing runs from distorting typical performance.

Effective capacity is not a raw theoretical maximum. It incorporates adjustments for operational constraints such as downtime, planned maintenance, or partial-day interruptions. These adjustments reflect accumulated domain expertise and were preserved during modernization. Rather than rewriting the logic, modernization surfaced and documented these heuristics while isolating them structurally from unrelated transformations. This preservation ensured continuity of trusted descriptive metrics while improving architectural transparency.

The output of this stage is a structured dataset indexed by date and operational unit, containing realized output, adjusted capacity, and computed CFR. This dataset serves two purposes: comparative ranking and contextual visualization. Importantly, the modernization process clarified its schema explicitly, documenting field definitions and dependencies to prevent silent coupling across scripts.

Parallel to CFR computation, daily capacity metrics are aggregated to provide a structural view of production potential. Daily capacity is computed as:

$$\text{capacity}_d = \text{CFR}_d \times \text{Effective Hours}_d$$

This stage consolidates adjusted capacity measures across operational units and enforces temporal continuity. The resulting time-indexed series forms the mathematical foundation for forecasting. During modernization, a key architectural clarification was introduced: descriptive aggregation must be completed entirely before predictive modeling is invoked. This separation prevents entanglement between cleaning logic and forecasting assumptions.

Forecasting constitutes the most analytically structured stage of the pipeline.

Let C_t denote adjusted daily capacity at time t . Future capacity is estimated using Holt's linear exponential smoothing, which decomposes the time series into level and trend components:

$$L_t = \alpha C_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Forecasts are generated as:

$$\hat{C}_{t+h} = L_t + \sum_{i=1}^h \phi^i T_t$$

where α and β are smoothing parameters selected through grid search and ϕ is a damping parameter that enforces mean-reversion over the planning horizon. This formulation reflects the operational reality that capacity fluctuates around stable levels rather than exhibiting unbounded growth.

To quantify forecast uncertainty, residuals from the fitted model are resampled using a block bootstrap procedure. This preserves temporal autocorrelation in production behavior and generates a distribution of plausible future capacity trajectories rather than a single deterministic forecast.

Forecast outputs are translated into decision-ready capacity plans using an asymmetric cost minimization framework consistent with the newsvendor model. Let q denote the planned capacity commitment and D denote realized capacity. The cost function is defined as:

$$C(q, D) = c_{\text{over}} \cdot \max(0, q - D) + c_{\text{under}} \cdot \max(0, D - q)$$

where:

- c_{over} is the cost of **over-committing capacity** (planning more than can be realized),
- c_{under} is the cost of **under-committing capacity** (failing to utilize available capacity).

The first term applies when the planned capacity exceeds realized capacity, while the second applies when realized capacity exceeds the plan. This formulation explicitly encodes the operational trade-off between unmet commitments and unused production potential.

Under this formulation, the optimal planned capacity is given by the quantile:

$$q^* = F^{-1} \left(\frac{c_{\text{over}}}{c_{\text{over}} + c_{\text{under}}} \right)$$

where F represents the empirical distribution of simulated capacity outcomes generated through the bootstrap procedure. In this application, over-committing capacity is assigned a higher penalty than under-committing, reflecting the operational priority of avoiding unmet commitments. This results in a conservative planning strategy grounded in quantified uncertainty.

Following forecasting, the pipeline performs comparative ranking of CFR across operational units. This stage operates cross-sectionally, computing relative performance positions for each time-period. Structurally, ranking is dependent solely on descriptive CFR outputs and remains isolated from forecasting components. This separation was

preserved intentionally to prevent predictive artifacts from influencing descriptive benchmarking.

The visualization layer consumes structured outputs from descriptive, predictive, and ranking stages. Implemented through a Streamlit application (Streamlit Inc., n.d.), the interface loads precomputed datasets rather than executing analytical logic dynamically. This architectural layering enforces a clear boundary between computation and presentation. Within the interface, historical and forecasted capacity values are visually distinguished, axis scaling is standardized, and encoding conventions remain stable across views. The application provides interactive filtering capabilities without mutating underlying analytical artifacts. By treating the interface as a presentation layer rather than a computational engine, modernization preserved reproducibility and reduced risk of silent logic drift.

Execution of the pipeline is orchestrated through a unified entry-point script that governs stage ordering and dependency resolution. Prior to modernization, execution order was inferred through domain knowledge, requiring the pipeline to be owned and operated by a single user. Modernization formalized this ordering, clarified intermediate outputs, and externalized configuration elements. The result is a deterministic, reproducible execution pathway capable of running in a clean environment without implicit assumptions. Stage-level visibility was introduced to improve traceability and reduce debugging ambiguity.

A comparison between the legacy and modernized architectures reveals that modernization altered structure rather than purpose. Stage boundaries are now explicit. Modeling logic is isolated and governed. Configuration parameters are externalized.

Execution is standardized. Visualization distinguishes descriptive from predictive outputs. Yet the core descriptive metrics trusted by domain stakeholders remain intact.



Figure 7: Modernized Pipeline Architecture Diagram

These figures illustrate the transformation from implicit coupling to explicit modular staging.

The project, considered as a whole, demonstrates that building a production-grade analytical pipeline requires more than assembling scripts. It requires deliberate decisions about data representation, schema stability, dependency isolation, modeling governance, execution determinism, and presentation transparency. The modernization framework-imposed discipline on these decisions, but the pipeline itself represents the realized artifact of that discipline.

In summary, the pipeline integrates descriptive performance metrics, aggregated capacity measures, governed short-horizon forecasting, uncertainty-aware scenario generation, decision optimization, comparative ranking, and interactive visualization within a reproducible computational architecture. The modernization process did not seek to replace operational knowledge but to stabilize and clarify it. The result is a system that preserves domain-aligned logic while achieving improved maintainability, interpretability, and reproducibility.

The modernization of the manufacturing analytical pipeline can be evaluated along three primary dimensions: structural integrity, operational continuity, and analytical governance. These dimensions align directly with the modernization objectives defined earlier in the thesis.

The most substantial improvement achieved through modernization is architectural clarity. In the legacy system, stage boundaries were implied by script ordering and

naming conventions. Dependencies existed but were not formally documented.

Configuration parameters were embedded within scripts, and modeling logic was partially intertwined with transformation processes.

In the modernized pipeline, stage responsibilities are explicitly defined. Descriptive computation, aggregation, forecasting, ranking, and visualization are structurally isolated. Input–output contracts between stages are documented. Modeling logic is separated from data cleaning logic. Configuration parameters are externalized. Execution order is governed through a single, deterministic entry point.

These changes reduce coupling and lower the probability that modifications in one component will produce unintended downstream effects. Structural clarity also improves auditability; practitioners can trace how raw data propagates through transformation, modeling, and presentation layers without reverse-engineering script behavior. While no numerical “modularity score” is computed, the qualitative shift from implicit to explicit architecture represents a measurable improvement in maintainability.

A critical constraint throughout modernization was preservation of domain trust. The descriptive CFR computation stage, which operational stakeholders rely upon daily, was not fundamentally rewritten. Instead, its logic was surfaced, clarified, and stabilized. No core descriptive metric was altered without validation. This ensured that modernization did not disrupt decision workflows. The pipeline remains operationally aligned with stakeholder expectations, and outputs remain interpretable within established business context. This continuity demonstrates that modernization need not be disruptive to be

meaningful. Structural improvements were introduced without invalidating historical performance interpretation.

Forecasting was transformed from an embedded computational extension into a governed analytical layer. Modeling assumptions are now explicitly documented. Parameters are configurable rather than hard-coded. Forecasted outputs are clearly distinguished from historical data. Where applicable, uncertainty bounds are preserved in the output schema.

This governance reduces hidden technical debt and prevents predictive outputs from being misinterpreted as deterministic extensions of observed trends. It also ensures that future modeling changes can be implemented without entangling transformation logic. The modernization therefore strengthens analytical discipline rather than expanding analytical complexity.

The introduction of a unified execution entry point and clarified dependency flow significantly improves reproducibility. The pipeline can be executed end-to-end in a controlled sequence without reliance on implicit ordering or contextual memory.

Stage-level transparency and standardized output writing reduce onboarding friction for future practitioners. The system is now portable in a way the legacy pipeline was not. Reproducibility is not merely a technical convenience; it is foundational to long-term maintainability and collaborative development.

The following dimensions summarize the transformation achieved:

Dimension	Legacy Pipeline	Modernized Pipeline
Stage Definition	Implicit, script-based	Explicit, documented boundaries
Modeling Isolation	Model Entanglement Causing debugging issues	Fully isolated from transformations
Configuration	Embedded in Excel Documents, and SQL Queries	Externalized and Structured
Execution	Assumed sequential	Formalized entry point
Visualization	Mixed visuals of descriptive and prescriptive findings	Explicit separation and labeling for better interpretation
Domain Heuristics	Embedded and undocumented	Preserved and documented
Reproducibility	Environment-dependent	Clean-environment executable

These changes demonstrate that modernization primarily altered architectural governance rather than analytical objectives.

This evaluation emphasizes structural and process improvements rather than predictive performance gains. While predictive accuracy was not the primary evaluation objective, the integration of forecasting, uncertainty modeling, and decision optimization ensured that outputs were not only analytically valid but operationally actionable. Future work may integrate structural and performance metrics in a controlled manner to further validate long-term impact.

Viewed as an engineering project, the modernized pipeline represents the disciplined evolution of a domain-embedded analytical system into a modular, reproducible, and governed architecture. The transformation did not require wholesale replacement, nor did it depend on advanced algorithmic complexity. Instead, it relied on explicit boundary definition, configuration externalization, modeling isolation, and presentation transparency.

The pipeline now functions not only as a decision-support tool but as a structured analytical asset. Its architecture supports future adaptation without requiring reconstruction.

In this sense, the modernized system serves as empirical validation of the modernization framework. The framework did not prescribe abstract principles; it produced a stable, operational artifact capable of sustaining analytical integrity under real-world constraints.

CONCLUSION

This thesis addressed a structural gap in contemporary data science practice: the absence of a prescriptive, experimentally validated methodology for modernizing legacy data pipelines. While the discipline has matured rapidly in modeling sophistication and infrastructure scalability, formal guidance for disciplined transformation of inherited systems remains limited. Organizations frequently operate pipelines that evolved incrementally, embedding domain heuristics, undocumented transformations, and tightly coupled components. Modernization in such contexts is often approached through ad hoc refactoring or wholesale replacement, neither of which provides structured governance.

The central objective of this research was to develop and validate a modernization framework capable of guiding controlled transformation of legacy data pipelines while preserving embedded domain knowledge. Using Design Science Research as the methodological foundation, the framework was constructed iteratively within a real manufacturing analytical system. Each candidate modernization step was proposed, implemented, evaluated using structured process metrics, and either incorporated or removed based on empirical outcomes.

Through this process, 6 sequential stabilization stages emerged:

1. Meet with Domain Expert
2. Explicit Structural Audit and Visual Decomposition
3. Functional Evaluation and Modernization Target Identification
4. Modeling and Machine Learning Considerations

5. Visualization and Communication Design

6. Standardize Implementation, Reproducibility, and System-Level Validation

These stages were not imposed a priori but discovered through iterative refinement. A proposed analytical readiness stage was removed after evaluation revealed redundancy, reinforcing the principle that modernization components must demonstrate distinct structural value. This was not an isolated case; multiple candidate stages were consolidated, revised, or excluded over successive iterations to eliminate duplication, reduce unnecessary abstraction, and ensure measurable structural contribution. Similarly, several retained stages underwent refinement before reaching their final form, reflecting an iterative process of aligning each component with the framework's objectives of modularity, reproducibility, and operational alignment.

Application of the framework to the manufacturing pipeline demonstrated that modernization is fundamentally architectural rather than algorithmic. The transformation did not depend on increased modeling complexity. Instead, it achieved structural clarity, modeling isolation, configuration externalization, deterministic execution, and improved interpretability while preserving trusted descriptive metrics. The resulting system functions as a governed analytical asset rather than a collection of loosely connected scripts.

CONTRIBUTIONS

This research formalizes modernization as a finite and disciplined protocol distinct from lifecycle methodologies such as CRISP-DM. Whereas CRISP-DM governs pipeline

construction, the framework developed here governs pipeline transformation. The distinction clarifies an important methodological gap within the discipline.

This research demonstrates the integration of modular design principles, cognitive transparency considerations, and Design Science evaluation into a cohesive modernization procedure. By grounding evaluation in process-oriented metrics rather than predictive performance alone, the research avoids metric recursion and emphasizes architectural integrity.

This research provides empirical evidence that legacy pipelines can be stabilized through incremental structural governance without erasing domain knowledge. This finding challenges the common assumption that starting from scratch is the most efficient modernization strategy.

The study also has limitations. Validation occurred within a single operational context, and while the structural characteristics observed are common across industries, further empirical application across domains would strengthen generalizability. Additionally, evaluation emphasized structural and governance improvements rather than quantitative business performance gains. Future research may integrate structural metrics with performance outcomes under controlled experimental design.

FUTURE WORK

Future research should focus on the empirical validation of this framework across diverse organizational contexts to determine how domain complexity dictates modernization sequencing. While the Data Pipeline Modernization Framework was developed and

applied within a single organizational context, its step-by-step structure and emphasis on modularity, domain grounding, and operational usability suggest meaningful potential for generalization across a range of legacy pipeline modernization efforts. The framework's reliance on iterative collaboration with a domain expert, however, surfaces one limitation that warrants further development. In its current form, Step 1 assumes a single domain expert as the primary point of contact, with the structured conversation guide designed accordingly. In practice, many organizations distribute pipeline knowledge across multiple users, a data engineer familiar with the technical infrastructure, an operations manager who interprets the analytical outputs, and a business analyst who originally defined the pipeline's objectives may each hold distinct and non-overlapping knowledge of the same system. In such cases, the current conversation guide may be insufficient for navigating multiple expert perspectives, reconciling conflicting accounts of pipeline intent, or synthesizing domain knowledge that is fragmented across roles. Future work should therefore revisit Step 1 to develop a more robust multi-stakeholder engagement protocol. Strengthening this component of the framework would expand its practical applicability and better reflect the organizational complexity that characterizes many real-world pipeline modernization efforts. Additionally, the formalization of quantitative structural metrics, such as coupling indices or configuration externalization measures, would provide a rigorous objective layer to supplement qualitative evaluation. A third trajectory involves the development of automated diagnostic tooling to assist practitioners in conducting structural audits and identifying pipeline entanglement.

Furthermore, extending this framework to production machine learning environments is essential, as factors such as model drift, data versioning, and retraining governance introduce unique dimensions of structural complexity.

As the field of Data Science matures, long-term sustainability will depend less on marginal algorithmic gains and more on disciplined architectural governance. Modern infrastructure alone cannot remediate fundamentally fragile analytical workflows. This thesis provides a structured methodology for the responsible evolution of legacy systems, ensuring that domain expertise is preserved while improving maintainability and interpretability. Consequently, this research establishes pipeline modernization as a distinct and necessary area of methodological development within the broader Data Science discipline.

REFERENCES

Adar, E., Dontcheva, M., Fogarty, J., & Weld, D. S. (2021). *Automatic Y-axis Rescaling in Dynamic Visualizations*. NSF Public Access Repository.

Assunção, W. K. G., Marchezan, L., Arkoh, L., Egyed, A., & Ramler, R. (2025). Contemporary software modernization: Strategies, driving forces, and research opportunities. *ACM Transactions on Software Engineering and Methodology*, 34(5), Article 142. doi.org

Basili, V. R., Caldiera, G., & Rombach, H. D. (1994). The Goal Question Metric approach. In *Encyclopedia of Software Engineering* (Vol. 1, pp. 528–532). Wiley.

Biswas, S., Wardat, M., & Rajan, H. (2022). The art and practice of data science pipelines: A comprehensive study of data science pipelines in theory, in-the-small, and in-the-large. *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*, 1434–1446.

Camm, J. D., Fry, M. J., & Shaffer, S. M. (2023). Data science and decision science skills: Are they different and does it matter? *Harvard Data Science Review*, 5(3), Article 32.

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

Cleveland, W. S. (2001). Data science: An action plan for expanding the technical areas of the field of statistics. *International Statistical Review*, 69(1), 21–26.

Endsley, M. R. (2011). *Designing for Situation Awareness: An Approach to User-Centered Design* (2nd ed.). CRC Press.

Few, S. (2006). *Information Dashboard Design: The Effective Visual Communication of Data*. O'Reilly Media.

Foidl, H., Golendukhina, V., Ramler, R., & Felderer, M. (2024). Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *Journal of Systems and Software*, 207, 111855. doi.org

Gleicher, M. (2011). Considerations for Visualizing Model Outputs. *IEEE Transactions on Visualization and Computer Graphics*, 17(12), 2260–2269.

Green, T. R. G., & Petre, M. (1996). Usability analysis of visual programming environments: A “Cognitive Dimensions” framework. *Journal of Visual Languages & Computing*, 7(2), 131–174.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. doi.org

Hullman, J. (2019). Why authors don't visualize uncertainty. *IEEE Transactions on Visualization and Computer Graphics*, 26(1), 130–139.

IBM. (2013). *The CRISP-DM methodology*. IBM Analytics.

Kadakia, K. T., & DeSalvo, K. B. (2023). Transforming public health data systems to advance the population's health. *The Milbank Quarterly*, 101(S1), 674–699. doi.org

Kay, M., Kola, T., Hullman, J. R., & Munson, S. A. (2016). When (ish) is my bus? User expectations of timeliness in real-time forecasting systems. *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, 5035–5047.

McGowan, L. D., Peng, R. D., & Hicks, S. C. (2023). Design principles for data analysis. *Journal of Computational and Graphical Statistics*, 32(4), 1192–1207.

Munzner, T. (2014). *Visualization Analysis and Design*. CRC Press.

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. doi.org

Sanders, E. B. N., & Stappers, P. J. (2008). Co-creation and the new landscapes of design. *CoDesign*, 4(1), 5–18. doi.org

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Proceedings of the 29th International Conference on Neural Information Processing Systems (NIPS '15)*, 2503–2511.

Sperrle, F., El-Assady, M., Koch, S., Keim, D., & Schreck, T. (2021). VIAL: A Generalized Process Model for Visual Interactive Analysis of Machine Learning. *IEEE Computer Graphics and Applications*, 41(3), 61–71.

Streamlit Inc. (n.d.). *Streamlit documentation*. <https://docs.streamlit.io/>

Sweller, J. (2011). Cognitive load theory. In J. P. Mestre & B. H. Ross (Eds.), *The psychology of learning and motivation: Cognition in education* (pp. 37–76). Elsevier Academic Press. <https://doi.org/10.1016/B978-0-12-387691-1.00002-8>

Tukey, J. W. (1962). The future of data analysis. *Annals of Mathematical Statistics*, 33(1), 1–67.

Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.

APPENDIX A: PROPOSED FRAMEWORK PROCEDURE SCORE SHEET

**Note: the final step incorporated into the thesis will differ slightly due to edits made to the wording of the steps after the acceptance of the step into the framework.*

MEET WITH DOMAIN EXPERT SECTION

Proposed Framework Procedure
Meet with Domain Expert: In this meeting it is essential to pinpoint the areas of the data pipeline your domain expert finds important and the areas in which the pipeline is excelling but the points in which the pipeline needs to be improved and modernized. To do so it is important to target your questions to streamline the meeting and target the answers. You need to be following the WHY, WHAT, and WHERE Method. In which you are asking WHY are we deciding to modernize this data pipeline? WHAT is the ultimate goal of this pipeline? What are the portions of the pipeline that you wish to see preserved? And what are the portions of the pipeline you want to prioritize in the modernization process? WHERE is this pipeline used within the broader data infrastructure used.

Effectiveness	Score: 2
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: Effective in giving the data scientist an overview of the pipeline but really needed more guidance in how to extract the pertinent information such as what are the steps if any and what are the resources available to the data scientist.	

Framework Alignment	Score: No
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Does not guide the data scientist to evaluate the pipeline through the lens fo steps. Need to improve the overall approach to trying to audit and asses the pipeline	

Data Scientist Usability	Score: 4
--------------------------	----------

Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.
Observation: Easy to apply this step allow the step is in need to repair else where

Operational Validity	Score: 3
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Enjoyed the conversation and meeting surrounding the pipeline. Would like to ensure the meetings prior to work starting on the pipeline is kept to a minimum this step did not guide the data scientist to capture all the needed information therefore needs to be improved.	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: Easy to implement within the modernization process and did not increase the complexity of the work.	

Reject or Accept Hypothesis
Notes: The step is rejected and needs to be revised before approval. Suggested Rework Criteria: overall the step needs to be more detailed and needs to emphasize modularity and usability within the pipeline and how these two aspects are crucial to the design of the pipeline from start to finish. ** Need to add in portions to cite from previous papers I am thinking the papers for the process metrics need to be included.

Proposed Framework Procedure

Meeting with your Domain Expert before starting to modernize the data pipeline is crucial because contextual understanding is the foundation of effective data work; technical redesigns that are disconnected from domain goals often fail to generate meaningful improvements **Biswas et al. (2022)**. Often, the pitfalls of data pipelines stem from their inability to fulfill the objectives initially defined by the Domain Expert within the operational constraints and infrastructure of the organization **Green and Petre (1996)**. By prioritizing the contextualization of the pipeline within the broader data infrastructure, the organizational limitations, and the goals articulated by the Domain Expert, the data scientist is better equipped to modernize a system whose structure is rooted in the practicality of the original problem statement and aligned with real operational needs.

As a data scientist it may be tempting to begin your audit of the data pipeline independently as part of the first step of the CRISP-DM framework **(IBM, 2013)**. However the modernization of a data pipeline has distinctive properties that require the data scientist practitioner to work in tandem with the Domain Expert to both gain business understanding as well as guide a formalized conversation surrounding the overall goals of the modernization process.

As previously stated data pipelines are considered pipelines-in-the-small **Biswas et al. (2022)** in which they have been designed to carry out a very distinct set of steps to solve a very specific problem. However, oftentimes pipelines-in-the-small lack modularity and documentation. **Biswas et al. (2022)**. Because of this the way in which data scientists need to approach a conversation with the domain expert requires them to create questions that guide the Domain Expert through an explanation of the legacy data pipeline that:

1. Allows the data scientist to understand key aspects of the pipeline in which the Domain Expert are content with the current functionality and output
2. Ensures the data scientist has an in-depth understanding of the current workflow of the legacy data pipeline

3. Highlights the key aspects of the legacy data pipeline that need to be improved and where these aspects of the pipeline fall within the larger pipeline and overall data workflow of the organization.

Legacy data pipelines need modernization for a plethora of reasons whether it is simply incapable of producing the correct analytical insight for the organization, not compatible with the current infrastructure or shows an inability to be maintained by a large portion of the organization who rely on this pipeline. To target these issues it is important pipelines and the modernization process of legacy data pipelines prioritize modularity. Modularity describes the practice of designing a system as distinct parts each responsible for a set of functions. The property of modularity is a foundational aspect of software engineering practices set forth to ensure code infrastructure is resistant to bugs and change **Parnas (1972)**.

When meeting with your Domain Expert to discuss the current data pipeline, it is essential for you, as the data scientist, to structure the conversation around a clear, step-by-step walkthrough of the pipeline's functions. Although a legacy pipeline may lack explicit modularity, it still follows a series of implicit stages, collecting, formatting, analyzing, and distributing data, that are fundamental to its operation. Developing a deep understanding of these stages not only clarifies how the legacy pipeline currently performs but also equips you to design a modernization strategy that preserves and enhances modularity throughout the new system process.

Operational Usability is another important aspect to prioritize when first meeting with your Domain Expert. Through discussions it needs to be made clear who within the organization will be interacting with your modernized data pipeline. This answer will inform how you will structure not only your documentation of the overall pipeline but also the finalized analytical output of the data pipeline. Understanding who your end-user is is exceedingly important in the design process **McGowan et al. (2023)**. This step is crucial to ensure usability along with maintainability within the organization.

An observed pitfall of legacy data pipelines is their inability to be followed, used and managed by a large portion of the organization. This is an inherent issue within legacy data pipelines leading to their obsolescence. In your discussion with your Domain Expert make a clear decision about the end user of this pipeline to ensure its longevity and utility within the organization.

Finally, data scientists must develop a clear understanding of the resources, constraints, and expectations associated with the modernized data pipeline. This involves identifying the current implementation environment (e.g., the programming language, platform, or orchestration system in which the legacy pipeline operates), as well as the target environment in which the modernized version is expected to be delivered. Equally important is assessing the availability, accessibility, and reliability of the data currently flowing through the system.

This stage is *not* the time for a deep exploratory dive into all available organizational data. Doing so may lead to misguided assumptions about the pipeline’s potential, particularly when portions of relevant data are incomplete, unavailable, or not yet collected. Overanalyzing the data too early can create false confidence in the pipeline’s apparent capacity or its integration within the broader data infrastructure. Instead, data scientists should focus their inquiry on the specific inputs and outputs currently being processed by the pipeline. This targeted approach allows for a deeper understanding of how data flows and interacts within the system, without constructing premature or inaccurate narratives about its limitations.

It is also important to recognize that the data scientist’s role is not to simply replicate the existing pipeline under a new structure or technology stack. The partnership between Domain Expert and data scientist should instead aim to reimagine the pipeline’s design by identifying where untapped data sources or latent metadata could enhance its performance and functionality. In many legacy systems, the data required to achieve modernization already exists, either in underutilized data stores, embedded metadata, or parallel processes awaiting integration. **Kadakia and DeSalvo (2023)**

At the conclusion of this meeting with the Domain Expert, the data scientist should walk away with a comprehensive understanding of the legacy pipeline’s strengths and weaknesses, a clear articulation of its overarching functional goal, an identification of the intended end users and their requirements, and a prioritized outline of the specific stages that require improvement or redesign. Most importantly, the data scientist will have established a shared contextual foundation with the Domain Expert, clarifying the pipeline’s operational purpose, technical dependencies, and the organizational expectations guiding modernization. This foundational understanding serves as the bridge between the existing system and its envisioned future state, ensuring that any subsequent technical work is grounded in practical reality rather than speculative assumptions about data, infrastructure, or capability.

Effectiveness	Score: 5
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: Yes extremely effective in the main goal of preserving business acumen and operational validity in the pipeline	

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Promotes maintainability of the pipeline	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: In depth understanding of how to interact with the DE. Could be improved via writing edits	

Operational Validity	Score: 5
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Domain Expert Felt this step was adequate in explaining the current state of the pipeline	

Feasibility	Score:
Determines whether the step is practical to implement without excessive complexity	
Observation:	

Reject or Accept Hypothesis
Notes: Accept into Framework (needs to be revised for readability)

Citations to Be Used in this Portion of the Framework
Biswas, S., Wardat, M., and Rajan, H. (2022). <i>The art and practice of data science pipelines: A comprehensive study of data science pipelines in theory, in-the-small, and in-the-large</i>. In Proceedings of the 44th International Conference on Software Engineering (ICSE '22) (pp. 1434–1446). ACM.

Green, T. R. G., and Petre, M. (1996). *Usability analysis of visual programming environments: A “Cognitive Dimensions” framework*. *Journal of Visual Languages & Computing*, 7(2), 131–174.

IBM. (2013). *The CRISP-DM methodology*. IBM Analytics.

Kadakia, K. T., and DeSalvo, K. B. (2023). *Transforming public health data systems to advance the population’s health*. *Milbank Quarterly*, 101(S1), 674–699.
<https://doi.org/10.1111/1468-0009.12618>

McGowan, L. D., Peng, R. D., and Hicks, S. C. (2023). *Design principles for data analysis*. *Journal of Computational and Graphical Statistics*, 32(4), 1192–1207.

Parnas, D. L. (1972). *On the criteria to be used in decomposing systems into modules*. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>

~~STRUCTURAL DECOMPOSITION OF LEGACY PIPELINE~~ EXPLICIT STRUCTURAL AUDIT AND VISUAL DECOMPOSITION OF LEGACY DATA PIPELINE

Proposed Framework Procedure

Before beginning the development of a modernization framework, the data scientist should review the existing data pipeline to gain a general sense of how data flows through the system. After meeting with the Domain Expert, the data scientist independently inspects available datasets, scripts, and outputs to identify obvious issues or inefficiencies.

This audit is conducted primarily through exploratory examination of the data and a surface-level review of the codebase, without formally documenting pipeline stages or dependencies. The goal of this step is to become familiar with the pipeline’s behavior and identify areas that appear outdated or error-prone.

Insights from this audit are used to inform later modernization decisions, such as tool replacement or restructuring. While this approach allows the data scientist to move quickly into technical redesign, it assumes that the structure of the pipeline and its outputs are sufficiently understood through inspection alone.

Effectiveness

Score: 2

Assesses whether the framework step contributes meaningfully to the modernization goal
Observation: This step is effective in providing the data scientist with a high-level overview of the existing pipeline. However, it lacks sufficient guidance on how to extract the most pertinent information required for modernization. Specifically, the step does not direct the data scientist to explicitly identify pipeline stages, functional boundaries, or available technical and organizational resources. As a result, the audit remains descriptive rather than diagnostic, limiting its contribution to informed modernization decisions.

Framework Alignment	Score: No
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: The step does not encourage the data scientist to assess the pipeline through a modular or stage-based lens. Without explicit emphasis on decomposing the pipeline into functional components, the approach risks reinforcing existing structural issues and does not support maintainable or extensible pipeline architecture. The overall audit methodology requires refinement to align with modular design principles.	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: The step is straightforward and easy for a data scientist to apply in practice. However, its usability is undermined by a lack of structured guidance, which reduces consistency across applications. While accessible, the step requires revision to ensure it produces comparable and reusable outputs across different pipeline modernization efforts.	

Operational Validity	Score: 3
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: The step facilitates productive initial conversations with the Domain Expert and supports early contextual understanding. However, it does not adequately guide the data scientist in capturing all necessary operational details	

within a limited number of meetings. As a result, important contextual information may be overlooked, and the process risks inefficiency. Additional structure is needed to ensure alignment with business needs while minimizing unnecessary pre-development overhead.

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: The step is easy to implement within the modernization process and does not introduce additional technical or organizational complexity. Its simplicity makes it highly feasible; however, this feasibility comes at the cost of analytical rigor and depth.	

Reject or Accept Hypothesis
Notes: This step is rejected in its current form and requires revision before approval. While feasible and easy to execute, it does not sufficiently emphasize modularity, structured assessment, or systematic information capture. Suggested Rework Criteria: The revised step should introduce clearer guidance for decomposing the pipeline into functional stages, explicitly prioritize modularity and usability, and articulate how these design principles support effective pipeline modernization from initial assessment through final deployment. Additionally, process-oriented evaluation metrics from prior literature should be incorporated to strengthen the methodological grounding of this step.

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
Before constructing the modernization framework, the data scientist conducts a structured audit of the existing data pipeline following an initial meeting with the Domain Expert. This step focuses on documenting the primary functions of the pipeline by identifying major stages such as data ingestion, transformation, analysis, and output generation. During this process,

the data scientist works alongside the Domain Expert to clarify the purpose of each stage, the data inputs and outputs involved, and any known limitations or manual interventions required. The audit also includes a preliminary inventory of technical resources, including programming languages, platforms, and tools currently used to support the pipeline. While this step introduces an explicit stage-based perspective, it stops short of formally defining modular boundaries or standardized interfaces between components. As a result, the audit improves transparency and shared understanding but does not yet fully support systematic comparison or redesign across different pipeline contexts.

Effectiveness	Score: 4
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: This step meaningfully improves the modernization process by introducing a structured, stage-based audit of the legacy pipeline. By explicitly identifying major pipeline functions and associated resources, the data scientist gains clearer insight into where modernization efforts should be focused. However, the absence of formal criteria for evaluating stage performance or interactions limits the depth of actionable conclusions that can be drawn.	

Framework Alignment	Score: 3
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: The step begins to promote modular thinking by encouraging decomposition into functional stages. However, it does not yet require the definition of clear interfaces, dependencies, or responsibility boundaries between stages. As a result, the approach partially aligns with modular design principles but does not fully support maintainability or extensibility.	

Data Scientist Usability	Score: 3
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: The step provides clearer guidance than the initial version and can be followed consistently across similar pipelines. Nevertheless, the lack of standardized templates or documentation artifacts introduces variability in how different data scientists might perform and record the audit, reducing repeatability.	

Operational Validity	Score: 4
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Continued collaboration with the Domain Expert ensures that the audit remains grounded in operational reality. The focus on pipeline purpose and known constraints improves alignment with business needs. However, the step does not yet ensure that all relevant contextual details are captured efficiently or prioritized according to operational impact.	

Feasibility	Score: 4
Determines whether the step is practical to implement without excessive complexity	
Observation: The step remains practical and implementable within typical modernization timelines. While it requires more effort and coordination than the initial version, it does not introduce prohibitive complexity or tooling requirements.	

Reject or Accept Hypothesis
Notes:

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
Before beginning the development of a modernization framework, the data scientist should conduct a deliberate and structured review of the existing data pipeline to develop an initial understanding of how data flows through the system. This review is not intended to produce redesign decisions, but rather to establish technical orientation and contextual grounding. Following an initial meeting with the Domain Expert, the data scientist independently inspects available datasets, scripts, intermediate artifacts, and analytical outputs to identify obvious inefficiencies, inconsistencies, or breakdowns in pipeline behavior. However, independent

inspection alone is insufficient; without domain grounding, technical observations risk misrepresenting the intent, constraints, or operational role of the pipeline (Green & Petre, 1996). Step 1 therefore plays a critical role in anchoring technical analysis in domain expertise and operational context. The resulting pipeline visualization can then be carried forward as a shared reference artifact, supporting more focused and productive discussions with the Domain Expert throughout later stages of the modernization process.

At this stage, the data scientist must create an explicit visualization of the data pipeline. Data science pipelines are frequently designed as pipelines-in-the-small, systems constructed to solve narrowly defined problems and often lacking formal documentation or modular structure (Biswas et al., 2022). As a result, much of the pipeline's logic exists implicitly rather than explicitly. Beginning with the high-level outline developed during discussions with the Domain Expert, the data scientist should construct a visual representation that traces the flow of data from its point of origin through each transformation, analytical computation, and output stage. This visualization serves as a cognitive artifact that externalizes structure, making dependencies, assumptions, and hidden coupling visible for systematic evaluation.

When the pipeline operates within a broader operational system, such as a supply chain, manufacturing environment, or service workflow, the visualization should extend beyond computational components alone. The data scientist must understand where data originates within the physical or organizational process and where analytical outputs are ultimately consumed. Visualizing these relationships ensures a system-level understanding of the pipeline and reduces the risk of optimizing isolated components without regard for downstream operational impact (Biswas et al., 2022).

Stage-Level Audit Checklist (Required Artifact)

For **each step** identified in the pipeline visualization, the data scientist should create a structured list capturing the following key factors:

- **Input(s):**
What data is consumed at this stage? Specify variables, files, or streams used as inputs.
- **Input Origin:**
Does the input originate from a previous stage within the same pipeline, or is it sourced from an external system or parallel pipeline?
- **Output(s) and Format:**
Where is the data produced at this stage sent next, and in what format is it expected to exist (e.g., database table, CSV file, spreadsheet, API payload)?
- **Current Data State:**
What is the state of the data at this point in the pipeline? For example, is the data raw, partially processed, fully structured, metadata-driven, manually curated, or inconsistently formatted? Documenting data state is critical for identifying sources of quality degradation and uncertainty (Foidl et al., 2024; Sculley et al., 2015).
- **Domain Expert Commentary:**
Record any observations, concerns, or explanations provided by the Domain Expert

regarding this stage, including which components are trusted, which are brittle, and which outputs are operationally meaningful.

- **Open Questions and Uncertainties:**

Identify gaps in understanding, ambiguities, or assumptions uncovered during the audit. These questions should be explicitly tracked and revisited in subsequent touchpoints with the Domain Expert. Maintaining a clear distinction between confirmed knowledge and unknowns reduces the risk of assumption-driven redesign (**Green & Petre, 1996**).

- **Preliminary Modernization Hypotheses:**

Document early observations regarding where improved modularity, clearer interfaces, enhanced data accessibility, or integration of additional data sources may improve pipeline performance. These hypotheses are not yet solutions but serve as inputs to later design and optimization stages.

At each stage of the audit, the data scientist should treat this checklist as a living artifact that evolves through continued engagement with the Domain Expert. The goal is not to exhaustively analyze all available organizational data, but to develop a precise understanding of how data currently flows through the pipeline and how each stage contributes to decision-making. Premature exploration of unrelated data sources can create misleading narratives about pipeline capability and should be avoided at this stage (**IBM, 2013**).

This structured audit explicitly supports modular design by encouraging the decomposition of the pipeline into functionally coherent components with well-defined responsibilities. Modularity is a foundational principle of robust software systems and is essential for creating pipelines that are maintainable, extensible, and resilient to change (**Parnas, 1972**). By identifying functional boundaries, data dependencies, and interfaces early in the framework, the data scientist establishes the structural foundation necessary for principled modernization.

Finally, this step enables the data scientist and Domain Expert to move beyond simple replication of the legacy pipeline. The shared understanding developed through visualization and structured documentation allows both parties to identify underutilized data sources, latent metadata, or parallel processes that could be integrated to enhance pipeline performance and decision relevance. Prior work in public-sector data modernization demonstrates that significant gains often arise from leveraging existing but fragmented data assets rather than introducing entirely new data collection mechanisms (Kadakia & DeSalvo, 2023). By the conclusion of this step, the data scientist should possess a comprehensive, explicitly documented understanding of the legacy pipeline's structure, constraints, and opportunities, providing a rigorous and operationally grounded foundation for subsequent modernization and optimization efforts.

In addition to documenting pipeline structure and data flow, this step captures preliminary observations regarding functional limitations, inefficiencies, and sources of friction within individual pipeline stages. These observations are intentionally descriptive rather than prescriptive. The purpose of Step 2 is to surface and document issues as they exist, not to determine whether or how they should be addressed. Decisions regarding preservation, refactoring, or redesign are explicitly deferred to the next stage of the framework.

Effectiveness	Score: 5
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: This step contributes meaningfully to the modernization goal by producing an explicit, structured understanding of the legacy pipeline’s functional stages, data flows, and dependencies. The visualization and documentation artifacts directly support informed and defensible modernization decisions.	

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: The step strongly promotes modular, maintainable pipeline architecture by explicitly decomposing the pipeline into functional stages and identifying clear boundaries, dependencies, and interfaces in alignment with established software engineering principles.	

Data Scientist Usability	Score: 5
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: The step is clearly defined, repeatable, and reusable across pipeline contexts. The structured questions, visualization requirement, and documentation expectations provide consistent guidance for data scientists with varying levels of experience.	

Operational Validity	Score: 5
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Continued integration of Domain Expert commentary ensures alignment with business context and operational realities. The step captures critical information efficiently while minimizing unnecessary meetings or redundant analysis.	

Feasibility	Score: 4
Determines whether the step is practical to implement without excessive complexity	
Observation: While the step requires moderate effort and coordination, it remains practical to implement within typical modernization timelines and does not introduce excessive technical or organizational complexity.	

Reject or Accept Hypothesis
Notes: This step meets all evaluation criteria and provides a rigorous, structured foundation for subsequent stages of the modernization framework.

Citations to Be Used in this Portion of the Framework
Biswas, S., Wardat, M., & Rajan, H. (2022). The art and practice of data science pipelines: A comprehensive study of data science pipelines in theory, in-the-small, and in-the-large. <i>Proceedings of the 44th International Conference on Software Engineering (ICSE '22)</i>, 1434–1446. https://doi.org/10.1145/3510003.3510207 Foidl, H., Golendukhina, V., Ramler, R., & Felderer, M. (2024). Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. <i>Journal of Systems and Software</i>, 207, 111855. https://doi.org/10.1016/j.jss.2023.111855 Green, T. R. G., & Petre, M. (1996). Usability analysis of visual programming environments: A “Cognitive Dimensions” framework. <i>Journal of Visual Languages & Computing</i>, 7(2), 131–174. IBM. (2013). <i>The CRISP-DM methodology</i>. IBM Analytics. Kadakia, K. T., & DeSalvo, K. B. (2023). Transforming public health data systems to advance the population’s health. <i>Milbank Quarterly</i>, 101(S1), 674–699. https://doi.org/10.1111/1468-0009.12618 Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. <i>Communications of the ACM</i>, 15(12), 1053–1058. https://doi.org/10.1145/361598.361623 Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., ... Dennison, D. (2015). Hidden technical debt in machine learning systems. <i>Proceedings of the 29th</i>

FUNCTIONAL EVALUATION AND MODERNIZATION TARGET IDENTIFICATION

Proposed Framework Procedure

After completing the structural audit of the legacy data pipeline, the data scientist begins identifying which components of the pipeline should be modernized. Using the observations documented in the previous step, the data scientist determines which stages appear outdated, inefficient, or difficult to maintain and selects these as candidates for improvement. Decisions regarding modernization are based primarily on the data scientist's judgment, informal discussions with the Domain Expert, and perceived technical limitations within the pipeline.

At this stage, the data scientist may choose to focus modernization efforts on components that are most visible, easiest to modify, or most frequently cited as problematic. While Domain Expert input may influence these choices, the decision process is not governed by explicit criteria or structured prioritization. As a result, modernization targets are selected opportunistically rather than systematically, and tradeoffs between impact, effort, and operational risk remain largely implicit.

Effectiveness

Score: 2

Assesses whether the framework step contributes meaningfully to the modernization goal

Observation: This step initiates modernization decision-making but does not meaningfully guide it. Without explicit decision criteria or prioritization logic, the step produces modernization targets that are weakly justified and difficult to defend.

Framework Alignment

Score: 1

Evaluates whether the step promotes modular, maintainable pipeline architecture

Observation: The step does not promote modular or maintainable architecture. Decisions are made without reference to functional boundaries or system-level structure, increasing the risk of reinforcing existing coupling and technical debt.

Data Scientist Usability	Score: 3
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: While the step is easy to follow, it relies heavily on subjective judgment. This limits repeatability and makes it difficult for different practitioners to arrive at comparable decisions.	

Operational Validity	Score: 2
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Although Domain Expert input may be considered, it is not systematically incorporated. This creates a risk that modernization decisions prioritize technical convenience over operational impact.	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: The step is highly feasible and requires minimal additional effort. However, this ease comes at the expense of rigor and decision quality.	

Reject or Accept Hypothesis
Notes: This version of Step 3 fails to provide a structured, defensible approach to modernization decision-making. While it initiates action, it does not ensure that modernization efforts are aligned with system goals, architectural principles, or operational priorities.

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure

After completing the structural audit and diagnostic documentation of the legacy data pipeline, the data scientist begins the process of evaluating which components of the pipeline warrant modernization. This step builds directly on the artifacts produced in Step 2 and shifts the focus from understanding how the pipeline operates to making preliminary judgments about where change may be beneficial. Rather than relying solely on intuition, the data scientist introduces a set of guiding criteria to support more deliberate modernization decisions.

At this stage, each pipeline component is reviewed individually with respect to its functional role within the system. The data scientist considers whether the component continues to support the pipeline's overarching objective, whether it can accommodate evolving operational needs, and whether it can be reasonably maintained given current organizational resources. These evaluations are informed by the documented inputs, outputs, dependencies, and Domain Expert commentary collected during the audit phase. However, the criteria used in this assessment remain qualitative and are not yet formalized into a standardized evaluation framework.

In applying these criteria, the data scientist documents observed strengths and limitations of each component. For example, certain stages may perform their intended function reliably but require extensive manual intervention, while others may be technically sound yet poorly understood by pipeline users. This documentation helps distinguish between components that are functionally adequate but structurally fragile and those that are misaligned with current operational goals. Importantly, this step emphasizes careful reasoning rather than solution design, ensuring that decisions are grounded in observed behavior rather than premature technical preferences.

Throughout this review, the data scientist engages with the Domain Expert to validate interpretations and ensure that perceived issues correspond to real operational challenges. These discussions help clarify whether identified limitations meaningfully affect decision-making or whether they represent tolerable inefficiencies within the broader system. Despite this collaboration, the process does not yet require explicit tradeoff analysis between competing modernization targets, nor does it enforce consistent documentation of decision rationale across pipeline components.

The outcome of this step is a refined but still provisional understanding of which components of the pipeline may benefit from modernization. While the data scientist is able to articulate reasons for focusing on certain stages over others, these reasons remain descriptive rather than prescriptive. Modernization candidates are identified, but they are not yet categorized, ranked, or linked to a formal prioritization strategy. As a result, this version improves decision awareness but does not yet produce a defensible modernization plan.

Effectiveness	Score: 3
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: This version contributes more meaningfully to the modernization goal than prior versions by introducing explicit evaluation criteria and documenting reasoning at the component level. However, the absence of formal decision rules limits its ability to reliably guide modernization actions.	

Framework Alignment	Score: 3
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: By evaluating components individually, the step supports modular thinking. Nevertheless, it does not require explicit classification or boundary definition, weakening alignment with maintainable architectural design principles.	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: The step is easy to follow and naturally extends artifacts from the diagnostic phase. However, the lack of standardized templates or scoring mechanisms reduces consistency across implementations.	

Operational Validity	Score: 3
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Engagement with the Domain Expert improves alignment with operational context, but the step does not yet ensure that modernization targets are selected based on business impact rather than technical convenience.	

Feasibility	Score: 4
Determines whether the step is practical to implement without excessive complexity	

Observation: The step is feasible within typical modernization efforts and requires moderate coordination and documentation. It introduces additional effort without excessive complexity.

Reject or Accept Hypothesis

Notes:

Citations to Be Used in this Portion of the Framework

This version represents a clear improvement over the initial decision-making approach but still lacks formal prioritization, classification, and repeatable decision logic. Further refinement is required before the step can be accepted.

Proposed Framework Procedure

After completing the structural audit and diagnostic documentation of the legacy data pipeline, the data scientist transitions from descriptive analysis to explicit modernization decision-making. Unlike prior steps, which focus on understanding how the pipeline operates and where limitations exist, this step requires the data scientist to make provisional judgments about how individual pipeline components should be treated moving forward. These judgments are grounded in the artifacts produced during Step 2 and informed by established principles of software modularity, usability, and pipeline quality (**Parnas, 1972; Biswas et al., 2022**).

At this stage, each pipeline component identified in the structural decomposition is evaluated independently and assigned to a preliminary modernization category. These categories reflect distinct modernization strategies rather than technical solutions and include preserving components that function effectively, refactoring components that meet functional requirements but suffer from structural or usability limitations, and redesigning components that are fundamentally misaligned with current operational goals or organizational constraints. Introducing explicit classification marks a critical shift in the framework from observation to decision and helps prevent modernization efforts from defaulting to full system rebuilds (**Assunção et al., 2025**).

The classification process is guided by a small set of decision-oriented criteria derived from prior literature on data pipeline quality and system usability. Components are evaluated based on their reliability, transparency, maintainability, and alignment with current business needs (**Foidl et al., 2024; Green & Petre, 1996**). Rather than treating all identified issues as equally problematic, this step encourages the data scientist to distinguish between tolerable

inefficiencies and limitations that materially affect decision-making or system sustainability. These distinctions are documented explicitly to support later justification and review.

Domain Expert input remains essential during this step but serves a different role than in earlier stages. Rather than primarily explaining pipeline behavior, the Domain Expert now helps validate whether proposed classifications reflect operational reality. For example, a component that appears technically fragile may nonetheless be operationally acceptable if it supports a low-risk or infrequently used process. Conversely, components that appear minor from a technical perspective may warrant redesign if they support critical decisions or are heavily relied upon by pipeline users (McGowan et al., 2023). This validation step helps ensure that modernization decisions remain grounded in real organizational impact rather than technical preference alone.

Importantly, this version of the step does not yet require formal prioritization or tradeoff analysis across components. While components are classified, they are not ranked or sequenced, and resource constraints are not explicitly considered. As a result, the output of this step is a set of justified but provisional modernization decisions rather than a finalized modernization plan. This limitation is intentional: introducing classification before prioritization allows the data scientist to establish clear reasoning and shared understanding before committing to specific implementation paths (IBM, 2013).

The outcome of this step is a documented mapping between pipeline components and proposed modernization strategies, accompanied by written justification and Domain Expert validation. This artifact provides a defensible intermediate decision layer that bridges structural diagnosis and implementation planning. However, without explicit prioritization, scoring, or effort–impact analysis, the step remains incomplete and requires further refinement before it can fully support execution-level modernization decisions.

Effectiveness	Score: 4
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: This step meaningfully advances the modernization process by introducing explicit decision categories and justification grounded in structural and operational evidence. However, the absence of prioritization limits its ability to directly guide implementation.	

Framework Alignment	Score:
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: By classifying components based on modular and usability considerations, the step aligns well with maintainable architecture principles. Full	

alignment is limited by the lack of explicit interface or dependency evaluation across components.

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: The step provides clear guidance and decision structure, improving repeatability. However, without standardized scoring or templates, some variability in classification rationale remains.	

Operational Validity	Score: 4
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Structured validation with the Domain Expert ensures that classifications reflect operational priorities. Still, the lack of explicit impact assessment limits confidence that the most critical components will be addressed first.	

Feasibility	Score: 4
Determines whether the step is practical to implement without excessive complexity	
Observation: The step is feasible to implement within typical modernization efforts and requires moderate documentation and coordination. Complexity remains manageable.	

Reject or Accept Hypothesis
Notes: This version represents a strong intermediate step that successfully transitions the framework from diagnosis to decision-making. To be fully accepted, the step must incorporate explicit prioritization, tradeoff analysis, and resource-awareness.version

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure

Following the classification of pipeline components into preliminary modernization categories, the data scientist advances from provisional decision-making to structured prioritization. The goal of this step is to determine not only *what* should be modernized, but *where effort should be allocated first*, given limited time, resources, and organizational constraints. This step transforms classification into an actionable modernization strategy while remaining agnostic to specific technologies or modeling techniques.

Step 3.1: Confirm Modernization Categories

Using the classification artifact produced in Version 3, the data scientist first confirms the assigned modernization category for each pipeline component:

- Preserve: Components that reliably fulfill their functional role and align with current operational needs.
- Refactor: Components that produce correct outputs but suffer from structural fragility, poor documentation, or limited usability.
- Redesign: Components that are misaligned with current objectives, introduce excessive operational risk, or constrain future extensibility.

These categories reflect distinct modernization strategies rather than implementation choices and are grounded in established principles of modular system design and pipeline quality (Parnas, 1972; Biswas et al., 2022).

Step 3.2: Evaluate Components Along Decision-Relevant Dimensions

Each pipeline component is then evaluated along a common set of decision-oriented dimensions. These dimensions are intentionally limited in number to ensure usability while capturing the most important tradeoffs identified in prior work on data pipelines and system usability (Foidl et al., 2024; Green & Petre, 1996):

1. Operational Impact:
To what extent does this component influence downstream decisions, reporting, or system behavior?
2. Risk and Fragility:
How likely is failure, error propagation, or misinterpretation at this stage, and what are the consequences?
3. Maintainability:
How difficult is the component to modify, debug, or extend given current documentation and structure?

4. Alignment with Current Goals:

Does the component support present-day business or operational objectives, or is it optimized for outdated requirements?

For each dimension, the data scientist records qualitative assessments supported by evidence from the structural audit and Domain Expert input.

Step 3.3: Conduct Relative Impact–Effort Reasoning

Rather than attempting precise cost estimation, this version of the framework introduces relative impact–effort reasoning. For each component, the data scientist considers:

- Expected impact of modernization (low / medium / high)
- Estimated effort required (low / medium / high)

This approach reflects best practices in early-stage system redesign, where precise estimates are often unavailable but relative comparisons remain informative (Assunção et al., 2025). Components that offer high impact at relatively low effort are flagged as early candidates for modernization, while high-effort components are deferred unless they pose substantial operational risk.

Step 3.4: Produce a Preliminary Modernization Roadmap

Based on the evaluations above, the data scientist produces a preliminary modernization roadmap that:

- Orders components by relative priority
- Identifies near-term, mid-term, and long-term modernization candidates
- Explicitly documents tradeoffs and deferrals

This roadmap is not yet a commitment to implementation but serves as a structured decision artifact that can be reviewed, revised, and validated with the Domain Expert. Importantly, the roadmap makes explicit which components will *not* be addressed immediately, reducing scope creep and unmanaged complexity (Sculley et al., 2015).

Step 3.5: Validate Priorities with the Domain Expert

Finally, the proposed prioritization is reviewed with the Domain Expert to confirm that sequencing decisions align with operational realities. This validation step focuses on prioritization rather than classification, ensuring that high-impact operational needs are addressed early and that modernization does not disrupt critical workflows. Feedback from this review is incorporated into the roadmap, and any disagreements are documented for transparency (McGowan et al., 2023).

Effectiveness	Score: 4
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: This version substantially advances the modernization goal by converting classification into an actionable prioritization strategy. However, the lack of formal scoring limits decision defensibility in resource-constrained environments.	

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: The step strongly aligns with modular, maintainable architecture principles by explicitly differentiating preservation, refactoring, and redesign strategies and sequencing modernization accordingly.	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: The procedural structure improves usability and repeatability. Still, reliance on qualitative judgment introduces some variability across practitioners.	

Operational Validity	Score: 5
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Explicit validation with the Domain Expert ensures that prioritization reflects real operational needs and constraints.	

Feasibility	Score: 4
Determines whether the step is practical to implement without excessive complexity	
Observation: The step is practical to implement and does not require detailed cost modeling. Moderate coordination and documentation effort is required.	

Reject or Accept Hypothesis

Notes: This version is suitable for guiding early modernization planning but requires formalization of evaluation and prioritization criteria to fully support defensible decision-making.

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure

Once the legacy data pipeline has been structurally decomposed and diagnostically documented, the data scientist must transition from understanding the system to making informed modernization decisions. The purpose of this step is to translate descriptive insights into a structured decision framework that determines what should change, what should remain, and in what order modernization efforts should occur. This step is intentionally method-agnostic and focuses on decision quality rather than implementation detail.

Rather than treating modernization as a binary choice between preserving or rebuilding the pipeline, this step introduces a structured decision process that recognizes varying levels of intervention. By doing so, it prevents unnecessary system disruption while ensuring that modernization effort is directed toward components that meaningfully limit performance, usability, or long-term sustainability.

Importantly, the purpose of this step is **not** to predict post-modernization performance or quantify final outcomes. Instead, it identifies how changes to individual pipeline components are likely to alter the behavior, assumptions, and requirements of downstream stages. By examining how modifications propagate through the pipeline, the data scientist can anticipate structural consequences of change prior to implementation.

Step 3.1: Establish Modernization Treatment Categories

Using the pipeline components identified in Step 2, the data scientist assigns each component to one of three modernization treatment categories:

- **Preserve:**
Components that reliably fulfill their functional role, align with current operational needs, and do not meaningfully constrain future extensibility.
- **Refactor:**
Components that produce correct outputs but exhibit structural weaknesses such as poor modularity, unclear logic, limited documentation, or high maintenance overhead.

- **Redesign:**
Components that are misaligned with current objectives, introduce significant operational risk, or fundamentally limit the pipeline's ability to evolve.

These categories reflect **levels of intervention**, not specific technologies or tools. Introducing explicit treatment categories aligns modernization decisions with established principles of modular system design and avoids the common pitfall of defaulting to full system replacement (Parnas, 1972; Biswas et al., 2022).

Step 3.2: Evaluate Components Along Decision-Relevant Dimensions

Each pipeline component is evaluated along a consistent set of decision-oriented dimensions. These dimensions balance analytical rigor with practical usability and reflect common sources of pipeline fragility identified in prior research (Foidl et al., 2024; Green & Petre, 1996):

- **Operational Impact:**
To what extent does this component influence downstream decisions, reporting, or organizational action?
- **Risk and Fragility:**
How likely is failure, error propagation, or misinterpretation at this stage, and what are the consequences?
- **Maintainability:**
How difficult is the component to modify, debug, or extend given its current structure and documentation?
- **Alignment with Current Goals:**
Does the component support present-day business objectives, or is it optimized for outdated requirements?

For each dimension, the data scientist records concise qualitative assessments supported by evidence from the structural audit and validated Domain Expert input.

Step 3.3: Change Sensitivity and Downstream Impact Analysis

Rather than estimating outcomes or implementation costs, this step evaluates change. Rather than estimating outcomes or implementation costs, this step evaluates **change sensitivity**: how modifications to a pipeline component are expected to affect downstream stages based on documented dependencies and assumptions. The goal is to understand how structural changes propagate through the pipeline and to identify components whose modification would require corresponding adjustments elsewhere in the system.

For each pipeline component identified for potential modernization, the data scientist examines how a change in the nature of the component's output would alter the inputs, assumptions, or behavior of subsequent stages. This includes assessing whether downstream components assume static versus dynamic data, deterministic versus probabilistic values, or

fixed schemas versus evolving data structures. These assumptions are surfaced directly from the structural audit and visualization produced in Step 2.

In this context, **impact** is defined as the extent to which modifying a component necessitates changes in downstream processing, validation, interpretation, or decision logic. Components whose outputs are consumed by many downstream stages, or whose outputs form the basis of critical decisions, are considered highly sensitive to change. Conversely, components whose outputs are isolated or loosely coupled introduce fewer downstream implications.

Similarly, **effort** is interpreted as the degree of structural adjustment required across the pipeline in response to change. A component may be simple to modify in isolation but costly to modernize if downstream stages rely on implicit assumptions that no longer hold. By focusing on change propagation rather than implementation complexity alone, this analysis avoids arbitrary estimation and grounds prioritization in observable system structure.

Assessments are documented using relative categories (e.g., low, moderate, high), accompanied by written justification explicitly referencing downstream dependencies and assumptions. This approach supports transparent decision-making without requiring speculative performance estimates or premature optimization.

Example: Change Propagation in a Legacy Pipeline

Consider a pipeline stage that currently provides static historical averages to downstream reporting and planning components. If this stage is modernized to instead output predicted values generated by a machine learning model, several downstream implications arise. Reporting components may need to accommodate uncertainty or confidence intervals rather than single-point estimates. Planning logic may need to distinguish between observed and predicted values, and validation steps may need to be introduced to monitor model performance over time. Although the modification occurs at a single stage, its impact propagates across multiple downstream components. This change would therefore be classified as high-impact, not because the model itself is complex, but because it alters assumptions embedded throughout the pipeline.

Step 3.4: Construct a Preliminary Modernization Roadmap

Based on the evaluations above, the data scientist constructs a preliminary modernization roadmap that:

- Orders components by relative priority
- Identifies near-term, mid-term, and long-term modernization targets
- Explicitly documents tradeoffs, deferrals, and rationale

This roadmap functions as a decision artifact rather than an implementation plan. By making prioritization explicit, the roadmap reduces scope creep, surfaces implicit assumptions, and helps manage the accumulation of technical debt during incremental modernization (**Sculley et al., 2015**).

Step 3.5: Validate Priorities Through Domain Expert Review

The proposed prioritization is reviewed with the Domain Expert to ensure alignment with operational realities. Unlike earlier stages, this discussion focuses on sequencing and impact rather than system explanation. The Domain Expert confirms whether prioritized components correspond to critical workflows, time-sensitive decisions, or known operational pain points. Any disagreements or constraints are documented to preserve transparency and traceability in the decision process (**McGowan et al., 2023**).

At the conclusion of this step, the data scientist possesses a structured, justified, and operationally validated modernization plan that identifies where modernization effort should be directed and why. While prioritization has been established, decisions remain comparative rather than formally scored, preserving flexibility while preparing the framework for more formal evaluation in the final version of this step.

Effectiveness	Score: 5
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: This step meaningfully advances the modernization process by translating diagnostic insights into structured, defensible decisions about where and how modernization should occur. By focusing on treatment categories, decision-relevant dimensions, and change propagation, the step creates a clear bridge between system understanding and actionable planning without requiring premature implementation assumptions.	

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: The explicit classification of components into preserve, refactor, and redesign categories strongly reinforces modular thinking. The emphasis on change sensitivity and downstream dependency analysis further aligns modernization decisions with maintainable system design principles.	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	

Observation: The step is clearly structured, procedural, and supported by concrete examples, making it highly usable and repeatable. Minor variability may remain in how practitioners interpret qualitative categories, but this is mitigated through required written justification tied to audit artifacts.

Operational Validity	Score: 5
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Change sensitivity analysis and formal Domain Expert validation ensure that modernization decisions reflect operational realities rather than technical preference. The step explicitly captures how changes alter assumptions embedded in downstream processes, preserving alignment with real-world use.	

Feasibility	Score: 4
Determines whether the step is practical to implement without excessive complexity	
Observation: The step is feasible within typical modernization efforts and does not require detailed cost modeling or implementation work. While it requires thoughtful documentation and coordination, this effort is proportional to the value of the decisions it supports.	

Reject or Accept Hypothesis
Notes: This step satisfies all evaluation criteria and provides a rigorous, method-agnostic decision layer between pipeline diagnosis and technical implementation. It is suitable for inclusion as a required stage in the proposed modernization framework.

Citations to Be Used in this Portion of the Framework
Assunção, W. K. G., Marchezan, L., Arkoh, L., Egyed, A., & Ramler, R. (2025). Contemporary software modernization: Strategies, driving forces, and research opportunities. <i>ACM Transactions on Software Engineering and Methodology</i> , 34(5), Article 142. https://doi.org/10.1145/3708527

Biswas, S., Wardat, M., & Rajan, H. (2022). The art and practice of data science pipelines: A comprehensive study of data science pipelines in theory, in-the-small, and in-the-large. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)* (pp. 1434–1446). ACM.

Foidl, H., Golendukhina, V., Ramler, R., & Felderer, M. (2024). Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *Journal of Systems and Software*, 207, 111855. <https://doi.org/10.1016/j.jss.2023.111855>

Green, T. R. G., & Petre, M. (1996). Usability analysis of visual programming environments: A “Cognitive Dimensions” framework. *Journal of Visual Languages & Computing*, 7(2), 131–174.

McGowan, L. D., Peng, R. D., & Hicks, S. C. (2023). Design principles for data analysis. *Journal of Computational and Graphical Statistics*, 32(4), 1192–1207.

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., ... Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Proceedings of the 29th International Conference on Neural Information Processing Systems (NIPS '15)* (pp. 2503–2511).

~~ANALYTICAL STRATEGY SELECTION AND READINESS ASSESSMENT~~

~~ENDED UP REMOVING THIS STEP COMPLETELY~~

Proposed Framework Procedure	
After identifying modernization priorities within the pipeline, the data scientist begins to consider how the modernized system may be used for analysis. This involves reflecting on whether additional analytical techniques, such as modeling or visualization, could be applied to generate new insights. The data scientist may begin thinking about potential analytical approaches based on personal experience or familiarity with available tools. This step serves as a transition from planning to technical work.	

Effectiveness	Score: 1
---------------	----------

Assesses whether the framework step contributes meaningfully to the modernization goal
Observation: Does not meaningfully guide analytical decisions

Framework Alignment	Score: 1
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Analytical thinking is disconnected from prior steps.	

Data Scientist Usability	Score: 3
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: Easy to follow but offers little value.	

Operational Validity	Score: 1
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Not grounded in domain or decision needs.	

Feasibility	Score: N/A
Determines whether the step is practical to implement without excessive complexity	
Observation: Unsure how long this will take	

Reject or Accept Hypothesis
Notes: Need to determine the time frame it would take to implement the step

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
After modernization decisions have been established, the data scientist reflects on how the pipeline should support analysis and insight generation. Rather than immediately selecting models or tools, this step encourages consideration of the kinds of questions the modernized pipeline is expected to answer. The data scientist evaluates whether the pipeline’s purpose is primarily descriptive, diagnostic, or predictive, and considers whether analytical complexity is warranted. This reflection helps prevent premature technical implementation.

Effectiveness	Score: 2
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: Introduces analytical intent but remains vague.	

Framework Alignment	Score: 3
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Begins connecting analysis to pipeline purpose.	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: Clearer but still underspecified.	

Operational Validity	Score: 3
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Some alignment with decision needs.	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation:	

Reject or Accept Hypothesis
Notes: The bones of this step are present yet the step is not contributing enough specific information to be accepted into the framework

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
Once modernization priorities have been defined and validated, the data scientist must explicitly determine how the modernized pipeline is intended to support analysis and decision-making. This step focuses on establishing analytical direction by clarifying the <i>functional role of analysis</i> within the system, rather than selecting specific techniques or tools. The goal is to ensure that analytical complexity is introduced intentionally and in direct response to operational need. At this stage, the data scientist evaluates the decisions supported by the pipeline and considers whether analytical outputs are intended to describe historical behavior, monitor ongoing performance, support planning activities, or inform operational actions. This reflection prevents analytical development from being driven by technical opportunity alone and instead anchors it in the pipeline's purpose. Importantly, this step does not assume that predictive modeling or machine learning is required; descriptive or rule-based approaches may be sufficient depending on context. In addition, the data scientist considers whether the available data is suitable for the intended analytical role. Factors such as data completeness, temporal structure, and stability are assessed relative to analytical goals rather than in absolute terms. This version introduces the notion of analytical readiness, but it stops short of explicitly documenting assumptions or readiness gaps, limiting its ability to fully guide subsequent implementation.

Effectiveness	Score: 4
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: Clarifies analytical purpose and prevents premature technical decisions.	

Framework Alignment	Score: 4
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Maintains separation between planning and implementation.	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: Reusable and conceptually clear.	

Operational Validity	Score: 4
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Anchored in decision needs but not fully documented.	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: Conceptual and low burden.	

Reject or Accept Hypothesis	
Notes: Not quite enough information within this step to provide any real improvements	

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure

After modernization priorities have been identified and validated, the data scientist must determine how the modernized data pipeline should support analysis and decision-making. The purpose of this step is to establish analytical readiness and analytical direction by explicitly articulating the role that analysis is expected to play within the system. This step functions as a conceptual bridge between modernization planning and technical implementation, ensuring that subsequent modeling and visualization efforts are grounded in deliberate reasoning rather than exploratory impulse or tool-driven enthusiasm.

A critical premise of this step is that improved data infrastructure does not inherently justify the use of advanced analytical techniques. While modernized pipelines may enable new analytical possibilities, the introduction of modeling, forecasting, or automated decision logic must be justified by the operational role of the pipeline and the decisions it supports. Treating analytical development as an automatic next phase risks misalignment between analytical outputs and organizational needs, a well-documented failure mode in data-driven systems (Cleveland, 2001; McGowan et al., 2023). This step therefore requires the data scientist to pause and reflect before engaging in technical implementation.

To establish analytical direction, the data scientist revisits the decision context documented in earlier steps and clarifies the intended function of analysis within the pipeline. In some cases, the primary role of the pipeline may be descriptive, providing structured summaries of historical or current system behavior. In other contexts, analysis may serve a monitoring or diagnostic role, supporting anomaly detection, quality assurance, or operational oversight. More complex roles, such as forecasting, scenario evaluation, or decision support, introduce additional analytical and interpretive demands and should only be pursued when they are clearly warranted by the decision environment. By explicitly identifying the analytical role, this step prevents analytical complexity from exceeding the informational needs of users (Tukey, 1962).

Equally important is an assessment of whether the available data is suitable for the intended analytical role. Analytical readiness is evaluated not in absolute terms, but relative to purpose. Data that is sufficient for descriptive reporting may be inadequate for predictive modeling, and data that supports forecasting trends may not be appropriate for automated decision-making. Characteristics such as data completeness, temporal resolution, stability of data-generating processes, and interpretability are considered in relation to analytical ambition rather than as isolated quality metrics (Foidl et al., 2024). This perspective emphasizes *fitness for use* over abstract notions of data quality.

This step also requires explicit consideration of interpretability, governance, and operational constraints. Analytical approaches that produce opaque or probabilistic outputs may be inappropriate in environments where decisions must be explained, audited, or manually validated. Conversely, more complex analytical methods may be justified in contexts where decisions are high-impact, time-sensitive, or repeated at scale. By incorporating these considerations before technical work begins, the

framework aligns analytical direction with organizational capacity and user expectations, a key principle in effective data analysis design (Green & Petre, 1996; McGowan et al., 2023).

Importantly, this step does not select specific models, algorithms, or tools. Instead, it identifies classes of analytical approaches that are appropriate given the pipeline's goals and constraints, such as descriptive analytics, rule-based logic, predictive modeling, or scenario-based analysis. Any assumptions underlying this analytical direction, such as the availability of historical data, the need for uncertainty communication, or the stability of input data, are documented explicitly. Where gaps exist between desired analytical capability and current data readiness, these gaps are recorded as constraints to be addressed during implementation rather than treated as failures of the framework itself. This distinction preserves conceptual clarity while setting realistic expectations for subsequent development (IBM, 2013).

By the conclusion of this step, the data scientist has a defensible understanding of whether analysis is warranted, what role analysis should play, and what conditions must hold for analytical development to succeed. This shared understanding ensures that modeling and visualization decisions made in later stages are grounded in decision relevance, data suitability, and operational reality. As such, Step 4 represents the final conceptual checkpoint before engaging in code, models, or visual artifacts, reinforcing the framework's emphasis on intentional, decision-driven data science practice.

Effectiveness	Score: 1
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: After further consideration this step is a continuation and repeat of the previous step therefore is ineffective	

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: The step preserves a clean separation between conceptual planning and technical execution, reinforcing modularity and decision-driven system design.	

Data Scientist Usability	Score: 0
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: This step adds undo stress and work to the data scientist without a productive outcome.	

Operational Validity	Score: N/A
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Was not evaluated via Domain Expert	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: The step is entirely conceptual and can be completed using information gathered in prior stages, imposing no technical or computational burden.	

Reject or Accept Hypothesis
Notes: This step was determined to provide no new information for the framework. The ideas developed in this step were considered in the revisioning of the previous steps and the creation of the steps to follow.

Citations to Be Used in this Portion of the Framework
Cleveland, W. S. (2001). Data science: An action plan for expanding the technical areas of the field of statistics. <i>International Statistical Review</i>, 69(1), 21–26. Foidl, H., Golendukhina, V., Ramler, R., & Felderer, M. (2024). Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. <i>Journal of Systems and Software</i>, 207, 111855. https://doi.org/10.1016/j.jss.2023.111855 Green, T. R. G., & Petre, M. (1996). Usability analysis of visual programming environments: A “Cognitive Dimensions” framework. <i>Journal of Visual Languages & Computing</i>, 7(2), 131–174. IBM. (2013). <i>The CRISP-DM methodology</i>. IBM Analytics.

McGowan, L. D., Peng, R. D., & Hicks, S. C. (2023). Design principles for data analysis. *Journal of Computational and Graphical Statistics*, 32(4), 1192–1207.

Tukey, J. W. (1962). The future of data analysis. *Annals of Mathematical Statistics*, 33(1), 1–67.

MODELING AND MACHINE LEARNING CONSIDERATIONS

Proposed Framework Procedure

After determining that analysis is appropriate for the modernized data pipeline, the data scientist may begin developing models to generate predictions or insights. Machine learning techniques can be applied to improve accuracy, automate decisions, or identify patterns in the data. The choice of model depends on the data available and the goals of the analysis. Once models are developed, their outputs can be used to support decision-making within the organization.

Effectiveness

Score: 1

Assesses whether the framework step contributes meaningfully to the modernization goal

Observation: The step assumes modeling is necessary and provides no guidance for determining appropriateness.

Framework Alignment

Score: 1

Evaluates whether the step promotes modular, maintainable pipeline architecture

Observation: Modeling is treated as a default activity rather than a decision grounded in prior steps.

Data Scientist Usability

Score: 3

Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.

Observation: Familiar but underspecified; encourages tool-driven behavior.

Operational Validity	Score: 1
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: No consideration of interpretability, maintenance, or user context.	

Feasibility	Score: 3
Determines whether the step is practical to implement without excessive complexity	
Observation: Conceptually leaving too much room for error	

Reject or Accept Hypothesis
Notes:

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
Once analytical direction has been established, the data scientist considers whether predictive or learning-based models are appropriate for the modernized pipeline. This involves evaluating whether sufficient data exists to support modeling and whether predictive outputs would meaningfully improve decision quality. The data scientist may also reflect on whether machine learning techniques offer advantages over simpler analytical approaches. This version acknowledges that modeling is not always required, but it still frames machine learning as the primary analytical mechanism once analysis is deemed appropriate. Considerations such as model interpretability, robustness, and long-term maintenance are mentioned but not systematically integrated into the decision process. As a result, modeling decisions remain only loosely connected to operational constraints and pipeline design.

Effectiveness	Score: 1
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: The step assumes modeling is necessary and provides no guidance for determining appropriateness.	

Framework Alignment	Score: 1
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Modeling is treated as a default activity rather than a decision grounded in prior steps.	

Data Scientist Usability	Score: 3
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: Familiar but underspecified; encourages tool-driven behavior.	

Operational Validity	Score: 1
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: No consideration of interpretability, maintenance, or user context.	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: Easy to execute, but conceptually weak.	

Reject or Accept Hypothesis

Notes: Continue to develop this step further

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
Once analytical direction has been established, the data scientist considers whether predictive or learning-based models are appropriate for the modernized pipeline. This involves evaluating whether sufficient data exists to support modeling and whether predictive outputs would meaningfully improve decision quality. The data scientist may also reflect on whether machine learning techniques offer advantages over simpler analytical approaches. This version acknowledges that modeling is not always required, but it still frames machine learning as the primary analytical mechanism once analysis is deemed appropriate. Considerations such as model interpretability, robustness, and long-term maintenance are mentioned but not systematically integrated into the decision process. As a result, modeling decisions remain only loosely connected to operational constraints and pipeline design.

Effectiveness	Score: 3
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: Introduces the idea that modeling may not be necessary, but provides limited guidance.	

Framework Alignment	Score: 3
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Begins connecting modeling decisions to pipeline purpose.	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	

Observation: More cautious than Version 1, but still vague.

Operational Validity	Score: 3
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Some acknowledgment of operational concerns, but not deeply grounded.	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: Conceptually light and easy to apply.	

Reject or Accept Hypothesis
Notes: The ideas are continuing to take shape but is not nearly in depth enough for this to be accepted.

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
After analytical readiness and direction have been established, the data scientist evaluates whether predictive or learning-based models are appropriate within the modernized pipeline. This step explicitly reframes modeling as a <i>design decision</i> rather than an automatic extension of data availability. The data scientist considers whether modeling outputs would meaningfully enhance the decisions supported by the pipeline or whether simpler analytical approaches are sufficient. At this stage, the data scientist reflects on the nature of the decisions being supported and the role modeling would play within those decisions. Predictive models may be appropriate when

decisions involve uncertainty, forward-looking planning, or repeated allocation choices. Conversely, when decisions rely primarily on transparency, traceability, or stable relationships, rule-based or descriptive approaches may be more appropriate. This distinction ensures that modeling complexity is proportional to decision needs.

The data scientist also considers high-level modeling constraints, including interpretability requirements, data stability, and the organizational capacity to maintain models over time. However, while this version clarifies *when* modeling may be appropriate, it does not yet require explicit articulation of modeling assumptions or downstream implications, limiting its ability to fully guide responsible implementation.

Effectiveness	Score: 4
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: Strongly improves decision discipline around modeling use.	

Framework Alignment	Score: 4
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation:	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: Modeling is clearly subordinated to pipeline goals and structure.	

Operational Validity	Score: 4
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Conceptually clear and reusable.	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: No technical burden.	

Reject or Accept Hypothesis
Notes: The ideas of this step are concrete enough to conditionally accept the step (however will still be recorded as a rejection because there is still the need to edit). The step needs to include far more detail which will be added in the next step.

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
After modernization priorities and analytical scope have been established, the data scientist must determine whether predictive or learning-based models will be introduced into the modernized pipeline, and if so, under what explicit constraints. The purpose of this step is to govern modeling behavior so that forecasting, machine learning, and decision-science components are introduced only when they materially improve decision quality and can be supported operationally. This step is intentionally strict: it exists to prevent the common failure mode in which practitioners “try a variety of models” without explicitly pricing the interpretability, maintenance, and system-coupling consequences of doing so. Machine learning systems often accumulate hidden technical debt through entanglement, unstable data dependencies, and long-term maintenance burden; these costs must be surfaced before implementation (Sculley et al., 2015). Prohibited activities in Step 4: training models, running grid searches, fitting baselines for “quick comparison,” building dashboards to “see what the data looks like,” or selecting algorithms based on familiarity. Permitted activities in Step 4: classification of components, documentation of assumptions, governance decisions, and creation of required artifacts described below. At the conclusion of Step 4, the data scientist must produce a single auditable artifact: the Modeling Governance Record (MGR). This record is the gate that determines what modeling work is permitted later.

4.1 Construct the Candidate Modeling Register (CMR)

The data scientist begins by creating a Candidate Modeling Register (CMR) consisting only of pipeline components identified in Step 3 that satisfy all of the following eligibility conditions:

1. The component's output influences a downstream decision, planning activity, prioritization rule, or resource allocation process (directly or indirectly).
2. The component's output is time-sensitive or repeated (daily/weekly/monthly cadence), such that consistent model behavior matters.
3. The component produces a quantity where improved estimation or prediction would plausibly change decisions (not merely improve reporting aesthetics).

For each eligible component, the CMR must contain:

- Component ID (must match the component map from Step 3)
- Current function (one sentence describing what it does today)
- Downstream consumers (systems or people that use its output)
- Decision touchpoint (the decision meeting/workflow/process it informs)
- Decision cadence (e.g., daily scheduling, weekly planning)
- Failure impact statement (what happens if output is wrong or late)

Any component that fails one of the three eligibility conditions is classified as Non-Model Component and documented as excluded. This is not optional: excluded components must be recorded to prevent scope creep during implementation.

4.2 Define Modeling Intent Using a Standard Output Contract

For each component in the CMR, the data scientist defines a Model Output Contract (MOC). This is a strict specification that prevents model outputs from drifting into ambiguous use.

The MOC must include:

A. Output type

Select exactly one:

- Point estimate (single numeric output per time period)
- Interval estimate (estimate + confidence/credible interval)
- Distribution / scenario set (multiple scenarios or simulated outputs)
- Classification / ranking (category, priority class, or ordered list)

B. Time semantics

Specify:

- Prediction horizon (e.g., next hour, next day, next week)
- Update cadence (how often the output must refresh)
- Latency tolerance (maximum allowable delay before output is unusable)

C. Downstream usage rule

Specify exactly how the output is used:

- Advisory (humans review; output informs judgment)
- Trigger (output causes a predefined action if threshold crossed)
- Optimization input (output becomes an input to a decision model)
- Automated action (output directly triggers operations—high scrutiny)

D. Acceptable error profile

Without training any models, the data scientist sets the *shape* of acceptable error:

- Directional tolerance (is underestimation worse than overestimation?)
- Worst-case tolerance (what is the maximum tolerable deviation?)
- Stability requirement (is volatility in predictions acceptable?)

This prevents later implementation from optimizing accuracy while violating operational expectations.

4.3 Select the Analytical Paradigm and Lock It

For each MOC, the data scientist must assign exactly one paradigm:

1. Descriptive / Rule-Based (no learning; deterministic logic)
2. Forecasting (predict future values from past patterns)
3. Predictive ML (supervised learning with features)
4. Decision Science / Optimization-Linked (model outputs feed decisions)

This selection must be written as a one-paragraph justification referencing:

- decision touchpoint (from CMR),
- output type and time horizon (from MOC),
- and structural implications from Step 3 change-sensitivity analysis.

Default rule: if the justification cannot be written clearly, the paradigm defaults to Descriptive / Rule-Based (Cleveland, 2001).

This is a deliberate constraint to avoid introducing learning when the decision value is unclear.

4.4 Enforce Interpretability and Explanation Requirements

Interpretability is enforced as a system constraint, not a preference. For each CMR component, the data scientist assigns an Interpretability Requirement Level (IRL) and must comply with the associated restrictions.

Interpretability Requirement Levels

- IRL-1: Fully Interpretable
 - Outputs must be explainable in plain language with traceable feature influence.
 - Users must be able to answer: “Why did it output this?” without technical mediation.
- IRL-2: Interpretable with Formal Explanation Artifacts
 - Outputs may be complex, but must be accompanied by standardized explanations (feature importance summaries, rule approximations, documented drivers).
- IRL-3: Black-box permitted
 - Only allowed if output is not used for high-stakes operational decisions and is not externally audited.

Model Class Restrictions by IRL

IRL-1 (Allowed only):

- Linear regression / GLMs
- Rules / heuristics
- Small decision trees
- Simple time-series baselines (moving averages, exponential smoothing where explainable)

IRL-2 (Allowed with explanation artifacts):

- Regularized regression (lasso/ridge)
- Generalized additive models
- Gradient boosted trees or random forests *only if explanation artifacts are required and reviewed*
- Classical forecasting models with documented decomposition (trend/seasonality/residual)

IRL-3 (Restricted, exception-only):

- Deep neural networks
- Large ensembles with opaque dynamics
- Highly non-linear black-box learners

Exception rule: Any IRL-3 model requires a written exception memo inside the MGR explaining why IRL-1 or IRL-2 is insufficient and how the system will detect and contain failure.

This interpretability gating directly prevents the “hidden coupling” and “abstraction debt” described in Sculley et al. (2015), where model behavior becomes difficult to reason about and the system becomes harder to safely modify.

4.5 Document Data and Feature Assumptions as a Formal Assumption Ledger

For each component with a forecasting or ML paradigm, the data scientist creates a Model Assumption Ledger (MAL) containing:

A. Data sufficiency assumptions

- Minimum history required
- Coverage requirements (missingness tolerance by time window)
- Granularity requirements (hourly vs daily vs weekly)

B. Feature stability assumptions

- Feature definitions must remain consistent
- Units and schema must remain stable
- Data collection process changes must be logged

C. Temporal validity assumptions

- Relationships learned from historical data must remain meaningful
- Known regime shifts must be documented
- Seasonal/structural effects must be identified (even if not modeled yet)

Each assumption must be assigned one of:

- Supported (evidence exists from pipeline audit/domain input)
- Uncertain (plausible but not confirmed)
- Unsupported (contradicted by evidence)

If key assumptions are Unsupported, modeling is deferred.

4.6 Specify Monitoring, Maintenance, and Ownership as a Deployment Contract

To prevent “training something once and forgetting it,” the data scientist must create a Model Operations Contract (MOC-OPS) for each approved modeling component. This is mandatory and must include:

A. Ownership

- Named role/team responsible for:
 - monitoring
 - retraining/recalibration
 - responding to incidents

B. Monitoring requirements

Define what must be monitored:

- Input data drift (schema change, missingness spikes)
- Output drift (prediction distribution shift)
- Performance drift (if ground truth becomes available later)

C. Review cadence

- Weekly/monthly review schedule
- Trigger thresholds that require investigation

D. Retraining policy

- Event-driven retraining conditions (schema change, drift detected)
- Time-based retraining conditions (monthly/quarterly)

Sculley et al. (2015) emphasize that ML systems accrue hidden technical debt through ongoing “configuration” and “glue code” and through the continuing need to monitor dependencies. This contract makes those obligations explicit *before* modeling begins.

4.7 Define Failure Containment and Graceful Degradation Behavior

For every modeling component, the data scientist must define a Failure Mode and Degradation Plan (FMDP):

- What output is produced if the model fails?
 - last-known-good value
 - simple baseline forecast
 - deterministic rule fallback
- How will downstream steps be notified the output is degraded?
 - explicit “degraded mode” flag
 - confidence interval widening
 - alert log entry
- What prevents silent error propagation?
 - validation checks
 - threshold guards

- schema and range enforcement

Components without graceful degradation are restricted to IRL-1 methods or non-model approaches.

Outcome Artifact: Modeling Governance Record (MGR)

At the conclusion of Step 4, the data scientist produces a Modeling Governance Record that includes:

- Candidate Modeling Register (CMR)
- Model Output Contract (MOC) for each candidate
- Paradigm assignment and justification
- Interpretability Requirement Level + allowed model class restrictions
- Model Assumption Ledger (MAL)
- Model Operations Contract (MOC-OPS)
- Failure Mode and Degradation Plan (FMDP)

Only components with complete MGR entries are permitted to proceed to implementation. The MGR functions as a hard gate to prevent opportunistic model experimentation and to ensure that modeling, forecasting, and decision-science components are introduced only when justified, maintainable, and interpretable.

Effectiveness	Score: 5
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation:	

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation:	

Data Scientist Usability	Score: 5
---------------------------------	-----------------

Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.
Observation:

Operational Validity	Score: 5
Ensures the step produces results that remain aligned with domain context and business needs	
Observation:	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation:	

Reject or Accept Hypothesis
Notes: This step includes the level of detail I find acceptable.

Citations to Be Used in this Portion of the Framework
Camm, J. D., Galbreth, M., Bell, D. R., & Everitt, B. (2023). Data science and decision science skills: Are they different and does it matter? <i>Harvard Data Science Review</i>, 5(3), Article 32. Cleveland, W. S. (2001). Data science: An action plan for expanding the technical areas of the field of statistics. <i>International Statistical Review</i>, 69(1), 21–26. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. In <i>Proceedings of the 29th International Conference on Neural Information Processing Systems (NIPS '15)</i> (pp. 2503–2511).

VISUALIZATION AND COMMUNICATION DESIGN

Proposed Framework Procedure
After analytical outputs have been generated, the data scientist creates visualizations to present results to stakeholders. Charts, tables, and dashboards are used to summarize findings and make insights easier to understand. Visualization tools are selected based on availability and familiarity, and outputs are designed to display key results clearly. These visualizations are then shared with end users to support interpretation and decision-making.

Effectiveness	Score: 1
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: Visualization is treated as a reporting afterthought rather than a design activity.	

Framework Alignment	Score: 1
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Disconnected from prior decisions, users, and analytical intent.	

Data Scientist Usability	Score:
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: Easy to execute but conceptually shallow.	

Operational Validity	Score: 1
Ensures the step produces results that remain aligned with domain context and business needs	

Observation: No consideration of user context or decision use.

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: Trivially easy, but low value.	

Reject or Accept Hypothesis
Notes: Really just the first draft of this step.

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
Once analytical direction and modeling considerations have been established, the data scientist considers how analytical results should be communicated to end users. Visualization is used to translate analytical outputs into interpretable formats that support understanding and discussion. At this stage, the data scientist reflects on who the intended users are and what information they require to interpret results correctly. This version recognizes that visualization choices influence how results are perceived and acted upon. However, visualization remains primarily output-driven, focusing on presenting analytical results rather than shaping how decisions are made. Considerations such as cognitive load, interpretability, and the distinction between exploration and communication are acknowledged but not fully integrated into the design process.

Effectiveness	Score: 3
Assesses whether the framework step contributes meaningfully to the modernization goal	

Observation: Improves clarity but still treats visualization as secondary.

Framework Alignment	Score: 3
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Some alignment with user needs, but weak integration with earlier steps.	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: More guidance, but still underspecified.	

Operational Validity	Score: 3
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Acknowledges users but does not center decisions.	

Feasibility	Score: 4
Determines whether the step is practical to implement without excessive complexity	
Observation: Conceptual holes causing errors	

Reject or Accept Hypothesis
Notes: I will need to further read through by resources to contribute further to this step and move it from the reject to accept stage of this metric framework.

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
After analytical intent has been established and modeling decisions have been explicitly governed, the data scientist must design visual artifacts that translate analytical outputs into forms that support correct interpretation and responsible decision-making. The purpose of this step is to ensure that analytical results are not only technically valid, but legible and usable within the organization. Visualization is therefore treated as a core component of the system design, tightly coupled to analytical logic and operational context, rather than as a presentation layer applied after technical work is complete. This step exists to prevent a common failure mode in modernized pipelines: systems that produce improved analytics but lose the operational understanding that made legacy outputs actionable. Visualizations are the primary interface through which Domain Experts and decision-makers interact with the pipeline. As such, visualization design must preserve existing interpretive practices while making analytical changes explicit. Grounding Visualization Design in Decision Use and User Context The data scientist begins by anchoring each visualization to a specific decision, workflow, or judgment identified earlier in the framework. Visual artifacts are not designed generically; each must have a defined operational role. For example, a visualization supporting weekly planning discussions must emphasize stability, comparability across time, and clarity under time constraints, whereas a visualization supporting analytical review may allow for greater detail and flexibility. Visual outputs intended for operational decision-makers must prioritize clarity, consistency, and interpretive stability over visual novelty. Axes, scales, color encodings, and chart types should remain fixed wherever possible to support rapid recognition and comparison. Visual outputs intended for analytical stakeholders may support trend inspection or scenario comparison, but must remain bounded by the interpretive capacity of the intended audience. This alignment ensures that visualizations support decision-making rather than merely displaying analytical sophistication (McGowan et al., 2023). Leveraging Existing Visual Conventions to Preserve Operational Knowledge A critical requirement of this step is the deliberate reuse of visual forms already familiar to Domain Experts and operational stakeholders. In many organizations, legacy visualizations, such as recurring reports, dashboards, or spreadsheet summaries, encode accumulated operational knowledge. These artifacts carry shared understanding about normal behavior,

acceptable variation, and meaningful deviations. Replacing them outright risks severing the connection between improved analytics and established decision practices.

Accordingly, the data scientist must inventory existing visual artifacts that are currently circulated within the organization and identify which are relied upon for routine decision-making. When a legacy visualization is well-understood, widely used, and aligned with the pipeline's decision role, the preferred modernization strategy is to **retain its visual structure while modernizing its underlying data, calculations, or analytical logic**.

In practice, this means preserving familiar chart types, layouts, labeling conventions, and scales wherever possible. For example, a time-series chart historically used to monitor operational performance may remain visually unchanged, even as the upstream pipeline is modernized to automate updates, improve data quality, or standardize calculations. By maintaining visual continuity, the data scientist ensures that operational knowledge accumulated over time remains accessible within the modernized system.

This principle is especially important when introducing predictive or model-based outputs. Rather than introducing entirely new visual paradigms, modeled outputs should be layered onto existing visual forms using minimal, explicit extensions—such as dashed lines, shaded regions, or annotations. These additions signal analytical change while allowing users to rely on familiar interpretive frames. This approach reduces cognitive burden, supports adoption, and minimizes the risk of misinterpretation.

Visual Structure for Descriptive, Forecasting, and Model-Based Outputs

The form of each visualization must reflect the analytical role of the underlying output. Descriptive analyses should be communicated using representations that emphasize historical patterns, scale, and relative magnitude, avoiding unnecessary precision that may imply analytical certainty where none exists.

When forecasting or predictive models are present, visualizations must explicitly distinguish between observed historical data and modeled or projected outputs. Forecasts must never appear as a seamless continuation of observed data without clear visual separation. Doing so risks presenting predictions as facts rather than estimates. Appropriate techniques include changes in line style, shading, or segmentation that make the transition between observed and modeled values unambiguous.

For scenario-based or simulation outputs, visualizations must support comparison rather than singular interpretation. Fan charts, scenario bands, or multiple trajectories should be used to convey the range of plausible outcomes. The objective is to reinforce that model outputs represent structured uncertainty rather than deterministic truth.

Coupling Visualization Design to Machine Learning Behavior

When machine learning models are part of the pipeline, visualization design must explicitly reflect the conditional and learned nature of model outputs. Visual artifacts should never abstract away the fact that outputs are dependent on data, assumptions, and model validity.

For point predictions, visualizations must contextualize outputs within historical ranges or expected variability. For probabilistic outputs, uncertainty must be visually encoded rather than relegated to documentation. Where models are valid only under certain conditions or time horizons, these boundaries must be visible or easily accessible to users. In this way, the visualization layer becomes an active mechanism for enforcing responsible model use rather than a passive display of results.

Communicating Assumptions, Provenance, and Update Behavior

Each decision-facing visualization must be accompanied by concise supporting documentation that describes how the output was produced. This includes data sources, update frequency, transformation logic, and any modeling assumptions that materially affect interpretation. While this information does not need to be embedded directly within the visualization itself, it must be readily accessible at the point of use.

For model-based outputs, documentation must clarify whether results are derived from observed data, learned relationships, or simulated scenarios. Users must be able to determine when the visualization was last updated and whether underlying data or models have changed. This transparency supports trust and prevents outdated or misaligned interpretations.

Separating Exploratory and Decision-Facing Visuals

The data scientist must enforce a strict separation between exploratory visualizations used during development and decision-facing visual artifacts intended for organizational use. Exploratory visuals may be complex and provisional, supporting hypothesis generation or model debugging. These artifacts must be refined, simplified, and stabilized before being promoted to operational use.

Decision-facing visualizations must be versioned, documented, and treated as system components subject to change control. This ensures that users encounter only representations intentionally designed for their decision context and prevents exploratory complexity from being inadvertently operationalized.

Effectiveness	Score: 5
Assesses whether the framework step contributes meaningfully to the modernization goal	

Observation:

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation:	

Data Scientist Usability	Score: 5
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation:	

Operational Validity	Score: 5
Ensures the step produces results that remain aligned with domain context and business needs	
Observation:	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation:	

Reject or Accept Hypothesis
Notes:

Citations to Be Used in this Portion of the Framework
--

McGowan, L. D., Peng, R. D., & Hicks, S. C. (2023). Design principles for data analysis. *Journal of Computational and Graphical Statistics*, 32(4), 1192–1207.

Tukey, J. W. (1962). The future of data analysis. *Annals of Mathematical Statistics*, 33(1), 1–67.

IMPLEMENTATION, EVALUATION, AND ITERATIVE VALIDATION

Proposed Framework Procedure

After models and visualizations have been designed, the data scientist implements the modernized pipeline. This includes writing code, integrating data sources, and deploying analytical components. Once implemented, results are reviewed to ensure the pipeline functions as expected. Any issues identified are corrected, and the pipeline is finalized for use.

Effectiveness

Score: 1

Assesses whether the framework step contributes meaningfully to the modernization goal

Observation:

Framework Alignment

Score: 1

Evaluates whether the step promotes modular, maintainable pipeline architecture

Observation:

Data Scientist Usability

Score: 3

Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.

Observation:

Operational Validity	Score: 2
Ensures the step produces results that remain aligned with domain context and business needs	
Observation:	

Feasibility	Score: 1
Determines whether the step is practical to implement without excessive complexity	
Observation:	

Reject or Accept Hypothesis
Notes:

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
Once analytical and visualization designs have been finalized, the data scientist implements the modernized pipeline and evaluates its behavior in practice. Implementation includes integrating updated pipeline components, operationalizing analytical logic, and producing decision-facing outputs. Evaluation at this stage extends beyond technical correctness to include whether outputs are interpretable, stable, and aligned with the analytical intent established earlier in the framework. The data scientist reviews pipeline outputs with Domain Experts and end users to confirm that results are meaningful and actionable. This review helps identify discrepancies between intended and realized behavior, such as outputs that technically satisfy specifications but fail to support decision-making effectively. While this version introduces user-centered validation, evaluation remains informal and lacks explicit criteria tied to earlier framework decisions.

Effectiveness	Score: 3
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: Relatively effective in tying the framework up	

Framework Alignment	Score: 3
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Aligns with the goal of creating a full fledged pipeline	

Data Scientist Usability	Score: 2
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: Needs more detail to ensure the data scientist can use this without error.	

Operational Validity	Score: 2
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: Again needs more detail	

Feasibility	Score: N/A
Determines whether the step is practical to implement without excessive complexity	
Observation: Has not been fully understood the time on this.	

Reject or Accept Hypothesis
Notes:

Citations to Be Used in this Portion of the Framework

Proposed Framework Procedure
After analytical intent has been clarified, modeling decisions have been bounded, and visualization and communication artifacts have been designed, the data scientist proceeds with implementation under explicit framework constraints. The purpose of this step is not merely to operationalize technical components, but to verify that the implemented system behaves in accordance with the decisions, assumptions, and design intent established throughout earlier stages of the framework. Implementation is therefore treated as a validation activity rather than a terminal phase. This step begins with controlled implementation, in which redesigned pipeline components are integrated incrementally rather than deployed wholesale. The data scientist preserves validated functionality from the legacy system while introducing modernized components in the order specified by the modernization roadmap. This sequencing ensures that system behavior remains interpretable and that deviations from expected outcomes can be traced to specific changes rather than to cumulative disruption. Once implementation is complete, the data scientist performs alignment validation across three dimensions: analytical behavior, communicative behavior, and operational behavior. Analytical validation ensures that the implemented system produces outputs consistent with the analytical role defined in Step 4 and the modeling constraints articulated in Step 5. This includes verifying that predictive outputs, where present, behave within expected ranges and respect documented assumptions, rather than silently introducing new analytical behavior. Communicative validation ensures that visual artifacts and supporting documentation function as intended in practice. The data scientist confirms that end users can interpret outputs correctly, distinguish between observed and inferred values, and understand uncertainty where applicable. Visualizations are evaluated not for aesthetic quality, but for interpretive stability and decision alignment. Any discrepancies between intended and actual interpretation are documented as design failures rather than user error, reinforcing the framework's responsibility for communicative clarity. Operational validation focuses on how the implemented pipeline integrates into real workflows. The data scientist verifies that outputs are delivered at appropriate times, that update frequencies align with decision cycles, and that failure modes are visible rather than silent. This includes confirming that fallback logic functions as designed, that degraded performance does not propagate undetected, and that maintenance responsibilities defined in earlier steps are feasible in practice. A critical feature of this step is traceable discrepancy handling. When misalignment is observed between intended and realized system behavior, the data scientist does not apply ad hoc fixes. Instead, discrepancies are explicitly traced back to the relevant framework step, context alignment, structural decomposition, modernization decisions, analytical direction, modeling constraints, or visualization

design. This controlled feedback mechanism preserves the integrity of the framework while allowing iterative refinement where necessary.

The step concludes with the creation of a framework closure artifact, which documents:

- Final pipeline structure and component responsibilities
- Analytical roles and modeling boundaries
- Visualization artifacts and communication conventions
- Known limitations, assumptions, and deferred modernization targets

This artifact serves as both a validation record and a handoff document, ensuring that system behavior remains interpretable and maintainable beyond the initial modernization effort.

By the conclusion of this step, the data scientist has demonstrated not only that the modernized pipeline functions, but that it functions as designed, within the constraints and intentions established by the framework. This distinction separates successful framework application from mere technical deployment. The framework remains active through implementation, ensuring that modernization outcomes reflect disciplined design rather than incidental correctness.

Effectiveness	Score: 3
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: The step needs to be further explained to be accepted fully.	

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Aligns with the goal of creating a framework focused on producing a full pipeline	

Data Scientist Usability	Score: 4
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: The bones of a great explanation are there but unfortunately need form information to be accepted.	

Operational Validity	Score: 5
Ensures the step produces results that remain aligned with domain context and business needs	
Observation: This stepp loops the DE back in which is great.	

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: Feasible to complete a final audit and understanding of the overall pipeline	

Reject or Accept Hypothesis
Notes:

Citations to Be Used in this Portion of the Framework
Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. <i>MIS Quarterly</i>, 28(1), 75–105. https://doi.org/10.2307/25148625 IBM. (2013). <i>The CRISP-DM methodology</i>. IBM Analytics. McGowan, L. D., Peng, R. D., & Hicks, S. C. (2023). Design principles for data analysis. <i>Journal of Computational and Graphical Statistics</i>, 32(4), 1192–1207.

Proposed Framework Procedure
After analytical intent has been clarified, modeling decisions have been bounded, and visualization artifacts have been finalized, the data scientist proceeds with implementation under explicit framework constraints. The purpose of this step is to ensure that the modernized pipeline executes in a standardized, reproducible, and inspectable manner, and that its overall health can be assessed without requiring deep inspection of individual components.

Implementation is treated as a validation and stabilization phase rather than a terminal handoff. At this stage, the framework remains active and prescriptive, governing how the system runs, how correctness is verified, and how failures are surfaced.

The data scientist must standardize pipeline execution such that the entire system can be run end-to-end by another data scientist without reliance on undocumented knowledge or ad hoc intervention.

At a minimum, this requires the data scientist to produce all three of the following:

1. A single, explicit pipeline entry point
The pipeline must have a clearly defined execution mechanism (e.g., a master script, Makefile target, workflow DAG, or orchestration command) that specifies the order in which components run. Execution order must not be implicit or dependent on manual sequencing.
2. A configuration specification that externalizes assumptions
All environment-dependent values (paths, thresholds, time windows, model parameters, refresh cadence) must be defined in a configuration file or structured parameter interface rather than embedded directly in code. This ensures that changes to execution behavior are deliberate and traceable.
3. Deterministic execution guarantees where applicable
Given the same inputs and configuration, the pipeline must produce the same outputs. If stochastic elements are present (e.g., sampling, randomized model training), the data scientist must explicitly document and control randomness through fixed seeds or bounded variability statements.

Together, these requirements ensure that the modernized pipeline is reproducible, auditable, and transferable. A pipeline that cannot be reliably re-run by another data scientist is considered incomplete under the framework.

Once implemented, the data scientist must validate that the pipeline behaves according to the analytical, communicative, and operational intent defined in earlier steps.

This validation must include:

- confirming that descriptive components remain deterministic and interpretable,
- confirming that predictive or learning-based components operate only within approved horizons and assumptions,
- verifying that decision-facing visual artifacts match the finalized designs from Step 5,
- and explicitly testing fallback or degraded-mode behavior for components where such behavior was specified.

Validation failures must be documented as framework misalignment, not implementation bugs. Corrective action must be taken by revisiting the relevant framework step rather than patching behavior locally.

In addition to component-level validation, the data scientist must implement a system-level health view that allows stakeholders to assess whether the pipeline is functioning as expected without examining individual scripts, models, or datasets.

This birds-eye view must be implemented as a concrete artifact and must include, at minimum, the following elements:

1. Stage-level execution status

Each major pipeline stage (ingestion, transformation, modeling, visualization output) must expose a clear success/failure indicator. This may take the form of:

- o a status table,
- o a checklist-style dashboard,
- o or a structured log summary.

2. Data freshness and completeness indicators

The health view must indicate whether key inputs and outputs are up to date relative to their expected cadence (e.g., last updated timestamp, expected vs actual row counts). This allows users to detect stale or incomplete data without inspecting raw files.

3. Degraded or fallback mode visibility

If any component is operating in a fallback or degraded mode (e.g., baseline model used instead of predictive model), this state must be explicitly surfaced. Silent degradation is not permitted.

4. High-level anomaly signals

The health view should surface simple indicators of abnormal behavior, such as missing outputs, failed stages, or values outside expected ranges. These signals are not intended for diagnosis, but for early detection.

This birds-eye artifact may be implemented as a lightweight dashboard, a summary report generated at the end of each pipeline run, or a standardized status output consumed by downstream systems. The key requirement is that it is interpretable by individuals who are not intimately familiar with the pipeline's internal implementation.

If discrepancies are observed, such as unexpected outputs, misaligned visuals, or operational failures, the data scientist must not apply ad hoc fixes. Each discrepancy must be traced back to the framework step in which the relevant decision was made and addressed at that level.

This traceability ensures that iteration strengthens the framework rather than bypassing it.

By the conclusion of Step 6, the modernized pipeline must:

- execute through a standardized and reproducible mechanism,
- be runnable and understandable by another data scientist,
- expose a concrete birds-eye view of system health,
- and have been validated against the analytical, communicative, and operational intent defined by the framework.

At this point, modernization is considered complete not because the pipeline runs, but because it runs predictably, transparently, and observably. This distinction ensures that the framework produces systems that remain usable, maintainable, and interpretable beyond the initial implementation effort.

Effectiveness	Score: 5
Assesses whether the framework step contributes meaningfully to the modernization goal	
Observation: The addition of details has made this step effective.	

Framework Alignment	Score: 5
Evaluates whether the step promotes modular, maintainable pipeline architecture	
Observation: Aligns with the goal of creating a framework focused on producing a full pipeline	

Data Scientist Usability	Score: 5
Measures how easy the step is for a data scientist to follow, reuse, and apply consistently.	
Observation: More information has been added to aid in the data scientists usability of this step.	

Operational Validity	Score: 5
----------------------	----------

Ensures the step produces results that remain aligned with domain context and business needs
Observation: This stepp loops the DE back in which is great.

Feasibility	Score: 5
Determines whether the step is practical to implement without excessive complexity	
Observation: Feasible to complete a final audit and understanding of the overall pipeline	

Reject or Accept Hypothesis
Notes:

Citations to Be Used in this Portion of the Framework
Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. <i>MIS Quarterly</i>, 28(1), 75–105. https://doi.org/10.2307/25148625 IBM. (2013). <i>The CRISP-DM methodology</i>. IBM Analytics. McGowan, L. D., Peng, R. D., & Hicks, S. C. (2023). Design principles for data analysis. <i>Journal of Computational and Graphical Statistics</i>, 32(4), 1192–1207.

APPENDIX B: Operations Manager User Manual

PLANT CAPACITY PLANNER

Operations Manager User Manual

What this tool does:

Turns 18 months of your plant's own run data into a 30-day capacity plan your operations team and sales team can both read and act on.

1. What Is This Tool?

The Plant Capacity Planner is a dashboard that answers the most common questions your operations and sales teams ask every week:

- How much can we actually make in the next 30 days?
- Which production lines are our biggest contributors, and which are the bottlenecks?
- Are we running efficiently, or are we leaving output on the table?
- What can sales safely promise to customers?
- If I add more downtime or changeovers, what happens to output?

It does this by reading your plant's own historical production records, 18 months of actual run data, and using that history to project realistic capacity for the coming month. There is no manual data entry required to use the forecasting features; the numbers come from what your plant has actually done, not from engineering estimates or vendor specs.

1.1 What the Tool Tracks

The tool is built around your plant's Barrel, the formula/process lines that correspond to specific product families in your RCCP framework:

Barrels	Product Family	Grade	Notes
B01, B02, B03	Specific Product	70A	Specific notes for this barrel
B04, B05, B06	Specific Product	60A	Specific notes for this barrel
B07, B08, B09	Specific Product	80A	Specific notes for this barrel

Barrels	Product Family	Grade	Notes
B10, B11, B12	Specific Product	80B	Specific notes for this barrel
B13	Specific Product	90A	Specific notes for this barrel

Each Barrel can fill into any container size, Quarts, Gallons, or Pails (5-gallon), depending on the production run. The tool tracks all three and converts everything to a common gallon-equivalent so you can compare output across lines fairly.

1.2 The Unit You Will See Everywhere: Gallon-Equivalents

Because different container sizes hold different amounts of liquid, the tool converts everything to gallon-equivalents before adding them up. This is the only way to fairly compare a quart run against a pail run.

Container	Gallons Each	What This Means
Quart (Qt)	0.25 gal	4 quarts = 1 gallon-equivalent
Gallon (Gal)	1.00 gal	1:1 conversion
Pail (5-Gal)	5.00 gal	1 pail = 5 gallon-equivalents

Why this matters for totals

If you see "5,600 gal/day" in the dashboard, that number already accounts for the full mix of quarts, gallons, and pails your lines are running. It is not just gallon-sized containers, it is the total liquid volume expressed in a common unit. A mid-size plant like yours typically runs 3,000-8,000 gallon-equivalents per day.

2. Opening the Tool and Finding Your Way Around

2.1 How to Open the Tool

The Capacity Planner runs in your web browser: the same way you would open a website. You do not need to install anything. Your IT team or the person who set up the tool will give you the web address (it looks something like: <http://localhost:8501>).

1. Open your web browser (Chrome, Edge, or Safari all work).

2. Type or paste the address into the address bar at the top and press Enter.
3. The Plant Capacity Planner will load within a few seconds.

If the page does not load

The tool requires the pipeline to have been run first. Ask whoever manages the system to run "99_run_all.py" to generate the latest data before you open the tool. To check the health of all pipeline stages at once, run: python pipeline_health.py

2.2 The Four Tabs

Once the tool opens, you will see four tabs across the top of the screen. Think of them as four different views into the same data:

Tab Name	What It Is For
Capacity Overview	Your daily operations view. Shows the full planned output picture, how it compares to what the plant can realistically achieve, and which weeks look heavy or light.
Sales & Commitments	Answers the question: "What can we safely promise to customers?" Shows capacity broken down by container type (Quart / Gallon / Pail) with low / typical / high ranges.
Efficiency Gap Analysis	Shows where your capacity is being lost, downtime, changeovers, or run rate, and what closing the gap to 90% efficiency would mean in real gallons.
What-If Planning Wizard	Lets you model a custom plan by adjusting your shift schedule, downtime, changeovers, and efficiency assumptions. Run this first, it feeds the other three tabs.

Start here every time

The recommended workflow is: open the What-If Planning Wizard first, configure your assumptions, click Run Plan, then review the results in the Capacity Overview and Sales tabs.

3. The What-If Planning Wizard

This is where you build your capacity plan. Work through the six steps from top to bottom, then click Run Plan. The whole process takes about two minutes.

Step 1: How Long Are You Planning For?

Set three things:

- Plan start date: the first day of your planning window (usually today or next Monday).
- Number of days to plan, typically 7 to 30. For a monthly operations meeting, use 30.
- Simulation quality: "Standard (200 futures)" is the right choice for day-to-day use. Only switch to "Thorough" if you need tighter uncertainty bands for a customer commitment.

Step 2: What Kind of Week Is This?

Choose the preset that best describes your upcoming period. This sets sensible defaults for downtime, changeovers, and efficiency, you can fine-tune them in Step 4.

Preset	When to Use It
Normal week	Standard operations: no unusual events expected.
High downtime week	More equipment issues or scheduled maintenance than usual.
Heavy changeover week	Many product or SKU switches expected. Reduces net run time significantly.
Staffing shortage	Reduced crew: expect lower throughput and some efficiency loss.
Custom	Set every value manually in Step 4. Use when none of the above fit.

Step 3: Set Your Shift Schedule and Mark Closures

The tool pre-fills a default shift schedule (3 x 8-hour shifts on weekdays, 2 x 12-hour shifts on weekends). Edit the table to match your actual schedule for the planning period.

Below the schedule, a daily calendar appears showing every day in your planning window. Use the "Closed (zero production)" checkbox to mark any holidays, shutdown days, or planned maintenance days. The tool will set those days to zero output automatically.

Holiday auto-fill

If the "Auto-mark all federal holidays as CLOSED" button is visible, click it to mark US federal holidays automatically. You can still uncheck individual days if your plant runs through a holiday.

Step 4: Fine-Tune Your Operational Assumptions

Three inputs control how the model adjusts from your historical baseline:

- Expected downtime per shift (minutes): Average unplanned downtime per shift: breakdowns, minor stoppages, waiting for materials. The historical baseline is about 30 minutes. Increase this if you expect a difficult week.

- Expected line efficiency (%): 100% means the plant will perform at its historical typical rate. Set to 90% if you expect a slower-than-usual week; set to 110% if the lines have been running exceptionally well and you expect that to continue.
- Changeovers per line per day: How many product or SKU switches each line does in a day. More changeovers = less run time. The default is around 3 per day.

When to leave these at defaults

If you chose a preset in Step 2, these values are already set appropriately. Only change them if you have specific information about the upcoming period that is different from the preset.

The tool will warn you if any input looks unusually extreme, for example, downtime over 120 minutes per shift or more than 10 changeovers per day. These warnings do not stop you from running the plan; they are just reminders to double-check.

Step 5: How Should the Plan Handle Uncertainty?

The model simulates 200 possible futures (or more, depending on your quality setting) based on how variable your plant's output has been historically. Your planning style controls how cautious or aggressive the final plan target is:

- Balanced: Good default. The plan target sits at a level equally likely to be above or below actual output.
- Conservative (avoid over-promising): The plan is set lower, reducing the chance of missing a commitment. Use this when making firm customer promises.
- Aggressive (maximize output): The plan pushes for higher output targets. Higher risk of falling short. Use only for internal stretch targets.

Step 6: Run the Plan

Click the blue "Run Plan" button. The tool will run for a few seconds (longer if you chose Thorough simulation quality), then display your results immediately below the button.

The results show four summary numbers, total planned output, average per day, peak day, and lowest day, all in gallon-equivalents. A chart shows your planned output line against the realistic capacity range (P10 to P90).

Once the plan is generated, the Capacity Overview and Sales tabs automatically update to reflect your new plan. You do not need to do anything else to update them.

Downloading your plan

At the bottom of the results, two download buttons appear. Use "Download detailed plan (CSV)" if you need the day-by-day line-level numbers in a spreadsheet. Use "Download one-page summary (TXT)" for a plain-text summary you can paste into an email or attach to a meeting agenda.

4. Capacity Overview Tab

This tab gives you the full operations picture after you have run a plan. It is designed for your daily or weekly ops review.

4.1 The KPI Strip (Top Numbers)

Five numbers appear across the top of the page after you run a plan:

Number	What It Means
Total Planned Output (Gal-Equiv)	The total gallon-equivalents your plan calls for across the entire planning horizon. This is your plan number.
Typical Achievable (Median)	What the plant typically produces based on historical run rates. Half of simulated futures are above this; half below.
Conservative Estimate (P10)	Only 10% of simulated futures fall below this. Use for firm commitments where missing would be costly.
Optimistic Estimate (P90)	The plant achieves this only on its best 10% of days. Do not commit to this level with customers.
Plan vs. Typical	How much your plan exceeds or falls short of typical capacity. Shown as a percentage. Negative means your plan is conservative; positive means it is aggressive.

4.2 Plant Stress Badge

Below the KPI strip, a colored badge tells you whether your plan is realistic:

- Low stress / Very conservative: Your plan is within or below what the plant typically achieves. Good for a week when you want a comfortable buffer.
- Moderate stress: Your plan is slightly above typical. Achievable in a good week but watch closely.
- High stress: Your plan exceeds typical capacity by more than 10%. The plant will need to perform above its historical norm to hit this target. Consider switching to a Conservative planning style.

4.3 The Capacity Risk Chart

The main chart shows three lines:

- Your Plan (gal): The daily output your plan calls for.
- Realistic Max (P90): What the plant achieves only on its strongest days (top 10% historically). Treat this as the ceiling.

- Realistic Min (P10): What the plant achieves or exceeds 90% of the time. Treat this as the floor.

Your plan line should sit between the two realistic boundary lines. If it rises above the Realistic Max line on multiple days, you are over-committing. A green, yellow, or red badge above the chart tells you how many days have this problem.

4.4 Weekly Output Breakdown Chart

Below the risk chart, a stacked bar chart shows planned output by calendar week, with each color representing one HSD line. Use this to:

- Spot weeks that look unusually light (may indicate a shutdown day or under-scheduled shift).
- See which HSD lines are carrying the most volume in each week.
- Identify whether a heavy week is driven by one or two lines running hard or balanced across all lines.

4.5 Planned Output by Line Table

A table at the bottom of the tab shows each HSD line's planned output for the next 7 and next 30 days, alongside the typical (median) simulated capacity for that line. The "Plan vs. Typical" column shows a ratio: 1.00x means your plan exactly matches typical performance; above 1.10x means the plan is asking for more than the line typically delivers.

5. Sales & Commitments Tab

This tab is designed to be shared with your sales team or discussed in S&OP meetings. It translates the operations plan into the language sales needs: how much can we ship, broken down by container type, with clear low/likely/high ranges.

5.1 Top KPIs

- Total Available Capacity: Total planned gallon-equivalents for the full planning horizon.
- Average Daily Output: Planned output per day. A useful number to quote in sales conversations: "We plan to produce X gallons per day this month."
- Next 7-Day Forecast: Total planned output for the next 7 days only. Useful for near-term shipping commitments.

5.2 Estimated Capacity by Container Type

This section breaks down total planned capacity across your three container types: Quart (Qt), Gallon (Gal), and Pail (5-Gal). The split is based on each HSD line's historical container mix, how often each line has run quarts vs. gallons vs. pails in the past.

Important caveat for sales

This is an estimated mix based on historical patterns, not a confirmed production schedule. If a specific customer order requires a particular container type that deviates significantly from the historical mix, flag it to operations so the schedule can be adjusted accordingly.

5.3 Product Mix Risk Signal

A colored badge warns if the assumed product mix contains a high proportion of slow-running SKUs. Slow SKUs (complex formulas, difficult viscosities) reduce the plant's effective output even if the hours are available.

- Mix looks healthy: The assumed product mix is broadly efficient. The headline capacity number is reliable.
- Slow SKUs notable: A meaningful share of the mix is slow running. The actual output may come in 5-10% below the headline number.
- Slow SKUs dominate mix: A large portion of the assumed mix is slow-running SKUs. Treat the headline number with caution; actual output could be materially lower.

5.4 Uncertainty Range: Which Number to Promise

Number	What It Means	When to Use It
Conservative Commitment (P10)	Only 10% of simulated futures fall below this.	Firm customer commitments. Very safe to promise.
Most Likely Capacity (P50)	The median, half of futures are above, half below.	Internal planning and S&OP meetings.
Best-Case Capacity (P90)	Only achievable on the plant's best days.	Never commit this to customers.

The golden rule for sales commitments

Always promise based on the P10 (Conservative) figure. Promise the P50 only if you have a buffer in inventory or lead time to absorb a shortfall. Never promise based on P90.

6. Efficiency Gap Analysis Tab

This tab answers the question operations managers and plant leaders need: "We know we're not running at 100%: but exactly what are we leaving on the table, and what would getting to 90% efficiency actually mean?" All numbers here are based on your own plant's historical run data. The "best-day ceiling" is not an engineering theoretical maximum, it is what your plant achieved on its best 10% of days. The targets are realistic because they come from your own history.

6.1 Efficiency Badge and KPIs

At the top, a badge shows your current efficiency status:

- At or above 90% target: The plant is running at or near its realistic best-day performance. Excellent.
- Getting close to 90% target: Efficiency is between 85-90%. Meaningful room for improvement remains.
- Below typical range: Efficiency is below 85%. Something structural, downtime, changeovers, or run rate, is dragging output down.

Four KPI cards accompany the badge:

- Current Efficiency: Your median daily output rate as a percentage of your plant's P90 best-day rate.
- Gap to 90% Target: How many additional gallons per day the plant would produce if it reached 90% efficiency.
- Lost to Downtime: Gallons per day lost because lines are offline.
- Lost to Changeovers: Gallons per day lost to product/SKU switches.

6.2 Where Capacity Is Going: Loss Waterfall

A bar chart shows how the gap between current performance and the 90% target is split across three categories:

- Lost to Downtime: Lines are offline due to breakdowns or planned maintenance. Reducing downtime frequency or duration directly recovers these gallons.
- Lost to Changeovers: Lines are offline for product/SKU switches. Reducing the number of changeovers per day, or the time each changeover takes, recovers these gallons.
- Lost to Run Rate Gap: Lines are running but not as fast as they could. This is where formula complexity, ingredient variability, and operator skill show up. This is often the hardest loss to recover.

How to use this in practice

Focus improvement efforts on whichever bar is tallest. If Downtime is largest, prioritise preventive maintenance and rapid response to breakdowns. If Changeovers are largest, look at reducing SKU switches or running longer same-product runs. If Run Rate Gap is largest, investigate process consistency, input variability, and operator training.

6.3 Daily Plant Output Chart

This chart shows three lines over your 18-month history:

- Realized Output (gal): What the plant actually produced each day, in gallon-equivalents.
- Realistic Target (gal): The daily target based on 85% of your rolling 30-day best-day performance. The plant should meet or exceed this target on roughly half of days.

- **Best-Day Ceiling (gal):** What the plant achieves on its strongest days. Most days will fall below this line.

A badge above the chart tells you what percentage of historical days the plant met the realistic target. A healthy plant hits the target on 40-60% of days. If the percentage is much lower, look at the Loss Waterfall to understand why.

6.4 Run Rate Efficiency Trend

This chart shows whether the plant's efficiency has been improving, holding steady, or declining over time. The line is a rolling 30-day average of efficiency percentage compared to the best-day ceiling.

- **Trending up:** Operational improvements are working.
- **Flat:** Stable but not improving. Consider whether improvement initiatives are having an effect.
- **Trending down:** Something structural is degrading. Could be more downtime, a harder SKU mix, equipment ageing, or staffing changes.

6.5 Per-Line Summary Table

A table at the bottom of the tab shows each HSD line's current output vs. its 90% target and best-day maximum. The Gap column shows how many additional gallon-equivalents per day each line would contribute if it reached 90% efficiency. Focus improvement efforts on the lines with the largest gaps.

7. Understanding the Key Numbers

7.1 CFR: Container Fill Rate

CFR (Container Fill Rate) is the speed at which a line fills containers during actual run time, measured in units per hour. It is calculated from your own batch records:

$$\text{CFR} = \text{Good units produced} / \text{Actual run hours}$$

It only counts time the line was actually running, downtime and changeover hours are excluded. This makes CFR a pure measure of line speed, not overall utilisation. Typical CFR ranges for this plant:

Container Type	Typical CFR (units/hr)	In Gal-Equiv/hr	Notes
Quart	14-31 units/hr	3.5-8 gal-equiv/hr	High unit count, small volume each

Container Type	Typical CFR (units/hr)	In Gal-Equiv/hr	Notes
Gallon	10-23 units/hr	10-23 gal-equiv/hr	Best balance of speed and volume
Pail (5-Gal)	3-7 units/hr	15-35 gal-equiv/hr	Slow fill but high volume per unit

7.2 Effective Hours

Not all planned shift hours are productive. Effective hours are what remains after subtracting:

- Downtime hours (breakdowns, stoppages, material waits)
- Changeover hours (time to switch from one product/SKU to another)
- Sanitation hours (cleaning between runs)
- Break hours (operator breaks built into the shift)

Capacity = CFR x Effective Hours. Anything that reduces effective hours directly reduces output, even if the line's fill rate is excellent.

7.3 P10 / P50 / P90: What the Percentiles Mean

The tool generates 200 simulated versions of the coming month, each slightly different based on how variable your plant's performance has been historically. The percentiles summarize the range of those simulations:

- P10: Only 10% of simulated futures produce less than this. Very safe floor.
- P50: Half of simulated futures are above this, half below. The most likely outcome.
- P90: Only 10% of simulated futures produce more than this. Requires a strong week.

The gap between P10 and P90 is your uncertainty band. It reflects how variable your plant's output has been. A narrow band means the plant is very consistent. A wide band means output varies significantly from day to day or week to week.

8. Common Tasks: Step-by-Step

8.1 Building a Monthly Capacity Plan for an S&OP Meeting

4. Open the What-If Planning Wizard tab.
5. Set Plan start date to the first day of the planning month.
6. Set Number of days to plan to 30.

7. Set Simulation quality to Standard (200 futures).
8. Choose the preset that best matches the upcoming month's expected conditions.
9. In Step 3, mark any plant closure days (holidays, scheduled maintenance) as Closed.
10. Leave Step 4 at preset defaults unless you have specific information.
11. Choose Conservative planning style in Step 5.
12. Click Run Plan.
13. Review the results. Check the Plant Stress badge, it should be green or yellow.
14. Switch to the Sales & Commitments tab. Quote the P10 (Conservative Commitment) figure to sales.
15. Download the one-page summary text file and attach it to your meeting agenda.

8.2 Answering a Sales Request for a Specific Container Type

16. Run a plan in the What-If Wizard (or use the most recent plan if it is still current).
17. Go to the Sales & Commitments tab.
18. Find the "Estimated Capacity by Container Type" section.
19. Read off the Planned Units and Share of Total for the container type the customer is asking about.
20. Quote the P10 (Conservative Commitment) figure for that container type if you need a safe number.

Remember

The container mix is estimated from historical patterns. If the customer order requires a significantly different mix than usual, for example, all pails when you normally run mostly gallons, flag this to your scheduler so the line plan can be adjusted.

8.3 Understanding Why Capacity Dropped This Week

21. Go to the Efficiency Gap Analysis tab.
22. Check the Loss Waterfall chart. Which bar is tallest?
23. If Downtime is tallest: something is taking lines offline more than usual. Review equipment logs for this week.
24. If Changeovers is tallest: the SKU mix is driving more switches than normal. Look at whether the schedule can be sequenced to reduce changeovers.
25. If Run Rate Gap is tallest: lines are running but slower than their best. Check process logs, batch records, and operator notes.
26. Check the Run Rate Efficiency Trend chart. Is this week's dip a one-off or part of a longer downward trend?

8.4 Checking Whether a Specific Week Is Over-Committed

27. Run a plan in the What-If Wizard covering the week in question.

28. Go to the Capacity Overview tab.
29. Look at the Capacity Risk Chart. Are any days shown with a red over-committed flag?
30. If yes, expand the over-committed days table to see which specific dates and by how many gallons the plan exceeds the realistic maximum.
31. Either reduce the plan for those days or switch to a Conservative planning style and re-run.

9. Pipeline Health Check

The health check is a single command that inspects every stage of the data pipeline and immediately tells you whether each one succeeded, has warnings, or has failed. Run it any time the dashboard looks wrong, after a new data load, or as part of a regular weekly check.

9.1 Running the Check

Open a terminal or command prompt in the same folder as your pipeline scripts and run:

```
python pipeline_health.py
```

That single command checks all six pipeline stages simultaneously and prints a colour-coded report in under one second. No arguments are needed for standard use.

You do not need to understand the pipeline to read the report

Green bullets mean a stage is healthy. Yellow means something needs attention but the pipeline is still usable. Red means a stage has failed and the dashboard data may be missing or stale. The last line of the report always gives a plain verdict: PIPELINE HEALTHY, PIPELINE DEGRADED, or PIPELINE UNHEALTHY.

9.2 Reading the Report

The report prints a summary line for each stage, then a detailed breakdown beneath. Here is what each status means:

- OK (green checkmark): The stage completed successfully. All output files are present, correctly shaped, and within expected value ranges.
- WARN (yellow warning): The stage ran but something unusual was detected. The dashboard will still work, but you should investigate. Common causes: no config file found, plots are older than the underlying data, or a small number of null values in a non-critical column.
- FAIL (red cross): The stage did not produce its expected output, or the output is missing, empty, or out of range. The dashboard may show zeroes or fail to load until this is resolved. Run `99_run_all.py` to regenerate all outputs.
- SKIP: The check could not run because a required input was missing. Fix upstream FAIL stages first.

9.3 What Each Stage Checks

The health checker inspects all six stages of the pipeline in one pass. Below is a plain-language summary of what it verifies at each stage.

Stage 0: Raw Data Inputs

Confirms that all four input data files exist in the data/ folder: production.csv, calendar.csv, downtime_events.csv, and changeovers.csv. Checks that each file has the required columns and that the proportion of missing values in key fields is below five percent.

Stage 1: CFR Computation

Confirms that cfr_daily.csv was produced, contains all 13 HSD lines, has no null values in critical columns, and that CFR values fall within expected ranges for each container type. Also verifies that the gallon-equivalent conversion ratio is consistent with the known container mix.

Stage 2: Capacity Aggregation

Confirms that capacity_daily.csv exists, contains no negative or implausible values, and that the plant-level daily total falls within the expected range for a mid-size plant (roughly 3,000 to 8,000 gallon-equivalents per day). Also checks that no forecast-derived columns have leaked into this descriptive stage.

Stage 3: Visualization

Confirms that output plot files were generated in the out/plots/ folder and that they are not older than the underlying capacity data by more than one hour. Checks that the forecast plot is present.

Stage 4: Stochastic Forecasting

Confirms that all three forecast output files are present. Verifies that the quantile ordering is correct (P10 must be less than or equal to P50, which must be less than or equal to P90) in every row. Checks that the forecast covers at least 30 days and that the uncertainty spread between P10 and P90 is not implausibly wide. Confirms that no historical dates appear in the forecast output.

Stage 5: CFR Ranking

Confirms that sku_ranks.csv exists with all expected SKUs ranked. Verifies that no forecast-derived columns appear in this purely descriptive output, which would indicate a stage isolation violation.

Stage 6: Reproducibility

Checks that the single entry point script (99_run_all.py) is present, that a configuration file exists for externalized parameters, that no hardcoded absolute file paths appear in any pipeline script, and that a fixed random seed is set in the forecasting script to ensure deterministic output.

9.4 Command Options

The standard one-line command covers all normal use cases. Three optional flags are available for specific situations:

Command	What It Does
<code>python pipeline_health.py --watch</code>	Re-runs the check automatically every 30 seconds. Useful when monitoring a long pipeline run. Press Ctrl+C to stop.
<code>python pipeline_health.py --watch --interval 60</code>	Re-runs every 60 seconds instead of 30.
<code>python pipeline_health.py --json</code>	Outputs the report as machine-readable JSON instead of text. Use this if you want to feed the results into another tool or logging system.
<code>python pipeline_health.py --out_dir ./out --data_dir ./data</code>	Specify custom locations for the output and data folders if yours are not in the default locations.

9.5 What to Do When a Stage Fails

The most common cause of a pipeline failure is that the pipeline has not been run recently. If you see FAIL on any stage, start here:

- Open a terminal in the pipeline folder.
- Run: `python 99_run_all.py`
- Wait for the pipeline to complete (typically two to five minutes).
- Run the health check again: `python pipeline_health.py`
- If failures persist, contact your system administrator and share the full health report output.

The exit code tells you the overall result

The health checker exits with code 0 if all stages are healthy or only have warnings. It exits with code 1 if any stage has failed. This means it can be used in automated monitoring or scheduled scripts: if it returns 1, send an alert.

10. Glossary of Terms

Plain-language definitions of every term used in the tool.

Term	What It Means
CFR (Container Fill Rate)	The speed at which a line fills containers during actual run time, measured in units per hour. Does not include downtime or changeover time.
Gallon-Equivalent (gal)	A common unit for comparing output across container sizes. Quarts are divided by 4; gallons stay as-is; pails are multiplied by 5.
Effective Hours	Planned shift hours minus downtime, changeovers, sanitation, and breaks. The time the line is actually filling containers.
Capacity	CFR multiplied by effective hours. The total output a line can produce in a day.
HSD (Heat-Set Dispersion)	A formula/process line in the RCCP framework. Each HSD corresponds to a product family and can fill any container size.
W/C Grade	The work centre grade assigned to each HSD in the RCCP system. Used to match HSD lines to product families.
P10 / P50 / P90	Percentiles from the stochastic simulation. P10 = floor (very safe). P50 = most likely. P90 = ceiling (best-case days only).
Stochastic Simulation	The tool runs 200 slightly different versions of the future based on historical variability. The resulting range is your uncertainty band.
Newsvendor Plan	The mathematical method used to choose the plan target. It balances the cost of under-producing (missing a commitment) against the cost of over-planning (idle capacity).
RCCP (Rough Cut Capacity Plan)	The bi-weekly planning process used by the plant manager and regional supply chain. This tool is designed to support and inform RCCP meetings.
Downtime	Time when a line is not running due to breakdowns, material shortages, or operator issues. Reduces effective hours directly.
Changeover	The time taken to switch a line from one product or SKU to another. Includes cleaning, setup, and first-article inspection.
SKU (Stock Keeping Unit)	A specific product in a specific container size. A product in a gallon container and the same product in a quart container are two different SKUs.

Term	What It Means
Ramp Limit	An optional constraint that prevents the plan from jumping sharply between days. Limits the day-to-day change in planned output to a set number of gallons.
Plant Stress	A measure of how much the plan asks of the plant relative to what it typically achieves. High stress (>1.10x) means the plan requires above-historical performance.

11. Quick Reference Card

The Recommended Weekly Workflow

32. Open the What-If Planning Wizard.
33. Set your planning window (start date + number of days).
34. Choose your preset (Normal week, High downtime, etc.).
35. Mark any plant closure days in the daily calendar.
36. Adjust Step 4 assumptions only if you have specific information.
37. Choose Conservative planning style for anything customer-facing.
38. Click Run Plan.
39. Check the Plant Stress badge in the Capacity Overview tab.
40. Share the P10 number from the Sales tab with your sales team.
41. Download the one-page summary for your meeting.

Which Number to Use When

Situation	Use This Number
Firm customer commitment	P10: Conservative Commitment (Sales tab)
Internal S&OP planning	P50: Most Likely (Sales tab)
Stretch target for operations	P75-P90 range (use Aggressive planning style)
Daily ops check	Capacity Overview KPI strip: Plan vs. Typical
Improvement prioritisation	Efficiency Gap Analysis: Loss Waterfall chart

Situation	Use This Number
Week-by-week shipping forecast	Capacity Overview: Weekly Output Breakdown chart

Container Quick-Convert

Container	Size	To Convert to Gallons	Example
Quart (Qt)	0.25 gal	Divide units by 4	100 Qt = 25 gal
Gallon (Gal)	1.00 gal	No conversion needed	100 Gal = 100 gal
Pail (5-Gal)	5.00 gal	Multiply units by 5	100 Pails = 500 gal

If Something Looks Wrong

- Numbers are all zero: Run the What-If Wizard first and click Run Plan.
- Tool will not load: Ask your system administrator to run the data pipeline (99_run_all.py). Check stage status first by running: `python pipeline_health.py`
- Plan is always red (high stress): Switch to Conservative planning style, or reduce the number of days in your planning window.
- Capacity seems too high or low: Check the effective hours in your shift schedule (Step 3) and downtime assumptions (Step 4).
- Container mix looks wrong for sales: The mix is estimated from history. If this week's SKU plan differs significantly, flag it to your scheduler.

Lisette Kamper-Hinson

lisette.hinson@gmail.com | kamplm24@wfu.edu

EDUCATION

Wake Forest University

May 2026

M.S., Computer Science: Data Science Certificate

Thesis: From Legacy to Modern: A Framework for Diagnosing and Modernizing Data Pipelines for a Data Driven Workforce

University of Mississippi

May 2024

B.S., Computer Science: Data Science Emphasis | Minor in Mathematics

Chancellor's Honor Roll: 2020, 2021, 2022, 2023, 2024

CIS Outstanding Student Award, 2022

RESEARCH EXPERIENCE

Data Science Researcher

Wake Forest University | 2025–Present

- Developing a structured six-stage framework for modernizing legacy operational data pipelines in manufacturing environments, formalizing domain alignment, structural governance, reproducibility, and decision-aligned visualization as integrated system design decisions.
- Integrated Holt’s exponential smoothing and Newsvendor-based optimization methods to model production capacity uncertainty within a real-world manufacturing case study using read-only operational data.
- Applied uncertainty quantification and Monte Carlo simulation to evaluate scheduling and resource allocation trade-offs, improving pipeline maintainability, interpretability, and scalability.
- Drove capacity utilization from 70% to 90% through systematic bottleneck analysis and governed pipeline modernization.
- Delivered decision-aligned Power BI dashboards and stakeholder-facing tools used daily by operators and senior leadership at an external industry partner.

Data Science Intern

NASA Goddard Earth Sciences Data & Information Services Center (GES-DISC) | Summer 2023

- Designed and implemented a Neo4j knowledge graph and Weaviate vector database to enhance discoverability of over 1,500 structured and unstructured satellite datasets across NASA Earth science archives.
- Extracted and processed metadata from JSON files sourced from NASA GES-DISC datasets to construct a knowledge graph serving as a foundation for search engines and machine learning training tools.
- Integrated Hugging Face BERT transformer models via API, applying NLP and semantic similarity techniques to improve contextual dataset retrieval at scale.
- Developed a Python-based RESTful query interface enabling programmatic access to complex multi-format scientific datasets for non-technical research teams.
- Containerized and deployed platform components using Docker, managing version control via GitHub across the full project lifecycle.

Undergraduate Research Assistant

University of Mississippi | 2023–2024

- Conducted research on human detection of AI-generated deepfake text as a collaborative effort between the Computer Science and Writing Departments.
- Designed and deployed a Qualtrics survey to collect data on AI usage and literacy across the university community, managing distribution, follow-up reminders, and response tracking.

- Generated AI-created and human-written text samples, designed and executed controlled experiments to test detection capabilities, and applied statistical modeling and hypothesis testing to analyze results.
- Produced institutional-level reports and policy recommendations for university leadership based on behavioral dataset analysis.

PUBLICATIONS

Kamper-Hinson, L., Rezaiyan-Nojani, A., & Mehrabian, A. (2023). Enhancing Data and Knowledge Discoverability at NASA GES-DISC through Knowledge Graphs and Language Models. *AGU Fall Meeting Abstracts*, 2023(411), IN54A-01.

CONFERENCE PRESENTATIONS

American Geophysical Union (AGU) Fall Meeting

San Francisco, CA | December 2023

Enhancing Data and Knowledge Discoverability at NASA GES-DISC through Knowledge Graphs and Language Models

Presented to an international audience of scientific professionals, showcasing the impact of knowledge graph and vector database integration on improving NASA dataset search capabilities.

PROFESSIONAL EXPERIENCE

Fraud Analytics Intern

Regions Bank | Summer 2024

- Built 6 Power BI dashboards using DAX and PowerQuery connected to SQL Server with embedded Python analytical scripts, serving as primary BI developer for the fraud analytics team.
- Conducted end-to-end data science workflows including data collection, cleaning, and analysis using error matrices, regression analysis, and fraud activity aggregation across time periods.
- Identified \$2M+ in recoverable fraud losses through rule optimization modeling; compiled findings into audit-ready reports presented directly to risk and compliance leadership.
- Collaborated in an Agile environment, participating in twice-weekly SCRUM meetings to track progress and ensure project alignment with team goals.
- Built automated SQL Server to Power BI reporting pipelines, eliminating manual workflows and delivering recurring performance insights across product lines and customer segments.

TEACHING EXPERIENCE

Teaching Assistant: CSC 111 & 112: Introduction to Computer Science

Wake Forest University | 2024

- Assisted students in introductory Python programming coursework, held office hours, and supported lab sessions under faculty supervision.

Teaching Assistant: Java Programming I, II & III

University of Mississippi | 2021–2022

- Supported instruction across three levels of Java programming, holding weekly office hours, grading assignments, and assisting students with debugging and programming concepts.

HONORS, AWARDS & FELLOWSHIPS

- 2023 Mississippi Space Grant Recipient
- CIS Outstanding Student Award, University of Mississippi: 2022
- Chancellor's Honor Roll, University of Mississippi: 2020, 2021, 2022, 2023, 2024

HONOR SOCIETIES & SERVICE

- Vice President, Upsilon Pi Epsilon Honor Society (Computer Science), University of Mississippi Chapter: 2024
- Member, Tau Beta Pi Engineering Honor Society
- Member, Phi Kappa Phi Honor Society
- Member, Pi Beta Phi Fraternity